

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” 2026 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного
забезпечення інформаційних систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Програмне забезпечення онлайн платформи мініігор з
підтримкою мультиплеерної взаємодії та системи віртуальних
активів. Програмне забезпечення підсистеми мініігор з
підтримкою мультиплеерної взаємодії

Виконав студент IV курсу, групи ІП-22
(шифр групи)

Хиля Іван Вадимович
(прізвище, ім'я, по батькові) (підпис)

Керівник професор , д.т.н., проф., засл.діяч, Павлов О.А.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Консультант ст.викл., д-р філософії, Головченко М.М.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Рецензент доцент кафедри ІСТ, к.т.н., доц., Солдатова М. О.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) (підпис)

Засвідчую, що у цьому дипломному проєкті
немає запозичень з праць інших авторів без
відповідних посилань.

Студент _____
(підпис)

Національний технічний університет України
 «Київський політехнічний інститут імені Ігоря Сікорського»
 Факультет інформатики та обчислювальної техніки
 Кафедра інформатики та програмної інженерії
 Рівень вищої освіти – перший (бакалаврський)
 Спеціальність – 121 Інженерія програмного забезпечення
 Освітньо-професійна програма – Інженерія програмного забезпечення
 інформаційних систем

ЗАТВЕРДЖУЮ
 Завідувач кафедри
 _____ Едуард ЖАРИКОВ
 (підпис) (ім'я прізвище)
 “ ” _____ 2026 р.

ЗАВДАННЯ на дипломний проєкт студенту

Хилі Івану Вадимовичу
 (прізвище, ім'я, по батькові)

1. Тема проєкту Програмне забезпечення онлайн платформи мініігор підтримкою мультиплеєрної взаємодії та системи віртуальних активів. Програмне забезпечення підсистеми мініігор з підтримкою мультиплеєрної взаємодії
- керівник проєкту Павлов Олександр Анатолійович, д.т.н., засл. діяч, професор
 (прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «03» червня 2026 р. №2055-с

2. Термін подання студентом проєкту «13» червня 2026 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Передпроєктне обстеження предметної області: основні поняття, визначення та терміни; постановка завдання дипломного проєктування; аналіз та моделювання бізнес-процесів предметної області.

2) Розроблення вимог до програмного забезпечення: визначення варіантів використання програмного забезпечення; розроблення функціональних та нефункціональних вимог; постановка завдання на розробку програмного забезпечення.

3) Конструювання та розроблення програмного забезпечення: опис архітектури програмного забезпечення; обґрунтування архітектурних рішень та вибору засобів розробки; конструювання програмного забезпечення.

4) Аналіз якості та тестування програмного забезпечення: опис процесів тестування програмного забезпечення; опис контрольного прикладу.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використань

2) Схема структурна компонентів програмного забезпечення

3) Схема бізнес-процесу проходження мініігри

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «15» березня 2026 року

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	14.03.2026	
2	Аналіз існуючих методів розв'язання задачі	21.03.2026	
3	Постановка та формалізація задачі	03.04.2026	
4	Розробка інформаційного забезпечення	19.04.2026	
5	Алгоритмізація задачі	21.04.2026	
6	Обґрунтування вибору використаних технічних засобів	25.04.2026	
7	Розробка програмного забезпечення	13.05.2026	
8	Налагодження програми	23.05.2026	
9	Виконання графічних документів	24.05.2026	
10	Оформлення пояснювальної записки	28.05.2026	
11	Подання ДП на попередній захист	01.06.2026	
12	Подання ДП рецензенту	07.06.2026	
13	Подання ДП на основний захист	13.06.2026	

Студент

(підпис)

Іван ХИЛЯ

(ініціали, прізвище)

Керівник

(підпис)

Олександр ПАВЛОВ

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка є індивідуальною частиною комплексного дипломного проєкту та містить 37 таблиць, 14 рисунків та 16 джерел – загалом 68 сторінки. У межах цієї частини спроектовано та реалізовано функціональність онлайн-платформи мініігор Quizanchos, що охоплює виконання ігрової логіки одиночних та мультиплеєрних мініігор, обіг віртуальних активів у внутрішньому маркеті, а також керування обліковими записами користувачів разом із рейтинговою системою. Метою індивідуальної частини є забезпечення стабільної та ефективної роботи зазначеної функціональності платформи, надійної взаємодії між її компонентами та коректності операцій з даними користувачів і внутрішньою валютою.

У першому розділі виконано постановку завдання дипломного проєктування та було змодельовано основні бізнес-процеси платформи.

У другому розділі сформульовано варіанти використання, функціональні та нефункціональні вимоги до реалізованого функціоналу.

У третьому розділі спроектовано архітектуру за моделлю C4 та обґрунтовано стек ASP.NET Core 10, Entity Framework Core, PostgreSQL і SignalR; детально розкрито алгоритми проходження мініігри на узагальненому GameEngine, життєвого циклу мультиплеєрної кімнати, транзакційної купівлі активу та щомісячного оновлення рейтингу, описано відповідну частину структури бази даних і проаналізовано безпеку даних.

У четвертому розділі розроблено модульні тести з використанням xUnit.net, проведено мануальне тестування основних користувацьких сценаріїв за підготовленими тест-кейсами, а також продемонстровано роботу системи на контрольному прикладі.

КЛЮЧОВІ СЛОВА: ОНЛАЙН-ПЛАТФОРМА, МІНІІГРИ, МУЛЬТИПЛЕЄР, ASP.NET CORE, SIGNALR, ВІРТУАЛЬНІ АКТИВИ, РЕЙТИНГ, АВТЕНТИФІКАЦІЯ, POSTGRESQL, REAL-TIME, .NET.

ABSTRACT

The explanatory note is an individual part of the comprehensive diploma project and contains 37 tables, 14 figures, and 16 references, comprising a total of 68 pages. Within this individual part, the functionality of the Quizanchos online mini-game platform was designed and implemented, covering the execution of game logic for both single-player and multiplayer mini-games, circulation of virtual assets within the internal marketplace, as well as user account management together with the ranking system. The objective of the individual part is to ensure stable and efficient operation of the specified platform functionality, reliable interaction between its components, and correctness of operations involving user data and virtual currency.

The first chapter presents the problem statement of the diploma project and models the main business processes of the platform.

The second chapter formulates use cases together with functional and non-functional requirements for the implemented functionality.

The third chapter describes the designed architecture based on the C4 model and justifies the selection of the ASP.NET Core 10, Entity Framework Core, PostgreSQL, and SignalR technology stack; provides a detailed description of algorithms for mini-game execution based on a generalized GameEngine, multiplayer room lifecycle management, transactional asset purchasing, and monthly ranking updates; describes the corresponding part of the database structure and analyzes data security aspects.

The fourth chapter presents the developed unit tests using xUnit.net, manual testing of key user scenarios – registration and authentication, completion of single-player and multiplayer sessions, asset purchases within the marketplace, and ranking updates – according to predefined test cases, and demonstrates system operation using a control example.

KEYWORDS: ONLINE PLATFORM, MINI-GAMES, MULTIPLAYER, ASP.NET CORE, SIGNALR, VIRTUAL ASSETS, RANKING, AUTHENTICATION, POSTGRESQL, REAL-TIME, .NET.

Пояснювальна записка до дипломного проєкту

на тему: Програмне забезпечення онлайн платформи мініігор підтримкою мультиплеєрної взаємодії та системи віртуальних активів. Програмне забезпечення підсистеми мініігор з підтримкою мультиплеєрної взаємодії.

КПІ.ПІ-2227.045440.02.81

ЗМІСТ

ВСТУП	4
1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	6
1.1 Постановка завдання дипломного проєктування	6
1.2 Аналіз та моделювання бізнес-процесів.....	7
Висновки до розділу	10
2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	12
2.1 Варіанти використання програмного забезпечення.....	12
2.2 Розроблення функціональних вимог	21
2.3 Розроблення нефункціональних вимог.....	27
2.4 Постановка завдання на розробку програмного забезпечення	29
Висновки до розділу	30
3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	32
3.1 Архітектура програмного забезпечення.....	32
3.2 Архітектурні рішення та обґрунтування вибору засобів розробки	34
3.3 Конструювання програмного забезпечення.....	36
Висновки до розділу	39
4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	41
4.1 Опис процесів тестування.....	41
4.2 Опис контрольного прикладу	53
4.3 Висновки до розділу	59
ВИСНОВКИ.....	61
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	63
ДОДАТОК А ЗВІТ ПОДІБНОСТІ.....	65

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated development environment
API	– Application programming interface
SDK	– Software development kit
IT	– Інформаційні технології
OC	– Операційна система
БД	– База даних
SQL	– Structured query language
MVC	– Model-View-Controller
DI	– Dependency injection
UC	– Use Case
HTTP	– Протокол для обміну даними в Інтернеті
HTTPS	– HTTP з шифруванням для безпечного обміну даними
URL	– Адреса для доступу до ресурсів в Інтернеті
EF	– Entity Framework
TLS	– Transport Layer Security

ВСТУП

Ігрова індустрія дедалі активніше зміщується в бік онлайн-сервісів, де користувачі грають не наодинці, а разом з іншими людьми. Серед таких сервісів окрему нішу займають платформи мініігор – невеликих за тривалістю партій ігор, до яких можна швидко долучитися без тривалого навчання чи потужного обладнання. Саме доступність і простота входу роблять цей формат привабливим для широкої аудиторії, а підтримка спільної гри, рейтингів і внутрішньої економіки перетворює окремі мініігри на цілісне розважальне середовище, здатне утримувати користувачів тривалий час.

Платформа підтримує як одиночні мініігри, так і ігри з кількома учасниками, проте саме мультиплеєрна взаємодія в реальному часі є технічно найскладнішою її частиною. Якщо в одиночному режимі сервер обслуговує лише одного гравця, то в багатокористувацькому йому доводиться узгоджувати дії одразу кількох учасників: формувати ігрові кімнати, розподіляти гравців по командах, відстежувати приєднання та вихід учасників, запускати партію за певних умов і коректно завершувати сесію. Кожна неузгодженість стану, наприклад запуск гри з неповним складом або втрата ходу гравця, одразу псує враження від сервісу, тому до надійності й послідовності серверної логіки висувуються підвищені вимоги. Додаткову складність створює те, що різні мініігри мають власні правила, а отже, платформа повинна виконувати спільну логіку сесії й водночас передавати керування конкретній реалізації обраної гри.

Другою важливою складовою сучасних ігрових платформ є система віртуальних активів. Внутрішня валюта, предмети, преміум-можливості та персоналізація профілю формують економічну модель сервісу, яка підвищує залученість гравців і стимулює їхню активність. Реалізація такої моделі вимагає особливої уваги до цілісності операцій: під час купівлі сервер має коректно списати кошти, зафіксувати володіння предметом, не допустити повторного придбання вже наявного товару та правильно продовжити термін дії підписки. Помилки в цих операціях безпосередньо зачіпають баланс

користувачів, тому вони мають виконуватися узгоджено й передбачувано навіть за одночасних звернень[14].

Окрім ігрового процесу та економіки, повноцінна платформа потребує підсистем автентифікації, обліку користувачів і рейтингування. Вони відповідають за безпечний вхід у систему, керування профілями та правами доступу, а також за формування таблиці лідерів і нарахування очок за результатами партій, що підтримує змагальну складову сервісу та дає гравцям зрозумілий зворотний зв'язок про їхні досягнення. Поєднання всіх цих підсистем в одному програмному продукті ставить питання масштабованості та зручності супроводу: щоб платформу можна було розширювати новими іграми й функціями без переписування наявного коду, взаємодію між модулями доцільно будувати на чітко визначених інтерфейсах і слабкій зв'язності компонентів[1]. З огляду на це, розроблення стабільного, безпечного та ефективного серверного програмного забезпечення для онлайн-платформи мініігор є актуальним практичним завданням.

Метою роботи є створення серверних сервісів онлайн-платформи мініігор, які забезпечують узгоджену взаємодію між модулями автентифікації, мультиплеєрних кімнат, рейтингування та внутрішньоігрового маркету й гарантують стабільну роботу системи під навантаженням. Для досягнення цієї мети проаналізовано предметну область і наявні програмні рішення, визначено ключові бізнес-процеси та сформовано функціональні й нефункціональні вимоги до серверної частини. На цій основі спроектовано архітектуру серверних сервісів і обґрунтовано вибір технологій розробки, реалізовано механізм мультиплеєрної взаємодії в реальному часі та виконання ігрової логіки мініігор, систему рейтингування й прозорий обіг віртуальних активів, а також централізований облік користувачів і контроль доступу. Завершальним етапом стали тестування й оцінювання якості розробленого програмного забезпечення.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Постановка завдання дипломного проєктування

У межах комплексного дипломного проєкту передбачено дослідження предметної області, проєктування та розроблення програмного забезпечення, а також аналіз його ефективності, безпеки та якості. Виконання проєкту поділено на спільну частину, що охоплює загальні аспекти розробки системи, та індивідуальну частину, спрямовану на проєктування, реалізацію й тестування окремого функціоналу програмного продукту.

У межах спільної частини дипломного проєкту було виконано:

- проведення аналізу предметної області, визначення її ключових бізнес-процесів та формування загальної постановки завдання розробки;
- аналіз існуючих програмних продуктів, алгоритмічних і технічних рішень, що реалізують подібні функції;
- формування системних вимог до програмного забезпечення;
- проведення економічного аналізу розроблюваного програмного продукту;
- забезпечення інформаційної безпеки та захисту даних;
- розгортання програмного забезпечення та організація його супроводу.

В індивідуальній частині дипломного проєкту поставлено такі задачі:

- моделювання та оптимізація бізнес-процесів предметної області у частині, що стосується реалізованого функціоналу;
- формування функціональних та нефункціональних вимог до реалізованого функціоналу програмного забезпечення;
- постановка технічного завдання на розробку відповідної частини програмного забезпечення;
- проєктування архітектури реалізованої частини програмного забезпечення;

- обґрунтування вибору архітектурних рішень, технологій та засобів розробки;
- реалізація програмного забезпечення відповідно до сформованих вимог;
- проведення тестування та оцінювання якості реалізованого програмного продукту;
- розроблення супровідної та технічної документації індивідуальної частини.

1.2 Аналіз та моделювання бізнес-процесів

Для глибшого розуміння логіки роботи серверної частини та формування коректних вимог до неї необхідно проаналізувати ключові бізнес-процеси, які реалізує платформа в межах індивідуальної частини. Моделювання таких процесів дає змогу наочно показати послідовність дій користувача й системи, виявити умови переходів між станами та визначити точки, у яких сервер повинен забезпечувати узгодженість і цілісність даних. Це створює основу для подальшого проєктування архітектури та розподілу відповідальності між окремими сервісами.

Опис бізнес-процесу проходження мініігри було наведено у плакаті №1 графічних матеріалів.

Опис послідовності проходження мініігри користувачем:

- користувач відкриває мінігру та надсилає запит на приєднання до ігрової сесії;
- ігровий сервер ініціалізує сесію та перевіряє достатню кількість гравців для початку гри;
- у разі недостатньої кількості гравців система очікує підключення нових учасників протягом визначеного часу;
- після досягнення необхідної кількості гравців сервер надсилає сигнал про початок гри;

- користувач отримує початковий стан гри та виконує ігрові дії (хід);
- кожен хід користувача передається на сервер, де обробляється та оновлюється стан гри;
- оновлений стан гри надсилається всім учасникам сесії для синхронізації;
- процес виконання ходів та оновлення стану гри повторюється до моменту завершення гри;
- після завершення гри сервер обчислює результати та визначає очки кожного гравця;
- на основі отриманих результатів оновлюється рейтинг користувачів;
- фінальні результати надсилаються користувачам, після чого вони можуть переглянути підсумки гри;
- процес завершується після відображення результатів усім учасникам сесії.

Для опису бізнес-процесів обліку переможців та купівлі активу використано BPMN-моделі (рисунки 1.1-1.2).

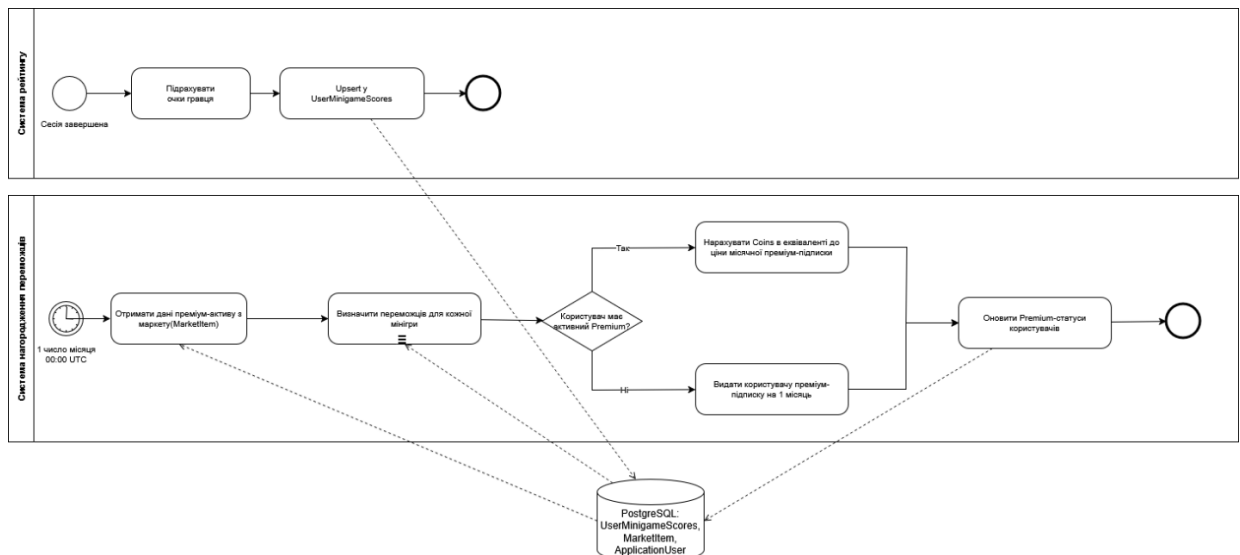


Рисунок 1.1 – Схема бізнес-процесу обліку переможців

Опис послідовності вибору переможців на основі рейтингу:

- після завершення кожної ігрової сесії система підраховує очки гравця за результатами партії та зберігає агрегований результат у

таблиці UserMinigameScores: наявний запис гравця у відповідній мінігрі оновлюється, а за відсутності створюється новий;

- першого числа кожного місяця ініціюється процедура нагородження, у межах якої з таблиці MarketItem визначається поточна вартість місячної преміум-підписки, що використовується як базовий розмір нагороди;

- для кожної зареєстрованої мініігри визначається переможець, тобто користувач із найвищим значенням рейтингу у відповідній категорії;

- для кожного переможця перевіряється наявність активної преміум-підписки: у разі її наявності на баланс користувача нараховуються монети в еквіваленті вартості місячної преміум-підписки, у протилежному випадку користувачеві надається преміум-підписка терміном на один місяць;

- після обробки всіх категорій виконується оновлення статусів користувачів відповідно до актуального терміну дії преміум-підписки, після чого процес завершується.

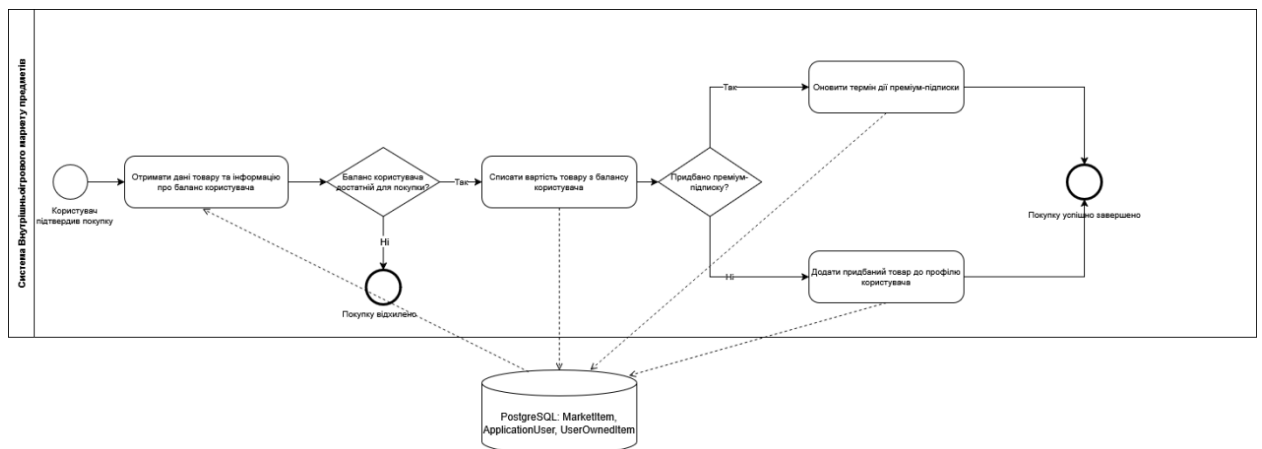


Рисунок 1.2 – Схема бізнес-процесу купівлі активу на внутрішньоігровому маркеті

Опис послідовності купівлі активу на маркеті:

- після підтвердження користувачем покупки система звертається до бази даних і отримує актуальні дані обраного товару (MarketItem) та інформацію про баланс користувача (ApplicationUser);

- виконується перевірка балансу користувача на достатність для покупки обраного товару: у разі недостатності коштів процес завершується відмовою без жодних змін у даних користувача;
- за умови достатності балансу із поточної кількості внутрішньоігрової валюти користувача списується вартість товару;
- визначається тип придбаного активу: якщо це преміум-підписка, користувачеві подовжується термін її дії на відповідну кількість місяців (`PremiumUntilUtc`); якщо це звичайний актив, він додається до профілю користувача у вигляді запису `UserOwnedItem`;
- після успішного збереження змін у базі даних процес завершується підтвердженням успішно виконаної покупки.

Висновки до розділу

У першому розділі визначено межі дипломного проєктування та виконано моделювання бізнес-процесів предметної області, що формують основу для подальшого розроблення програмного забезпечення.

У підрозділі 1.1 виконано постановку завдання дипломного проєктування у межах комплексного проєкту, у результаті чого роботу поділено на спільну та індивідуальну частини. До спільної частини віднесено загальні аспекти розробки системи – аналіз предметної області, дослідження аналогів, формування системних вимог, економічний аналіз, забезпечення інформаційної безпеки та розгортання програмного забезпечення з організацією супроводу. До індивідуальної частини віднесено задачі, безпосередньо пов'язані з реалізованим автором функціоналом – моделювання відповідних бізнес-процесів, формування функціональних і нефункціональних вимог, постановку технічного завдання, проєктування архітектури з обґрунтуванням обраних рішень, реалізацію програмного забезпечення, тестування та підготовку супровідної документації. Такий поділ забезпечує чітке розмежування зон відповідальності виконавців та однозначне визначення обсягу робіт індивідуальної частини.

У підрозділі 1.2 проведено аналіз та моделювання ключових бізнес-процесів предметної області у частині, що стосується реалізованого функціоналу, із застосуванням нотації BPMN. Розроблено три моделі: схема проходження мініігри, що описує життєвий цикл ігрової сесії від ініціалізації до синхронізації стану між учасниками в реальному часі та оновлення рейтингу за результатами партії; схема обліку переможців, що формалізує агрегацію рейтингу після кожної сесії та щомісячну процедуру нагородження переможця кожної категорії преміум-підпискою або еквівалентом внутрішньоігрової валюти; схема купівлі активу на внутрішньоігровому маркеті, що визначає послідовність операцій від підтвердження покупки до подовження преміум-підписки або додавання звичайного активу до профілю користувача. Розроблені моделі забезпечують однозначне розуміння логіки відповідної частини платформи та слугують основою для формування вимог і проєктування архітектури у наступних розділах роботи.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Головною функцією програмного забезпечення є проведення мультиплеєрних сесій мініігор між користувачами у режимі реального часу, а також підтримка системи віртуальних активів, що дозволяє придбавати й використовувати додаткові ігрові предмети. Всі функції програмного забезпечення продемонстровані у графічних матеріалах “Схема структурна варіантів використання”.

В таблицях 2.1-2.14 наведено опис варіантів використання програмного забезпечення.

Таблиця 2.1 – Варіант використання UC-01

Use case name	Реєстрація користувача
Use case ID	UC-01
Goals	Створення нового облікового запису на платформі
Actors	Гість (неzareєстрований користувач)
Trigger	Користувач бажає створити власний акаунт для отримання повного доступу до функціоналу платформи
Pre-conditions	—
Flow of Events	Користувач переходить на сторінку реєстрації. У відповідні поля вводиться електронна пошта, пароль та його повтор для підтвердження, а також встановлюється відмітка про прийняття умов сервісу. Після заповнення даних користувач натискає кнопку реєстрації. Система валідує введені дані, перевіряє унікальність електронної пошти, створює новий обліковий запис і перенаправляє користувача на сторінку входу.
Extension	У разі введення некоректних даних кнопка реєстрації стає неактивною, а конкретне поле з помилкою підсвічується відповідним повідомленням. Якщо обліковий запис із такою електронною поштою вже існує, відображається повідомлення про неможливість повторної реєстрації.

Продовження таблиці 2.1

Post-Condition	Створено новий обліковий запис, користувача перенаправлено на сторінку входу.
----------------	---

Таблиця 2.2 – Варіант використання UC-02

Use case name	Авторизація на платформі
Use case ID	UC-02
Goals	Отримання або завершення сесії користувача в системі
Actors	Гість, Авторизований користувач
Trigger	Користувач бажає увійти в обліковий запис або вийти із системи
Pre-conditions	Для входу за допомогою email/пароля користувач має зареєстрований обліковий запис
Flow of Events	Користувач переходить на сторінку входу та обирає спосіб автентифікації – за допомогою електронної пошти і пароля або через зовнішнього провайдера Google. У разі автентифікації через Google виконується перехід до сторінки провайдера, де користувач підтверджує надання дозволу на використання даних облікового запису. Після успішної автентифікації система створює сесію та надає доступ до повного функціоналу платформи. Авторизований користувач у будь-який момент може ініціювати вихід з акаунта, що завершує поточну сесію.
Extension	У разі введення невірних облікових даних, відмови провайдера Google або заблокованого облікового запису користувач отримує відповідне повідомлення про помилку.
Post-Condition	Користувач має активну сесію та доступ до повного функціоналу платформи, або сесія завершена та користувач переходить до стану «Гість».

Таблиця 2.3 – Варіант використання UC-03

Use case name	Перегляд каталогу мініігор
Use case ID	UC-03
Goals	Ознайомлення з переліком доступних на платформі мініігор
Actors	Гість, Авторизований користувач
Trigger	Користувач бажає побачити, які мініігри доступні на платформі

Продовження таблиці 2.3

Pre-conditions	—
Flow of Events	Користувач переходить у розділ каталогу мініігор. Система відображає перелік опублікованих мініігор з основною інформацією про кожну: назва, опис, категорія, ознака преміум-доступу, мініатюра. Користувач має можливість фільтрувати, сортувати та шукати мініігри за заданими критеріями, а також переглядати детальну інформацію про конкретну з них.
Extension	Якщо за обраними фільтрами не знайдено жодної мініігри або каталог порожній, відображається відповідне інформаційне повідомлення.
Post-Condition	Користувач ознайомився з каталогом і має можливість обрати конкретну мініігру для подальшої взаємодії.

Таблиця 2.4 – Варіант використання UC-04

Use case name	Перегляд рейтингу гравців
Use case ID	UC-04
Goals	Ознайомлення з рейтинговою таблицею гравців платформи
Actors	Гість, Авторизований користувач
Trigger	Користувач переходить на сторінку рейтингу
Pre-conditions	—
Flow of Events	Система відображає таблицю лідерів за загальним рейтингом гравців платформи з можливістю фільтрації за конкретною мініігрою чи періодом. Авторизованому користувачу додатково відображається його поточна позиція у рейтингу та різниця в балах до сусідніх позицій.
Extension	Якщо рейтинг ще не сформовано або за обраними фільтрами немає результатів, відображається відповідне інформаційне повідомлення.
Post-Condition	Користувач переглянув рейтингову таблицю та свою позицію в ній, стан системи не змінився.

Таблиця 2.5 – Варіант використання UC-05

Use case name	Редагування профілю
Use case ID	UC-05

Продовження таблиці 2.5

Goals	Перегляд та зміна персональних даних власного облікового запису
Actors	Авторизований користувач
Trigger	Користувач бажає оновити інформацію про себе або змінити зображення аватара
Pre-conditions	Користувач авторизований у системі
Flow of Events	Користувач переходить на сторінку власного профілю. Система відображає поточні дані користувача та дозволяє редагувати ім'я, аватар та інші персональні налаштування. Після внесення змін користувач підтверджує їх збереження, а система оновлює відповідні записи в базі даних.
Extension	У разі введення некоректних даних або помилки завантаження аватара система відхиляє зміни та виводить повідомлення про помилку.
Post-Condition	Зміни в профілі збережено, оновлені дані доступні користувачу.

Таблиця 2.6 – Варіант використання UC-06

Use case name	Налаштування і проходження мініігри в одиночному режимі
Use case ID	UC-06
Goals	Налаштування параметрів ігрової сесії та проходження обраної мініігри в одиночному режимі
Actors	Авторизований користувач
Trigger	Користувач обрав мінігру з каталогу та натиснув кнопку запуску в одиночному режимі
Pre-conditions	Користувач авторизований у системі; мінігра завантажена розробником і опублікована для гравців
Flow of Events	Користувач задає параметри сесії (рівень складності, тривалість, кількість раундів та інші специфічні параметри мініігри). Після підтвердження система ініціює одиночну ігрову сесію та забезпечує її проходження. Після завершення сесії користувачу відображаються результати проходження з нарахованими очками, а його статистика та позиція в рейтингу оновлюються.

Продовження таблиці 2.6

Extension	У разі помилки завантаження пакету мініігри відображається повідомлення про неможливість запуску. У разі обриву з'єднання сесія зберігається та залишається доступною для повернення (UC-08).
Post-Condition	Ігрова сесія успішно завершена, результати збережено, рейтинг гравця оновлено, відповідну кількість внутрішньоігрової валюти нараховано на баланс (за наявності).

Таблиця 2.7 – Варіант використання UC-07

Use case name	Налаштування і проходження мініігри в мультиплеєрному режимі
Use case ID	UC-07
Goals	Налаштування параметрів мультиплеєрної сесії, приєднання до інших гравців та проходження обраної мініігри в режимі реального часу
Actors	Авторизований користувач
Trigger	Користувач обрав мініігру з каталогу та натиснув кнопку запуску в мультиплеєрному режимі
Pre-conditions	Користувач авторизований у системі; мінігра підтримує мультиплеєрний режим; наявне стабільне з'єднання з сервером
Flow of Events	Користувач задає параметри сесії (максимальна кількість гравців, час очікування, тривалість ходу та інші специфічні параметри) та створює нову сесію або приєднується до існуючої доступної. Система здійснює пошук учасників і після досягнення необхідної кількості гравців ініціює гру. Під час сесії стан гри синхронізується між усіма учасниками в режимі реального часу через канал на основі веб-сокетів. Після завершення гри користувачу відображаються результати усіх учасників, а рейтинги оновлюються відповідно до отриманих балів.
Extension	У разі недостатньої кількості гравців упродовж заданого часу сесія скасовується. У разі обриву з'єднання користувача сесія зберігається й залишається доступною для повернення (UC-08); інші учасники продовжують гру або очікують повернення відповідно до налаштувань сесії.

Продовження таблиці 2.7

Post-Condition	Мультиплеєрна сесія успішно завершена, результати усіх гравців збережено, рейтинги оновлено.
----------------	--

Таблиця 2.8 – Варіант використання UC-08

Use case name	Управління активними ігровими сесіями
Use case ID	UC-08
Goals	Перегляд переліку незавершених ігрових сесій та повернення до однієї з них
Actors	Авторизований користувач
Trigger	Користувач бажає переглянути перелік активних сесій або продовжити перервану гру
Pre-conditions	Користувач авторизований у системі та має принаймні одну незавершену ігрову сесію
Flow of Events	Користувач переходить у розділ активних сесій, доступний у профілі. Система відображає перелік незавершених одиночних і мультиплеєрних сесій з інформацією про мінігру, режим, дату початку та поточний стан. Користувач обирає конкретну сесію та повертається до її проходження з точки переривання.
Extension	Якщо активних сесій немає, відображається відповідне інформаційне повідомлення. У разі неможливості відновлення сесії (вийшов час очікування, мультиплеєрна сесія завершена іншими учасниками) система виводить повідомлення про закриття сесії.
Post-Condition	Користувач повернувся до обраної сесії та продовжує її проходження.

Таблиця 2.9 – Варіант використання UC-09

Use case name	Перегляд внутрішньоігрового маркету
Use case ID	UC-09
Goals	Ознайомлення з асортиментом доступних у маркеті віртуальних предметів
Actors	Авторизований користувач
Trigger	Користувач бажає переглянути доступні предмети у внутрішньоігровому маркеті
Pre-conditions	Користувач авторизований у системі

Продовження таблиці 2.9

Flow of Events	Користувач переходить у розділ маркету. Система відображає каталог доступних предметів (аватари, емодзі, інші віртуальні активи) з можливістю фільтрації, сортування та перегляду деталей кожного предмета (опис, ціна у внутрішньоігровій валюті, попередній перегляд).
Extension	Якщо за обраними фільтрами не знайдено жодного предмета або маркет порожній, відображається інформаційне повідомлення.
Post-Condition	Користувач ознайомився з асортиментом маркету та може ініціювати купівлю обраного предмета.

Таблиця 2.10 – Варіант використання UC-10

Use case name	Купівля внутрішньоігрового активу
Use case ID	UC-10
Goals	Придбання обраного віртуального предмета за внутрішньоігрову валюту
Actors	Авторизований користувач
Trigger	Користувач обрав предмет у маркеті та підтвердив купівлю
Pre-conditions	Користувач авторизований у системі; предмет доступний у маркеті
Flow of Events	Користувач обирає предмет з маркету та натискає кнопку купівлі. Система перевіряє наявність достатньої кількості коштів на балансі, у транзакційному режимі списує відповідну суму внутрішньоігрової валюти та додає предмет до інвентарю користувача. Інформація про операцію автоматично фіксується в історії транзакцій (UC-13).
Extension	Якщо коштів на балансі недостатньо, система пропонує користувачу спочатку поповнити баланс (UC-12). Якщо предмет став недоступним між переглядом і купівлею, відображається відповідне повідомлення, а операція скасовується.
Post-Condition	Придбаний предмет збережено в інвентарі користувача, баланс оновлено, відповідний запис додано до історії транзакцій.

Таблиця 2.11 – Варіант використання UC-11

Use case name	Купівля преміум-підписки
---------------	--------------------------

Продовження таблиці 2.11

Use case ID	UC-11
Goals	Оформлення або продовження преміум-підписки для отримання розширеного функціоналу платформи
Actors	Авторизований користувач
Trigger	Користувач бажає придбати преміум-підписку або продовжити термін її дії
Pre-conditions	Користувач авторизований у системі
Flow of Events	Користувач переходить у розділ преміум-підписки, ознайомлюється з її перевагами та обирає тарифний план. Після підтвердження оплати через інтегрований платіжний шлюз система активує преміум-статус облікового запису та зберігає інформацію про термін дії підписки.
Extension	У разі помилки оплати користувач отримує повідомлення з можливістю повторити операцію. Якщо у користувача вже діє активна підписка, система пропонує продовження її терміну.
Post-Condition	Преміум-статус облікового запису активовано, користувач отримує доступ до розширеного функціоналу, відповідний запис додано до історії транзакцій.

Таблиця 2.12 – Варіант використання UC-12

Use case name	Поповнення балансу
Use case ID	UC-12
Goals	Збільшення балансу внутрішньоігрової валюти користувача за рахунок реальних коштів
Actors	Авторизований користувач
Trigger	Користувач бажає поповнити свій ігровий баланс
Pre-conditions	Користувач авторизований у системі
Flow of Events	Користувач переходить у розділ балансу та обирає бажану суму для поповнення. Система перенаправляє користувача до інтегрованого платіжного шлюзу для проведення оплати. Після успішної транзакції баланс користувача автоматично поповнюється на відповідну суму, а інформація про операцію зберігається в історії транзакцій.

Продовження таблиці 2.12

Extension	У разі помилки при здійсненні платежу користувач отримує повідомлення з можливістю повторити операцію. У разі підозрілої активності транзакція може бути направлена на ручне підтвердження адміністратором.
Post-Condition	Баланс користувача успішно поповнено, відповідний запис додано до історії транзакцій.

Таблиця 2.13 – Варіант використання UC-13

Use case name	Перегляд історії транзакцій
Use case ID	UC-13
Goals	Ознайомлення з історією власних фінансових операцій на платформі
Actors	Авторизований користувач
Trigger	Користувач бажає переглянути перелік власних транзакцій
Pre-conditions	Користувач авторизований у системі
Flow of Events	Користувач переходить у розділ історії транзакцій. Система відображає повний перелік фінансових операцій користувача (поповнення балансу, купівлі активів, оформлення преміум-підписки, нарахування нагород) з відповідними сумами, датами та статусами.
Extension	Якщо у користувача ще немає жодної транзакції, відображається інформаційне повідомлення.
Post-Condition	Користувач переглянув історію транзакцій, стан системи не змінився.

Таблиця 2.14 – Варіант використання UC-14

Use case name	Отримання місячної нагороди
Use case ID	UC-14
Goals	Отримання нагороди за результатами рейтингу за календарний місяць
Actors	Авторизований користувач
Trigger	Початок нового календарного місяця та факт перебування користувача в нагороджуваних позиціях рейтингу попереднього періоду
Pre-conditions	Користувач авторизований у системі та посів призове місце у місячному рейтингу

Продовження таблиці 2.14

Flow of Events	На початку нового місяця система формує перелік переможців на основі рейтингу за попередній період і розраховує відповідну нагороду для кожного. При наступному вході на платформу користувачу відображається повідомлення про отриману нагороду, а сума автоматично нараховується на його баланс внутрішньоігрової валюти.
Extension	Якщо користувач не потрапив до переможців, нагорода не нараховується і повідомлення не відображається.
Post-Condition	Нагороду нараховано на баланс користувача, відповідний запис додано до історії транзакцій.

2.2 Розроблення функціональних вимог

Програмне забезпечення індивідуальної частини платформи побудоване як набір функціонально відокремлених модулів, кожен з яких відповідає за окремий напрям роботи системи з боку гравця: облік користувачів, керування каталогом і одиночним проходженням мініігор, мультиплеєрна взаємодія в реальному часі, ведення рейтингу гравців, внутрішньоігровий маркет та управління віртуальними активами користувача. Таке розділення дозволяє виокремити вимоги до кожного компонента, оцінити їх важливість для роботи платформи в цілому та ризики, пов'язані з їх реалізацією.

Загальна модель вимог наведена в таблиці 2.15, а детальний опис кожної функціональної вимоги – у таблиці 2.16. Зв'язки між варіантами використання та функціональними вимогами відображені у матриці трасування на рисунку 2.2.

Пріоритет кожної вимоги визначався виходячи з її ролі у роботі платформи. Високий пріоритет присвоєно вимогам, без яких неможливе виконання основних сценаріїв застосунку, а саме автентифікації користувачів, проведення одиночних та мультиплеєрних сесій, синхронізації стану гри між учасниками у реальному часі, обігу внутрішньоігрової валюти та купівлі віртуальних активів у маркеті. Середній пріоритет отримали вимоги, що значно покращують користувацький досвід або розширюють функціональні

можливості платформи (перегляд статистики проходження, історії транзакцій, оформлення преміум-підписки, нарахування нагород), однак не блокують основну роботу системи в разі їх відсутності. Низький пріоритет присвоєно вимогам допоміжного характеру, які впливають переважно на зручність використання та можуть бути реалізовані поетапно після базової функціональності.

Оцінка ризиків відображає технічну складність реалізації кожної вимоги та її залежність від зовнішніх факторів. Високий ризик мають вимоги, пов'язані з мультиплеєрною взаємодією в реальному часі, інтеграцією із зовнішніми сервісами автентифікації та платіжними шлюзами, а також обробкою фінансово значущих операцій – зокрема синхронізація стану ігрових сесій, відновлення перерваних мультиплеєрних сесій, поповнення балансу через платіжний шлюз та купівля активів у маркеті. Середній ризик відповідає вимогам із помірною технічною складністю, де основна частина логіки реалізується засобами платформи, але присутні неочевидні граничні випадки, що потребують додаткового тестування[15]. Низький ризик характерний для вимог, що зводяться до типових операцій з даними в базі (CRUD-операції над сутностями користувачів, профілів, інвентарю) та не залежать від зовнішніх систем.

Таблиця 2.15 – Загальна модель вимог

№	Назва	ID Вимоги	Пріоритет	Ризики
1	Система обліку користувачів	FR-1	Високий	Високий
1.1	Реєстрація користувача	FR-2	Високий	Середній
1.2	Вхід в обліковий запис через email і пароль	FR-3	Високий	Середній
1.3	Вхід в обліковий запис через Google	FR-4	Високий	Високий
1.4	Вихід з облікового запису	FR-5	Високий	Низький
1.5	Редагування профілю та аватара	FR-6	Середній	Низький
2	Система проходження мініігор в одиночному режимі	FR-7	Високий	Середній
2.1	Перегляд каталогу мініігор	FR-8	Високий	Низький

Продовження таблиці 2.15

№	Назва	ID Вимоги	Пріоритет	Ризики
2.2	Налаштування параметрів ігрової сесії	FR-9	Високий	Середній
2.3	Запуск і проходження мініігри в одиночному режимі	FR-10	Високий	Середній
2.4	Перегляд результатів проходження	FR-11	Високий	Низький
3	Система мультиплеєрної взаємодії	FR-12	Високий	Високий
3.1	Створення та приєднання до мультиплеєрної сесії	FR-13	Високий	Високий
3.2	Синхронізація стану гри між учасниками в реальному часі	FR-14	Високий	Високий
3.3	Обробка ходів гравців у мультиплеєрному режимі	FR-15	Високий	Високий
3.4	Перегляд переліку активних ігрових сесій	FR-16	Середній	Низький
3.5	Повернення до перерваної ігрової сесії	FR-17	Високий	Високий
4	Рейтингова система	FR-18	Високий	Середній
4.1	Перегляд таблиці лідерів	FR-19	Високий	Низький
4.2	Перегляд власної позиції у рейтингу	FR-20	Середній	Низький
4.3	Оновлення рейтингу за результатами сесії	FR-21	Високий	Середній
4.4	Нарахування місячної нагороди переможцям	FR-22	Середній	Середній
5	Внутрішньоігровий маркет	FR-23	Високий	Високий
5.1	Перегляд каталогу маркету	FR-24	Високий	Низький
5.2	Купівля внутрішньоігрового активу	FR-25	Високий	Високий
6	Управління віртуальними активами користувача	FR-26	Високий	Високий
6.1	Перегляд інвентарю користувача	FR-27	Середній	Низький

Продовження таблиці 2.15

№	Назва	ID Вимоги	Пріоритет	Ризики
6.2	Поповнення балансу через платіжний шлюз	FR-28	Високий	Високий
6.3	Оформлення преміум-підписки	FR-29	Середній	Середній
6.4	Перегляд історії транзакцій	FR-30	Середній	Низький

Таблиця 2.16 – Перелік функціональних вимог

Назва	Опис
FR-1	Система обліку користувачів. Функціонал, що забезпечує реєстрацію, автентифікацію та керування обліковими записами користувачів платформи.
FR-2	Реєстрація користувача. Користувач створює новий обліковий запис, ввівши електронну пошту, пароль та підтвердження пароля.
FR-3	Вхід в обліковий запис через email і пароль. Користувач вводить пошту й пароль для отримання доступу до особистого кабінету.
FR-4	Вхід в обліковий запис через Google. Користувач здійснює автентифікацію через зовнішнього провайдера Google без потреби запам'ятовувати окремий пароль для платформи.
FR-5	Вихід з облікового запису. Користувач завершує поточну сесію та повертається до стану незареєстрованого користувача.
FR-6	Редагування профілю та аватара. Користувач переглядає й змінює дані власного профілю, а також завантажує зображення аватара.
FR-7	Система проходження мініігор в одиночному режимі. Функціонал перегляду, налаштування та запуску мініігор з каталогу для одиночного проходження.
FR-8	Перегляд каталогу мініігор. Користувач переглядає перелік доступних мініігор з можливістю фільтрації за категорією та переглядом опису.
FR-9	Налаштування параметрів ігрової сесії. Користувач задає параметри обраної мініігри: режим гри, рівень складності, кількість раундів та інші специфічні параметри.
FR-10	Запуск і проходження мініігри в одиночному режимі. Користувач запускає обрану мініігру в одиночному режимі та проходить ігрову сесію без участі інших гравців.

Продовження таблиці 2.16

Назва	Опис
FR-11	Перегляд результатів проходження. Після завершення мініігри користувач бачить підсумок проходження з нарахованими очками та позицією.
FR-12	Система мультиплеєрної взаємодії. Функціонал, що забезпечує одночасне проходження мініігор кількома користувачами в режимі реального часу із синхронізацією стану гри між усіма учасниками.
FR-13	Створення та приєднання до мультиплеєрної сесії. Користувач створює нову мультиплеєрну сесію з власними параметрами або приєднується до вже існуючої доступної сесії.
FR-14	Синхронізація стану гри між учасниками в реальному часі. Система забезпечує передачу та узгодження актуального стану ігрової сесії між усіма гравцями з мінімальними затримками.
FR-15	Обробка ходів гравців у мультиплеєрному режимі. Кожен хід учасника передається на сервер, обробляється відповідно до правил мініігри та транслюється всім гравцям сесії.
FR-16	Перегляд переліку активних ігрових сесій. Користувач переглядає перелік власних незавершених одиночних та мультиплеєрних сесій з інформацією про мінігру і дату початку.
FR-17	Повернення до перерваної ігрової сесії. Користувач може повернутися до раніше розпочатої незавершеної сесії та продовжити її проходження з місця обриву.
FR-18	Рейтингова система. Функціонал ведення рейтингу користувачів та оновлення показників за результатами проходження мініігор.
FR-19	Перегляд таблиці лідерів. Користувач переглядає загальну таблицю лідерів платформи з фільтрацією за періодом і мінігрою.
FR-20	Перегляд власної позиції у рейтингу. Користувач бачить власне місце у рейтингу та різницю в балах до сусідніх позицій.
FR-21	Оновлення рейтингу за результатами сесії. Система автоматично оновлює рейтингові показники гравця після завершення ігрової сесії відповідно до її результатів.
FR-22	Нарахування місячної нагороди переможцям. На початку нового місяця система визначає переможців за результатами

	рейтингу попереднього періоду та автоматично нараховує їм нагороду у вигляді внутрішньоігрової валюти.
FR-23	Внутрішньоігровий маркет. Функціонал маркету для перегляду та придбання внутрішньоігрових предметів користувачами.
FR-24	Перегляд каталогу маркету. Користувач переглядає доступні у маркеті предмети з можливістю фільтрації, сортування та перегляду деталей.
FR-25	Купівля внутрішньоігрового активу. Користувач придбаває обраний предмет за внутрішньоігрову валюту; предмет додається до інвентарю користувача, інформація про операцію фіксується в історії транзакцій.
FR-26	Управління віртуальними активами користувача. Функціонал керування внутрішніми активами користувача: балансом, інвентарем, преміум-підпискою та історією операцій.
FR-27	Перегляд інвентарю користувача. Користувач переглядає перелік власних придбаних активів та предметів.
FR-28	Поповнення балансу через платіжний шлюз. Користувач придбаває внутрішньоігрову валюту за реальні кошти через інтегрований платіжний шлюз; після успішної транзакції баланс автоматично поповнюється.
FR-29	Оформлення преміум-підписки. Користувач оформлює або продовжує преміум-підписку, що надає розширені можливості на платформі.
FR-30	Перегляд історії транзакцій. Користувач переглядає повну історію власних фінансових операцій з відповідними статусами.

Матриця трасування вимог, що відображає відповідність між варіантами використання UC-01-UC-14 та функціональними вимогами FR-1-FR-30, наведена на рисунку 2.1.

	FR-1	FR-2	FR-3	FR-4	FR-5	FR-6	FR-7	FR-8	FR-9	FR-10	FR-11	FR-12	FR-13	FR-14	FR-15	FR-16	FR-17	FR-18	FR-19	FR-20	FR-21	FR-22	FR-23	FR-24	FR-25	FR-26	FR-27	FR-28	FR-29	FR-30
UC-01	+	+																												
UC-02	+		+	+	+																									
UC-03							+	+																						
UC-04																		+	+	+										
UC-05	+					+																								
UC-06							+		+	+	+							+			+									
UC-07												+	+	+	+			+			+									
UC-08												+				+	+													
UC-09																							+	+						
UC-10																							+		+	+				+
UC-11																									+			+	+	
UC-12																									+		+		+	
UC-13																									+				+	
UC-14																		+				+				+				+

Рисунок 2.1 – Матриця трасування вимог

2.3 Розроблення нефункціональних вимог

Сторінки реалізованої частини проєктуються таким чином, щоб новий гравець міг пройти повний шлях від першого відкриття платформи до старту партії без сторонньої допомоги та звернення до довідки. Глибина навігаційної ієрархії від головної сторінки до будь-якого функціонального екрана, а саме профілю, рейтингу, форми відновлення пароля, історії операцій із балансом або сторінки преміум-статусу, не перевищує чотирьох рівнів, а доступ до основних дій облікового запису забезпечується з верхньої навігаційної панелі незалежно від поточної сторінки. Усі форми мають однотипне розташування полів введення, кнопок підтвердження та повідомлень про помилки, що дозволяє користувачеві орієнтуватися у нових сценаріях без додаткового навчання. Розмітка адаптується під мобільні, планшетні та десктопні пристрої, основні форми входу та реєстрації підтримують клавіатурну навігацію без використання миші, а критичні дії, такі як зміна пароля або видалення облікового запису, додатково супроводжуються підтверджувальним діалогом[2].

Безпека реалізованої частини забезпечується розмежуванням доступу за функціями та даними з визначеними ролями, причому кожен користувач отримує доступ виключно до власного профілю, балансу та персональних показників у рейтингу, а будь-яка спроба звернення до даних іншого

облікового запису блокується на серверній стороні. Робота з персональними даними здійснюється відповідно до Закону України «Про захист персональних даних» та принципів GDPR[3]: збирається мінімально необхідний обсяг даних (електронна адреса, нікнейм та опційний аватар), паролі зберігаються виключно у вигляді незворотної криптографічної функції з індивідуальною сіллю, користувачу гарантується право переглядати, редагувати та повністю видаляти власний обліковий запис із обробкою запиту на видалення не пізніше ніж за 24 години, а 100 % трафіку між клієнтом і сервером передається захищеним каналом TLS.

Для протидії зовнішнім атакам встановлюються конкретні кількісні обмеження: не більше 5 невдалих спроб входу поспіль на один обліковий запис із наступним блокуванням на 15 хвилин для запобігання brute-force[4], не більше 3 спроб введення коду відновлення пароля, обов'язкове підтвердження електронної адреси при реєстрації та повторна перевірка пошти при скиданні пароля, 100 % параметризованих звернень до бази даних для виключення SQL-ін'єкцій[5], а також передача ідентифікатора сесії лише через cookie з прапорцями, що блокують його зчитування скриптами та передачу поза захищеним каналом.

Цільові показники продуктивності реалізованої частини передбачають одночасну роботу не менше 200 авторизованих користувачів, обробку щонайменше 50 запитів автентифікації за секунду та підтримку не менше 100 одночасних переглядів сторінок рейтингу. Час відгуку запитів входу, реєстрації, відкриття профілю та сторінки рейтингу не повинен перевищувати 500 мс у 95 % випадків, а час повернення сторінки історії транзакцій із преміум-доступом 700 мс у 95 % випадків. Щомісячне фонове завдання перерахунку переможців категорій повинно завершуватися не довше ніж за 5 хвилин для бази з 10 000 активних користувачів, а відправлення листа підтвердження електронної адреси при реєстрації не пізніше ніж за 30 секунд з моменту створення облікового запису.

2.4 Постановка завдання на розробку програмного забезпечення

Завданням індивідуальної частини є розробка набору сервісів веб-платформи для проведення мініігор, що забезпечують проходження одиночних та мультиплеєрних ігрових сесій у реальному часі, керування багатокористувацькими кімнатами, обіг віртуальних активів у внутрішньоігровому маркеті, а також керування обліковими записами користувачів разом із рейтинговою системою та преміум-доступом.

Сервіс ігрової логіки відповідає за створення ігрової сесії, обробку ходів гравців, оновлення стану партії та визначення переможця, а також за збереження поточного стану сесії для можливості її відновлення. Сервіс мультиплеєрних кімнат організовує підбір гравців, формування команд, синхронізацію складу учасників та автоматичний запуск партії при заповненні кімнати. Сервіс внутрішньоігрового маркету реалізує транзакційну купівлю віртуальних активів та преміум-підписок із гарантією цілісності операцій з внутрішньою валютою. Сервіс облікових записів охоплює реєстрацію та автентифікацію (зокрема вхід через Google), керування персональним профілем, агрегування результатів у глобальний рейтинг, щомісячне нагородження переможців категорій та перевірку преміум-доступу.

Метою розробки є забезпечення стабільної та ефективної роботи переліченої функціональності платформи з мінімальною затримкою синхронізації стану ігрових сесій між учасниками, цілісністю транзакцій з віртуальною валютою у внутрішньоігровому маркеті та достовірним відображенням рейтингу гравців за категоріями мініігор.

Кінцевими користувачами розробки є гравці, які беруть участь в одиночних та мультиплеєрних мінііграх, купують активи на маркеті та переглядають власний профіль і рейтинг.

Розробка ставить перед собою досягнення наступних цілей:

- реалізація механізму проходження мініігри з обробкою ходів гравців, оновленням стану партії та визначенням переможця у режимах одиночної і мультиплеєрної гри;

- організація багатокористувацьких кімнат із підбором гравців, формуванням команд та синхронізацією стану учасників у реальному часі;
- побудова внутрішньоігрового маркету з транзакційною купівлею віртуальних активів і преміум-підписок;
- реалізація рейтингової системи з агрегацією результатів проходження мініігор та щомісячним визначенням переможців категорій з автоматичним нарахуванням винагороди;
- забезпечення механізму реєстрації, входу та централізованої перевірки преміум-доступу для решти модулів платформи.

Висновки до розділу

У другому розділі проведено комплексний аналіз вимог до індивідуальної частини програмного забезпечення веб-платформи для проведення мініігор, у результаті чого сформовано цілісне технічне завдання на її розробку.

У підрозділі 2.1 виокремлено та формалізовано чотирнадцять варіантів використання, що охоплюють усі ключові сценарії взаємодії з реалізованою частиною платформи з боку двох категорій акторів – гостя та авторизованого користувача. Описані варіанти використання охоплюють авторизацію та керування сесією користувача, керування персональним профілем, перегляд каталогу мініігор та рейтингу гравців, налаштування й проходження одиночних та мультиплеєрних ігрових сесій, придбання віртуальних активів у внутрішньоігровому маркеті, отримання преміум-винагород за результатами рейтингу. Для кожного варіанта використання визначено цілі, передумови, основний потік подій, розширення та постумови, що забезпечує однозначне розуміння очікуваної поведінки системи.

У підрозділі 2.2 на основі варіантів використання розроблено набір функціональних вимог, згрупованих у п'ять функціонально відокремлених модулів: облік користувачів, проходження мініігри, мультиплеєрні кімнати, рейтингова система та внутрішньоігровий маркет. Для кожної вимоги

визначено пріоритет та оцінку ризику реалізації, що дозволяє планувати поетапну розробку з урахуванням критичності функціоналу та технічної складності. Зв'язки між варіантами використання та функціональними вимогами представлено у вигляді матриці трасування, яка підтверджує повноту покриття сценаріїв використання функціональними вимогами.

У підрозділі 2.3 сформовано перелік нефункціональних вимог, які охоплюють зручність використання, конфігурованість на рівні параметрів, розмежування доступу за ролями та даними, відповідність законодавству у сфері захисту персональних даних, а також кількісні показники продуктивності та доступності реалізованої частини. Зокрема, визначено допустимий час відгуку основних запитів, мінімальну кількість одночасно активних користувачів та паралельних мультиплеєрних сесій, обмеження частоти невдалих спроб входу для запобігання атакам повного перебору.

У підрозділі 2.4 на основі результатів аналізу сформульовано постановку завдання на розробку програмного забезпечення, у якій визначено мету розробки, її призначення, перелік кінцевих користувачів та основні цілі, зокрема реалізацію механізму проходження мініігри, організацію мультиплеєрних кімнат, побудову рейтингової системи з автоматичним щомісячним нагородженням, реалізацію внутрішньоігрового маркету та забезпечення централізованого механізму автентифікації й перевірки преміум-доступу. Сформоване технічне завдання становить основу для подальшого моделювання та конструювання програмного забезпечення у наступних розділах роботи.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

Загальну архітектуру платформи Quizanchos (монолітний застосунок ASP.NET Core 10[6] з плагінним розширенням, контейнерну та контекстну діаграми моделі C4) було описано у спільній частині пояснювальної записки. У межах індивідуальної частини детально розкривається саме чотири сервіси: GameLogic (виконання ігрової логіки та маршрутизація викликів до плагінів мініігор), Rooms (керування життєвим циклом мультиплеєрних кімнат), Market (каталог віртуальних активів і транзакційна купівля) та Users (профілі, ролі, преміум-доступ, рейтинг та статистика).

Компонентний рівень моделі C4 для зазначених чотирьох сервісів подано у графічних матеріалах “Схема структурна компонентів програмного забезпечення”.

Сервіс GameLogic відповідає за виконання логіки конкретної мініігри в межах окремої ігрової сесії. Центральним його компонентом є GameLogicFactory, який за gameKey сесії звертається до IMinigameRegistry, отримує відповідний IMinigameDescriptor і через нього або створює новий екземпляр IGameEngine, або відновлює його зі збереженого стану. Запущений рушій загортається у типобезпечну обгортку GameEngineWrapper<TState, TMove>, яка приховує параметризацію за конкретним типом ходу TMove та надає однорідний інтерфейс IGameEngine. Збереження та оновлення стану виконує GameStatePersistence, що інкапсулює серіалізацію JSON та запис у GameStateState; саме цей компонент плагіни отримують через інтерфейс IGameStatePersistence з Quizanchos.Core і не звертаються напямучу до DbContext. Сповіщення клієнтів про зміну стану виконує SignalRGameNotifier, що транслює оновлення у групу game-{guid} хабу GameHub.

Сервіс Rooms керує життєвим циклом мультиплеєрних кімнат до моменту запуску ігрової сесії. Оркестратором є GameRoomService: він

приймає запити на створення, приєднання, зміну команди та вихід з кімнати, виконує перевірки бізнес-правил (мінімум два гравці, кількість команд, доступність преміум мініігор через PremiumAccessService) і делегує збереження стану кімнати IGameRoomManager. У поточній реалізації використовується InMemoryGameRoomManager на основі ConcurrentDictionary<Guid, GameRoom> – стан кімнат утримується в пам'яті процесу та не зберігається у базі даних, що відповідає типовому життєвому циклу лобі. Сповіщення інших учасників кімнати про події (нового гравця, готовність, запуск гри) виконує SignalRRoomNotifier через хаб GameRoomHub та групу room-{guid}. Коли кімната заповнюється, GameRoomService автоматично делегує запуск ігрової сесії сервісу GameService, який, у свою чергу, звертається до сервісу GameLogic для створення IGameEngine.

Сервіс Market відповідає за каталог віртуальних активів та операції купівлі. MarketService надає три ключові методи: GetCatalog (повертає список активів обраної категорії), GetUserOwnership (повертає те, що вже придбано користувачем) і Purchase (виконує транзакційну купівлю одного активу). Усі звернення до бази даних виконуються через інтерфейси IMarketItemRepository та IUserOwnedItemRepository доменного шару, що дозволяє підмінити реалізацію у модульних тестах.

Сервіс Users охоплює увесь набір операцій з профілями користувачів та супутніми сутностями. UserRetrieverService надає однорідний спосіб отримати ApplicationUser з ClaimsPrincipal або з ідентифікатора, UserProfileService реалізує перегляд та редагування профілю (зокрема завантаження аватарів у хмарне сховище Cloudinary[7] через офіційний .NET SDK), UserScoreService агрегує загальну кількість очок гравця. LeaderBoardService обслуговує сторінки рейтингу, а фонове завдання LeadersUpdaterJob (Quartz.NET[8], cron-розклад «перше число місяця») щомісяця перераховує таблицю leaderboard_entry за попередній період. PremiumAccessService забезпечує перевірку права доступу до преміум-мініігор за полем

ApplicationUser.PremiumUntilUtc і використовується сервісом Rooms перед створенням та приєднанням до кімнат.

3.2 Архітектурні рішення та обґрунтування вибору засобів розробки

У межах розробки сервісів Rooms, GameLogic, Market та Users прийнято низку рішень, спрямованих на одночасне виконання трьох вимог: підтримку плагінної моделі мініігор без жорсткого зв'язування з ядром, низькі затримки під час мультиплеєрних сесій та цілісність даних при операціях з внутрішньою валютою й обліковими записами. Загальні рішення щодо мови, сервера застосунків і СКБД узгоджено у спільному підрозділі 3.2; нижче розкривається лише та частина, що стосується безпосередньо цих чотирьох сервісів.

Для опису логіки кожної мініігри введено узагальнений інтерфейс `IGameLogic<TState, TMove>`, який поєднує фабрику стану, валідатор ходу та правила гри. Альтернативу – єдиний нетипізований інтерфейс із полем `object Move` – відхилено через втрату типобезпеки та необхідність ручної перевірки приведення типів усередині кожного плагіна. Узагальнений варіант, разом з обгорткою `GameEngineWrapper<TState, TMove>`, дає змогу зберегти типобезпеку всередині плагіна та водночас приховати конкретний тип ходу від хосту, який оперує лише базовим інтерфейсом `IGameEngine`.

Стан ігрової сесії плагіни не записують у власні таблиці. Натомість він проходить через інтерфейс `IGameStatePersistence` (хост-реалізація – `GameStatePersistence`), який серіалізує доменну модель у JSON та зберігає її в полі `state_json` таблиці `game_session_state`[9].

Для маршрутизації клієнтських запитів до конкретної реалізації мініігри використовується `IMinigameRegistry`, побудований під час старту застосунку із зібраного списку `IMinigameDescriptor`. Реєстр зведено до простої мапи `gameKey -> дескриптор`, що дозволяє отримати потрібну фабрику `IGameEngine` за $O(1)$ і не вимагає рефлексії під час виконання ходу. Аналогічно `IMinigameFrontendRegistry` відповідає за зіставлення `gameKey` з описом

фронтенд-частини (CSS- та JS-URL, метадані лобі) і використовується у Razor-в'ю.

Стан мультиплеєрних кімнат свідомо реалізовано як in-memory структуру (InMemoryGameRoomManager на основі ConcurrentDictionary) без запису у базу даних. Це мотивовано двома міркуваннями: по-перше, кімната як лобі-сутність живе короткий проміжок часу (хвилини), тож постійне зберігання не потрібне; по-друге, синхронізація десятків операцій join/leave через PostgreSQL[16] створювала б непотрібне навантаження.

Для двосторонньої взаємодії в реальному часі під час ігрових сесій і кімнат обрано SignalR[10] з транспортом WebSocket. Розглянуто також періодичне опитування (HTTP polling) та Server-Sent Events: перший підхід дає високу затримку та надмірне навантаження на сервер, другий не підтримує двосторонню взаємодію без додаткових REST-запитів. SignalR натомість інтегрований з ASP.NET Core Identity і дозволяє ідентифікувати учасника кімнати безпосередньо в обробнику без додаткових перевірок. Для розділення подій ігрових сесій та подій кімнат використано два окремі хаби – GameHub (/hubs/game) і GameRoomHub (/hubs/room), а групи названо за схемою game-{guid} та room-{guid}, що спрощує трансляцію подій лише серед учасників конкретної сесії або кімнати.

Сервіс Market виконує операцію купівлі активу як єдину транзакцію Entity Framework Core[11]: у межах BeginTransactionAsync виконуються перевірки балансу, активності активу та відсутності повторного придбання, а далі – списання внутрішньої валюти ApplicationUser.Coins і створення запису UserOwnedItem. Альтернативу без явної транзакції (з покладанням на SaveChanges) відхилено через ризик частково застосованих змін у разі помилки. Купівля преміум-підписки виконується в тому самому методі Purchase з окремою гілкою, яка оновлює поле PremiumUntilUtc користувача замість створення UserOwnedItem, що дає змогу повторно купувати підписку для продовження терміну дії.

Як систему автентифікації обрано вбудовану ASP.NET Core Identity з cookie-схемою з іменем «QAuth» та можливістю входу через Google OAuth 2.0. Це рішення детальніше обґрунтовано у спільному підрозділі 3.2 та в частині, присвяченій безпеці даних. Сервіс PremiumAccessService реалізовано як окремий компонент із методом EnsureUsersCanAccessMinigameAsync, що дозволяє іншим сервісам, зокрема GameRoomService та GameService, перевіряти право доступу до преміум-мінігри одним викликом без дублювання бізнес-логіки. Для зберігання аватарів використано хмарне сховище Cloudinary, яке забезпечує вбудований CDN та автоматичну оптимізацію зображень. Локальне зберігання у файловій системі контейнера WebApi свідомо не використовувалось через ризик втрати даних під час повторного розгортання застосунку.

Для організації фонового перерахунку рейтингу використано бібліотеку Quartz.NET. Альтернативні підходи, зокрема використання IHostedService із власним Timer або BackgroundService, були відхилені через відсутність підтримки cron-розкладів та механізмів обробки пропущених запусків. Використання Quartz.NET дозволило декларативно описати розклад виконання задачі LeadersUpdaterJob, яка запускається першого числа кожного місяця, без необхідності додаткової реалізації логіки таймерів.

3.3 Конструювання програмного забезпечення

У межах конструювання сервісів Rooms, GameLogic, Market та Users розроблено чотири ключові алгоритми: виконання ходу гравця з оновленням стану та сповіщенням учасників, життєвий цикл мультиплеєрної кімнати від створення до автоматичного запуску гри, транзакційну купівлю активу у маркеті та щомісячне оновлення таблиці лідерів. Алгоритми реалізовані у відповідних класах і охоплюють як перевірки бізнес-правил, так і взаємодію з зовнішніми компонентами (SignalR, EF Core, PremiumAccessService).

Алгоритм виконання ходу гравцем складається з таких послідовних дій:

- Клієнт надсилає об'єкт ходу через метод `MakeMove` хабу `GameHub`. Завдяки поліморфній десеріалізації `System.Text.Json` (тип-дискримінатор `gameType`, що формується з `MoveDiscriminator` усіх дескрипторів) хаб одразу отримує конкретний підтип `GameMove` відповідної мініігри;

- `GameHub` передає хід у `GameService`, який звертається до `GameLogicFactory`. Фабрика за `gameKey` ігрової сесії знаходить `IMinigameDescriptor` у `IMinigameRegistry` та або відновлює стан з `GameSessionState.StateJson` через `IGameStatePersistence.LoadAsync`, або повертає вже завантажений у пам'ять рушій;

- Рушій загорнутий у `GameEngineWrapper<TState, TMove>`. Метод `MakeMove` виконує перевірку типу ходу та делегує його реалізації `IGameLogic<TState, TMove>`: спершу викликається валідатор (`IGameValidator`), потім – функція правил (`IGameRules`), яка повертає новий стан і нараховує бали гравцям;

- Оновлений стан зберігається через `GameStatePersistence.UpdateAsync`: серіалізована JSON-форма пишеться у поле `state_json` таблиці `game_session_state` в одній транзакції з оновленням батьківського рядка `game_session` (поточний хід, лічильник раундів);

- `GameService` викликає `SignalRGameNotifier.NotifyStateChanged`, який через `IHubContext<GameHub>` транслює новий стан усім клієнтам у групі `game-{guid}`. Якщо рушій повідомив, що гра завершена (`NeedToFinish`), додатково створюється або оновлюється запис у `leaderboard_entry`, після чого клієнтам надсилається фінальне повідомлення з підсумками.

Алгоритм життєвого циклу мультиплеєрної кімнати, реалізований у `GameRoomService`, `IGameRoomManager` та `IRoomNotifier`, включає такі основні фази:

- Хост викликає `CreateRoomAsync`: сервіс перевіряє доступ до мініігри, відсутність хоста в іншій кімнаті та коректність параметрів (гравців

≥ 2 , команд ≥ 1 , гравців $\geq$ команд), після чого створює `GameRoom` з командами й додає хоста в першу команду;

- Інші гравці викликають `JoinRoomAsync`, `SwitchTeamAsync` або `LeaveRoomAsync`. Усі модифікації стану кімнати виконуються під замком `GameRoom.SyncRoot`, що гарантує цілісність структури команд за умов конкурентного доступу. Після кожної зміни `SignalRRoomNotifier` транслює оновлений склад кімнати у групу `room-{guid}`;

- Коли кількість гравців у кімнаті досягає `MaxPlayers` або хост викликає `LaunchRoomAsync`, статус кімнати переходить у `Launching`. `GameRoomService` звертається до `GameService` для створення ігрової сесії, отримує її ідентифікатор і транслює всім учасникам подію `RoomLaunched` з посиланням на сесію. Подальша взаємодія перемикається з `GameRoomHub` на `GameHub`, а сама кімната видаляється з `IGameRoomManager`;

- Кімнати, які не заповнилися протягом `IsJoinExpired`-таймауту, прибираються періодичним фоновим завданням, що звертається до `GetAvailableRooms`, і виключаються зі списку публічних лобі.

Алгоритм транзакційної купівлі активу, реалізований у `MarketService.Purchase`, передбачає виконання таких послідовних дій у межах однієї транзакції EF Core:

- Викликається `BeginTransactionAsync` на `DbContext`. Виконується пошук `MarketItem` за ідентифікатором та перевірка прапора `IsActive`; якщо актив неактивний, транзакція скасовується;

- Завантажується `ApplicationUser` за ідентифікатором користувача. Для звичайних активів додатково перевіряється `IUserOwnedItemRepository`: якщо актив вже належить користувачеві, повертається помилка, бо повторне придбання тієї самої косметики не передбачене;

- Якщо актив платний, поле `ApplicationUser.Coins` зменшується на `MarketItem.PriceCoins`. У разі недостатнього балансу транзакція скасовується;

- Для звичайного активу створюється запис `UserOwnedItem` з посиланням на `MarketItem` та поточним часом `PurchasedAtUtc`. Для активу

типу `PremiumSubscription` натомість оновлюється поле `ApplicationUser.PremiumUntilUtc` – час відлічується від поточного значення або від поточного UTC-моменту (залежно від того, що пізніше) і додає `MarketItem.DurationMonths` місяців;

– Виконується `SaveChangesAsync` і `CommitTransactionAsync`. Якщо будь-який крок завершився виключенням `HandledException`, виконується `RollbackAsync`, і клієнт отримує локалізоване повідомлення про причину відмови.

Алгоритм щомісячного оновлення таблиці лідерів реалізовано у фоновому завданні `LeadersUpdaterJob` (`Quartz.NET`, cron-розклад «00:00 першого числа кожного місяця»). На кожному запуску завдання звертається до `LeaderBoardService`, який агрегує закінчені `GameSession` за попередній календарний місяць у розрізі `gameTypeId` та `user_id`, обчислює найкращий результат і виконує `upsert` у таблицю `leaderboard_entry` для періоду `Monthly`. Для періоду `AllTime` аналогічна агрегація виконується без часового фільтра. Алгоритм свідомо винесено у фон, щоб не уповільнювати фінальні запити користувачів у момент завершення сесії.

Висновки до розділу

У третьому розділі було детально описано архітектуру та конструювання чотирьох сервісів серверного застосунку `Quizanchos.WebApi`: `GameLogic`, що виконує логіку конкретної мініігри через типобезпечну обгортку `GameEngineWrapper<TState, TMove>` та зберігає стан через `IGameStatePersistence`; `Rooms`, що керує життєвим циклом мультиплеєрних кімнат через `GameRoomService` та `InMemoryGameRoomManager` на основі `ConcurrentDictionary`, а сповіщення транслює через `SignalRRoomNotifier`; `Market`, що реалізує транзакційну купівлю активу у `MarketService` через інтерфейси `IMarketItemRepository` та `IUserOwnedItemRepository`; `Users`, що обслуговує профілі, ролі, преміум-доступ, рейтинг та статистику через набір сервісів `UserScoreService`, `LeaderBoardService`, `PremiumAccessService` у

поєднанні з фоновим завданням `LeadersUpdaterJob`. Для цих сервісів побудовано окрему компонентну діаграму C4 рівня 3.

Обґрунтовано ключові архітектурні рішення: винесення стану ігрової сесії у JSONB-поле `state_json` замість окремих доменних таблиць для кожної мініігри; використання узагальнених інтерфейсів `IGameLogic<TState, TMove>` та `GameEngineWrapper<TState, TMove>` для типобезпеки в межах плагіна та однорідного інтерфейсу `IGameEngine` для хосту; in-memory зберігання кімнат через `ConcurrentDictionary` з винесенням `IGameRoomManager` у окремий тип; використання `SignalR` з `WebSocket`-транспортом та двома окремими хабами `GameHub` і `GameRoomHub`; виконання операції купівлі активу як єдиної транзакції EF Core з явним `BeginTransactionAsync` і `RollbackAsync` при помилках; винесення `PremiumAccessService` у окремий сервіс задля повторного використання з інших шарів; зберігання аватарів у `Cloudinary` через офіційний SDK; використання `Quartz.NET` для щомісячного перерахунку таблиці лідерів. На рівні конструювання детально розкрито алгоритми виконання ходу гравця, життєвого циклу мультиплеєрної кімнати, транзакційної купівлі активу та щомісячного оновлення таблиці лідерів.

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Опис процесів тестування

Для перевірки правильності реалізації сервісів індивідуальної частини застосовано дворівневий підхід: автоматизоване модульне тестування на основі фреймворка xUnit.net та мануальне тестування ключових користувацьких сценаріїв у браузері.

Модульні тести розміщено у проєкті Quizanchos.Quiz.Tests (середовище .NET 10, фреймворк xUnit.net 2.9 [12]). Залежності сервісів підмінюються заглушками на основі бібліотеки Moq 4.20 [13] – зокрема інтерфейси менеджера кімнат, нотифікатора, репозиторіїв товарів, предметів користувача, рахунків та реєстру мінігор. Доступ до бази даних емулюється провайдером EF Core InMemory, що дає змогу перевіряти поведінку запитів без піднімання Postgres-контейнера й зберігати детермінованість прогону. Жоден тест не виконує мережових звернень, не звертається до Cloudinary та не надсилає пошту через SMTP – клас розсилання листів тестується лише на рівні логіки шаблонізації.

Усі тести побудовано за класичним шаблоном Arrange–Act–Assert. У секції Arrange ініціалізуються заглушки залежностей та контрольовані тестові дані; у секції Act викликається безпосередньо метод, що тестується; у секції Assert перевіряється повернений результат, виклики на mock-об'єктах (через Verify) та стан in-memory бази даних. Для перевірки винятків застосовується Assert.ThrowsAsync<HandledException>.

Загалом для сервісів індивідуальної частини розроблено 36 модульних тестів, згрупованих у вісім тестових класів. Структуру тестового проєкту наведено на рисунку 4.1, а деталізацію тестових класів представлено у таблиці 4.3

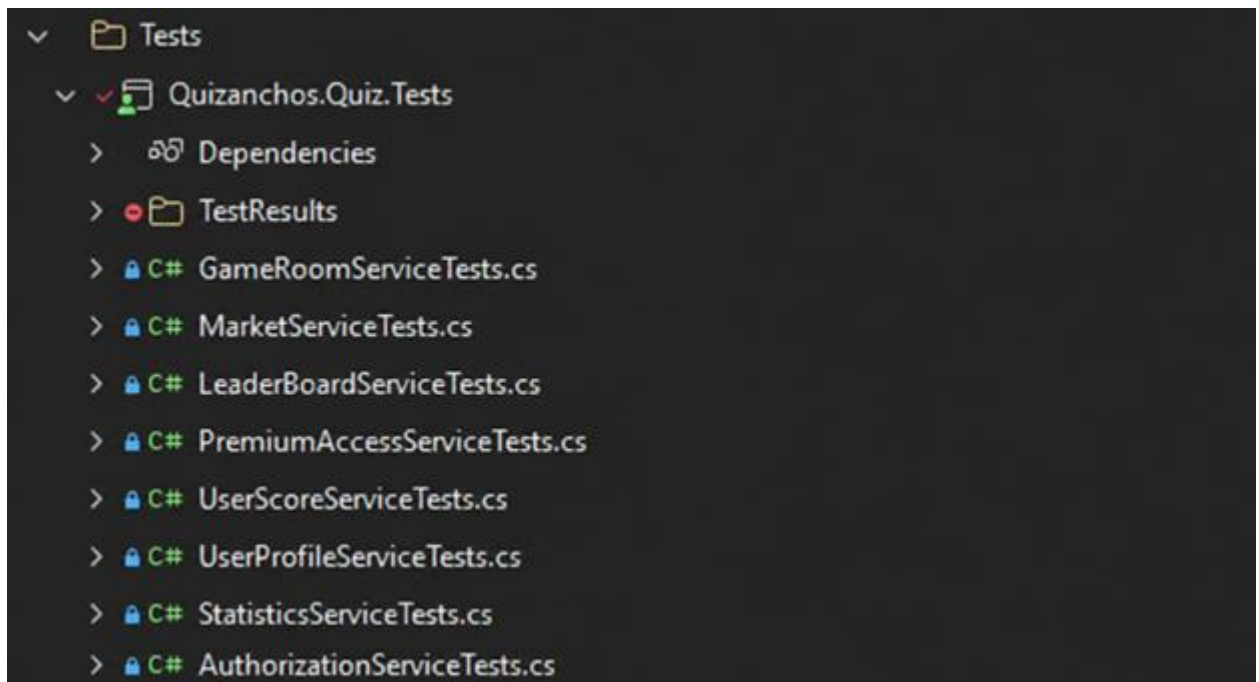


Рисунок 4.1 – Структура тестового проєкту Quizanchos.Quiz.Tests

Таблиця 4.1 – Структура проєкту Quizanchos.Quiz.Tests

Тестовий клас	Сервіс, що тестується	Кількість тестів
GameRoomServiceTests	Services/Rooms/GameRoomService.cs	8
MarketServiceTests	Services/Market/MarketService.cs	7
LeaderBoardServiceTests	Services/Users/LeaderBoardService.cs	4
PremiumAccessServiceTests	Services/Users/PremiumAccessService.cs	4
UserScoreServiceTests	Services/Users/UserScoreService.cs	4
UserProfileServiceTests	Services/Users/UserProfileService.cs	5
StatisticsServiceTests	Services/Users/StatisticsService.cs	3
AuthorizationServiceTests	Services/Auth/AuthorizationService.cs, EmailConfirmationPasswordUpdateService.cs	1
Разом		36

Клас GameRoomServiceTests містить 8 тестів і охоплює перевірку повного життєвого циклу мультиплеерної кімнати. У межах цього класу протестовано створення кімнати, автоматичне додавання хоста до першої команди, перевірку мінімальної кількості гравців, заборону одночасної участі

хоста у кількох кімнатах, обробку спроби приєднання до неіснуючої або простроченої кімнати, контроль заповненості команди, запуск гри після приєднання останнього гравця, а також автоматичне закриття кімнати після виходу хоста.

Клас `MarketServiceTests` містить 7 тестів і призначений для перевірки операцій внутрішньоігрового маркету. Тестами охоплено коректне формування каталогу активних предметів, сортування безкоштовних і платних позицій, обробку неіснуючих предметів, перевірку достатності балансу користувача, заборону повторної купівлі вже придбаного звичайного предмета, успішну купівлю аватара зі списанням монет і створенням запису про володіння, а також логіку нарахування та продовження преміум-підписки.

Клас `LeaderBoardServiceTests` містить 4 тести і перевіряє формування таблиці лідерів. У тестах розглянуто сортування користувачів за сумарною кількістю очок, формування рейтингу з фільтрацією за конкретною мінігрою, виключення користувачів із службовою роллю власника з рейтингу, а також валідацію параметрів пагінації.

Клас `PremiumAccessServiceTests` містить 4 тести і відповідає за перевірку контролю доступу до преміум-мініігор. Тестами підтверджено, що для звичайних мініігор додаткова перевірка не виконується, користувачі з активною преміум-підпискою отримують доступ, а участь у преміум-мінігрі блокується у випадку відсутності або завершення дії підписки хоча б в одного з учасників.

Клас `UserScoreServiceTests` містить 4 тести і перевіряє логіку нарахування очок користувачам. У межах тестування перевірено створення нового запису результату для пари користувач і мінігра, накопичення очок під час повторних оновлень, ігнорування нульових і від'ємних значень, а також підрахунок сумарного результату користувача за всіма типами мініігор.

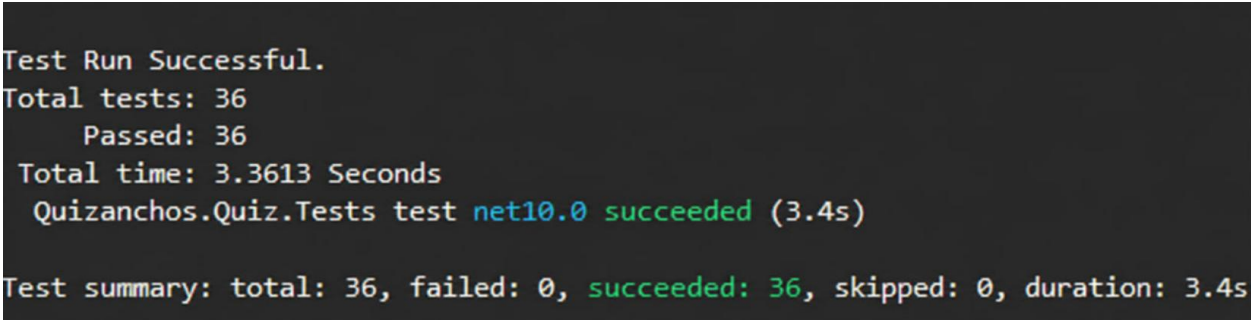
Клас `UserProfileServiceTests` містить 5 тестів і охоплює перевірку функцій, пов'язаних з обліковим записом користувача. Зокрема, протестовано обмеження розміру файлу аватара, перевірку допустимих MIME-типів,

контроль унікальності нікнейма, захист від некоректного поповнення балансу, а також заборону видалення акаунта за наявності фінансової історії.

Клас `StatisticsServiceTests` містить 3 тести і перевіряє коректність формування агрегованих показників для адміністративної панелі. Тестами охоплено валідацію діапазону дат, обмеження максимальної довжини періоду аналізу, а також обчислення середньої кількості одночасних сесій на основі тестового набору даних із частковим перетином у часі.

Клас `AuthorizationServiceTests` містить 1 тест і перевіряє окремий сценарій роботи механізму відновлення пароля. Тест підтверджує, що одноразовий код відновлення з вичерпаним терміном дії коректно відхиляється системою.

Запуск автоматизованих тестів виконується стандартною командою `dotnet test Quizanchos.WebApi.Tests` з кореневого каталогу рішення. Усі 36 тестових сценаріїв проходять успішно, без помилок та пропущених тестів. Підсумковий результат проходження тестів наведено на рисунку 4.2.



```
Test Run Successful.
Total tests: 36
  Passed: 36
Total time: 3.3613 Seconds
Quizanchos.Quiz.Tests test net10.0 succeeded (3.4s)
Test summary: total: 36, failed: 0, succeeded: 36, skipped: 0, duration: 3.4s
```

Рисунок 4.2 – Результати успішного проходження автоматизованих модульних тестів

Для перевірки реалізованої функціональності в умовах, наближених до реального використання платформи, виконано серію мануальних тестів. Кожен тест охоплює окремий сценарій взаємодії користувача із сервісами індивідуальної частини: автентифікацію, профіль користувача, мультиплеєрні кімнати, маркет внутрішньоігрових активів, таблицю лідерів та адміністративну панель. Результати тестування наведено у таблицях 4.2 – 4.21.

Таблиця 4.2 – Тест 2.1 Реєстрація користувача з валідними даними

Початковий стан системи	Користувач перебуває на сторінці реєстрації, не має облікового запису на платформі
Вхідні дані	Електронна пошта tester01@example.com, нікнейм tester01, пароль Str0ng!Pass2026, підтвердження пароля Str0ng!Pass2026
Опис проведення тесту	У форму реєстрації вводиться нова електронна пошта, нікнейм довжиною не менше трьох символів, пароль довжиною від 10 до 64 символів, який містить літеру, цифру та спеціальний символ, а також його підтвердження. Натискається кнопка «Зареєструватися»
Очікуваний результат	Створюється новий запис у таблиці AspNetUsers зі значенням EmailConfirmed = false, на вказану пошту надсилається лист з посиланням підтвердження, користувач перенаправляється на сторінку авторизації з повідомленням про необхідність підтвердити пошту
Фактичний результат	Збігається з очікуванням.

Таблиця 4.3 – Тест 2.2 Реєстрація з уже зайнятою електронною поштою

Початковий стан системи	У системі вже існує підтверджений обліковий запис з адресою tester01@example.com
Вхідні дані	Електронна пошта tester01@example.com, нікнейм another, пароль Anoth3r!Pass1
Опис проведення тесту	Користувач намагається повторно зареєструватись з вже зайнятою електронною поштою
Очікуваний результат	Реєстрація відхиляється, форма показує повідомлення про те, що пошта вже використовується, запис у базу даних не додається
Фактичний результат	Збігається з очікуванням.

Таблиця 4.4 – Тест 2.3 Підтвердження електронної пошти за посиланням

Початковий стан системи	Існує користувач tester01@example.com з EmailConfirmed = false, отриманий лист з токеном підтвердження
Вхідні дані	Дійсний токен підтвердження з листа
Опис проведення тесту	Користувач переходить за посиланням з листа, що містить токен підтвердження пошти

Продовження таблиці 4.4

Очікуваний результат	Поле EmailConfirmed у записі користувача оновлюється на true, відкривається сторінка з повідомленням про успішне підтвердження, користувач отримує можливість авторизуватися
Фактичний результат	Збігається з очікуваним.

Таблиця 4.5 – Тест 2.4 Авторизація з валідними даними та видача

QAuth

Початковий стан системи	Користувач з підтвердженою поштою перебуває на сторінці входу
Вхідні дані	Електронна пошта tester01@example.com, пароль Str0ng!Pass2026
Опис проведення тесту	У форму входу вводиться електронна пошта та коректний пароль, натискається кнопка «Увійти»
Очікуваний результат	Метод AuthorizationService.SignIn повертає успішний результат, у браузері встановлюється cookie QAuth з атрибутами HttpOnly, Secure, SameSite=Lax, користувач перенаправляється на головну сторінку платформи
Фактичний результат	Збігається з очікуваним.

Таблиця 4.6 – Тест 2.5 Авторизація з невалідним паролем

Початковий стан системи	Існує користувач tester01@example.com з підтвердженою поштою
Вхідні дані	Електронна пошта tester01@example.com, пароль WrongPass123!
Опис проведення тесту	У форму входу вводиться правильна електронна пошта та неправильний пароль, натискається кнопка «Увійти»
Очікуваний результат	Метод _signInManager.PasswordSignInAsync повертає неуспіх, виникає HandledException з повідомленням Invalid username or password, cookie не встановлюється
Фактичний результат	Збігається з очікуваним.

Таблиця 4.7 – Тест 2.6 Запит відновлення пароля

Початковий стан системи	Користувач tester01@example.com забув пароль, перебуває на сторінці «Забули пароль?»
Вхідні дані	Електронна пошта tester01@example.com

Продовження таблиці 4.7

Опис проведення тесту	На сторінці відновлення пароля вводиться електронна пошта, натискається кнопка «Надіслати код». Перевіряється отриманий лист
Очікуваний результат	На пошту користувача надсилається лист з одноразовим шестизначним кодом та терміном дії, у базі даних створюється запис про активний токен скидання пароля, користувач переходить до сторінки введення коду
Фактичний результат	Збігається з очікуванням.

Таблиця 4.8 – Тест 2.7 Підтвердження скидання пароля з простроченим кодом

Початковий стан системи	Лист з кодом відновлення був отриманий понад добу тому
Вхідні дані	Шестизначний код з листа, новий пароль BrandN3w!Pass4
Опис проведення тесту	Користувач вводить старий код, дата генерації якого перевищує дозволений термін дії
Очікуваний результат	Метод ConfirmPasswordResetAsync відхиляє запит з повідомленням про прострочений код, пароль не змінюється, запис токена видаляється або позначається як використаний
Фактичний результат	Збігається з очікуванням.

Таблиця 4.9 – Тест 2.8 Завантаження аватара дозволеного формату та розміру

Початковий стан системи	Користувач авторизований, перебуває на сторінці профілю
Вхідні дані	Зображення avatar.png розміром 320×320 пікселів, розмір файлу 180 КБ, MIME-тип image/png
Опис проведення тесту	Користувач натискає кнопку «Змінити аватар», обирає файл та підтверджує завантаження

Продовження таблиці 4.9

Очікуваний результат	Файл проходить перевірку розміру (≤ 5 МБ) та MIME-типу, передається у Cloudinary з трансформацією Width(512).Height(512).Crop("limit"), у профілі користувача оновлюється поле AvatarUrl на отримане SecureUrl, новий аватар відображається у шапці платформи
Фактичний результат	Збігається з очікуванням.

Таблиця 4.10 – Тест 2.9 Спроба завантажити аватар розміром понад 5 МБ

Початковий стан системи	Користувач авторизований, перебуває на сторінці профілю
Вхідні дані	Зображення big_photo.jpg розміром 6,3 МБ, MIME-тип image/jpeg
Опис проведення тесту	Користувач намагається завантажити файл, розмір якого перевищує верхнє обмеження сервісу
Очікуваний результат	Метод UpdateAvatar викидає HandledException з повідомленням Avatar must be 5 MB or smaller., поле AvatarUrl залишається незмінним, виклик до Cloudinary не виконується
Фактичний результат	Збігається з очікуванням.

Таблиця 4.11 – Тест 2.10 Експорт персональних даних користувача

Початковий стан системи	Авторизований користувач, що має історію ігрових сесій, придбаних предметів та поповнень балансу
Вхідні дані	Кнопка «Експортувати дані» у розділі налаштувань профілю
Опис проведення тесту	Користувач натискає кнопку експорту даних. Браузер ініціює завантаження JSON-файлу
Очікуваний результат	Файл user-data-export.json містить структуру з полями profile, scores, ownedItems, gameSessions, topUpOrders, де перелічені реальні дані користувача, без даних інших користувачів
Фактичний результат	Збігається з очікуванням.

Таблиця 4.12 – Тест 2.11 Видалення акаунта без історії поповнень

Початковий стан системи	Авторизований користувач без замовлень поповнення балансу
Вхідні дані	Поточний пароль Str0ng!Pass2026 у формі підтвердження видалення
Опис проведення тесту	Користувач переходить у розділ налаштувань, обирає «Видалити акаунт», вводить поточний пароль та підтверджує дію
Очікуваний результат	Метод DeleteAccountAsync перевіряє пароль, не знаходить записів у TopUpOrders для цього користувача, видаляє запис зAspNetUsers каскадно з пов'язаними сутностями, виконує SignOutAsync, користувач перенаправляється на головну сторінку як неавторизований
Фактичний результат	Збігається з очікуванням.

Таблиця 4.13 – Тест 2.12 Спроба видалити акаунт з історією поповнень

Початковий стан системи	Авторизований користувач, який раніше здійснив принаймні одне поповнення балансу (TopUpOrder)
Вхідні дані	Поточний пароль користувача у формі підтвердження видалення
Опис проведення тесту	Користувач намагається видалити обліковий запис, який має пов'язану історію фінансових операцій
Очікуваний результат	Метод DeleteAccountAsync викидає HandledException з повідомленням про необхідність звернення до підтримки, обліковий запис залишається активним, фінансова історія зберігається
Фактичний результат	Збігається з очікуванням.

Таблиця 4.14 – Тест 2.13 Створення мультиплеєрної кімнати

Початковий стан системи	Авторизований користувач, що не перебуває у жодній іншій кімнаті
Вхідні дані	Тип мініігри Quiz Multiplayer (MinigameType=3), максимальна кількість гравців 4, кількість команд 2, параметри гри (категорія, рівень складності)
Опис проведення тесту	Користувач переходить у розділ мініігри Quiz Multiplayer, натискає «Створити кімнату», задає параметри та підтверджує створення

Продовження таблиці 4.14

Очікуваний результат	У InMemoryGameRoomManager створюється запис з новим RoomId, формуються дві команди по два гравці, хост автоматично додається до команди 0, статус кімнати – WaitingForPlayers, інші користувачі бачать кімнату у списку доступних
Фактичний результат	Збігається з очікуванням.

Таблиця 4.15 – Тест 2.14 Приєднання до кімнати у вільну команду

Початковий стан системи	Існує кімната у статусі WaitingForPlayers з вільним місцем у команді 1
Вхідні дані	Ідентифікатор кімнати, індекс команди teamIndex = 1
Опис проведення тесту	Другий користувач переходить за посиланням на кімнату, обирає команду 1 та натискає «Приєднатися»
Очікуваний результат	Гравець додається у список Players команди 1, через SignalR-хаб GameRoomHub усі учасники кімнати отримують повідомлення NotifyPlayerJoinedRoom та NotifyRoomUpdated зі свіжим станом кімнати
Фактичний результат	Збігається з очікуванням.

Таблиця 4.16 – Тест 2.15 Автоматичний запуск гри при заповненні останнього місця

Початковий стан системи	Кімната з трьома гравцями, обмеження – чотири гравці, статус WaitingForPlayers
Вхідні дані	Четвертий гравець приєднується до останнього вільного місця
Опис проведення тесту	Четвертий користувач приєднується до останнього вільного місця у будь-якій команді
Очікуваний результат	Статус кімнати переходить у GameRoomStatus.Launching, GameRoomService викликає GameService.CreateMultiPlayerGameAsync, кімната отримує LaunchedGameId, статус оновлюється на GameStarted, усі гравці отримують повідомлення NotifyGameStarting із посиланням на ігрову сесію
Фактичний результат	Збігається з очікуванням.

Таблиця 4.17 – Тест 2.16 Вихід хоста з кімнати

Початковий стан системи	Кімната у статусі WaitingForPlayers, у ній перебуває хост і два інші гравці
Вхідні дані	Запит хоста на вихід (LeaveRoom)
Опис проведення тесту	Хост кімнати натискає кнопку «Покинути кімнату»
Очікуваний результат	Метод LeaveRoomAsync визначає, що користувач є хостом, статус кімнати переводиться у Closed, усі учасники отримують повідомлення NotifyRoomClosed з причиною Host left the room, запис видаляється з InMemoryGameRoomManager
Фактичний результат	Збігається з очікуванням.

Таблиця 4.18 – Тест 2.17 Купівля внутрішньоігрового активу за достатньої кількості монет

Початковий стан системи	Авторизований користувач з балансом 500 монет, у каталозі активний предмет «Emoji Pack» вартістю 150 монет, який ще не належить користувачу
Вхідні дані	Ідентифікатор предмета «Emoji Pack»
Опис проведення тесту	Користувач переходить на сторінку маркету, натискає «Купити» на предметі «Emoji Pack», підтверджує покупку у модальному вікні
Очікуваний результат	Метод MarketService.Purchase відкриває транзакцію EF, перевіряє наявність предмета у UserOwnedItems (немає), списує 150 монет з балансу (нове значення 350), створює запис у UserOwnedItems, фіксує транзакцію, повертає MarketPurchaseResultDto з оновленим балансом
Фактичний результат	Збігається з очікуванням.

Таблиця 4.19 – Тест 2.18 Купівля при недостатньому балансі

Початковий стан системи	Авторизований користувач з балансом 50 монет, у каталозі активний предмет «Premium Avatar» вартістю 200 монет
Вхідні дані	Ідентифікатор предмета «Premium Avatar»
Опис проведення тесту	Користувач намагається купити предмет, ціна якого перевищує його баланс

Продовження таблиці 4.19

Очікуваний результат	Метод Purchase визначає user.Coins < marketItem.PriceCoins, викидає HandledException з повідомленням Not enough coins., транзакція скасовується через RollbackAsync, баланс залишається незмінним (50 монет), запис у UserOwnedItems не створюється
Фактичний результат	Збігається з очікуванням.

Таблиця 4.20 – Тест 2.19 Купівля преміум-підписки з активною попередньою підпискою

Початковий стан системи	Авторизований користувач з активною преміум-підпискою до 2026-08-15, у каталозі предмет «Premium 1 Month» (DurationMonths=1)
Вхідні дані	Ідентифікатор предмета «Premium 1 Month»
Опис проведення тесту	Користувач купує преміум-підписку на місяць за наявності активної
Очікуваний результат	Поле PremiumUntilUtc оновлюється на 2026-09-15 (продовження від поточної дати закінчення, а не від моменту покупки), на сторінці маркету не створюється запис у UserOwnedItems, оскільки тип PremiumSubscription оброблюється окремо
Фактичний результат	Збігається з очікуванням.

Таблиця 4.21 – Тест 2.20 Перегляд таблиці лідерів з фільтром за мінігрою та виключенням Owner

Початковий стан системи	У системі зареєстровано 15 користувачів, серед яких один має роль Owner, з різними значеннями UserMinigameScore за різними мінігровими типами
Вхідні дані	Параметри пагінації skip=0, take=10, фільтр minigameType=1 (Quiz)
Опис проведення тесту	Користувач переходить на сторінку «Таблиця лідерів», обирає у фільтрі мінігру Quiz, налаштування пагінації – перші десять записів

Продовження таблиці 4.21

Очікуваний результат	Метод <code>LeaderBoardService.GetLeaderBoardAsync(10, 0, 1)</code> повертає список з 10 елементів <code>ApplicationUserInLeaderBoardDto</code> , відсортованих за спаданням <code>Score</code> для мініігри з <code>MinigameType=1</code> , користувач з роллю <code>Owner</code> у списку відсутній, перший елемент має <code>Position=1</code> , поле <code>Coins</code> коректно відображає поточний баланс
Фактичний результат	Збігається з очікуваним.

Перелічених у таблицях 4.2 – 4.21 двадцяти мануальних тестів достатньо для покриття основних користувацьких сценаріїв, реалізованих у сервісах індивідуальної частини. Усі тести виконано вручну у браузері Chromium 132, результати кожного з тестів збігаються з очікуваними.

4.2 Опис контрольного прикладу

Контрольний приклад послідовно демонструє наскрізний сценарій взаємодії користувача із сервісами індивідуальної частини: від реєстрації облікового запису до купівлі активу та перевірки оновленого рейтингу. Сценарій охоплює усі ключові потоки даних – автентифікацію, керування профілем, мультиплеєрні кімнати, маркет та таблицю лідерів – і дозволяє переконатися, що описані у попередніх підпунктах сервіси працюють узгоджено між собою.

Робота починається з головної сторінки, на якій представлені каталог доступних мініігор, скорочена таблиця лідерів та елементи входу й реєстрації (рисунок 4.3).

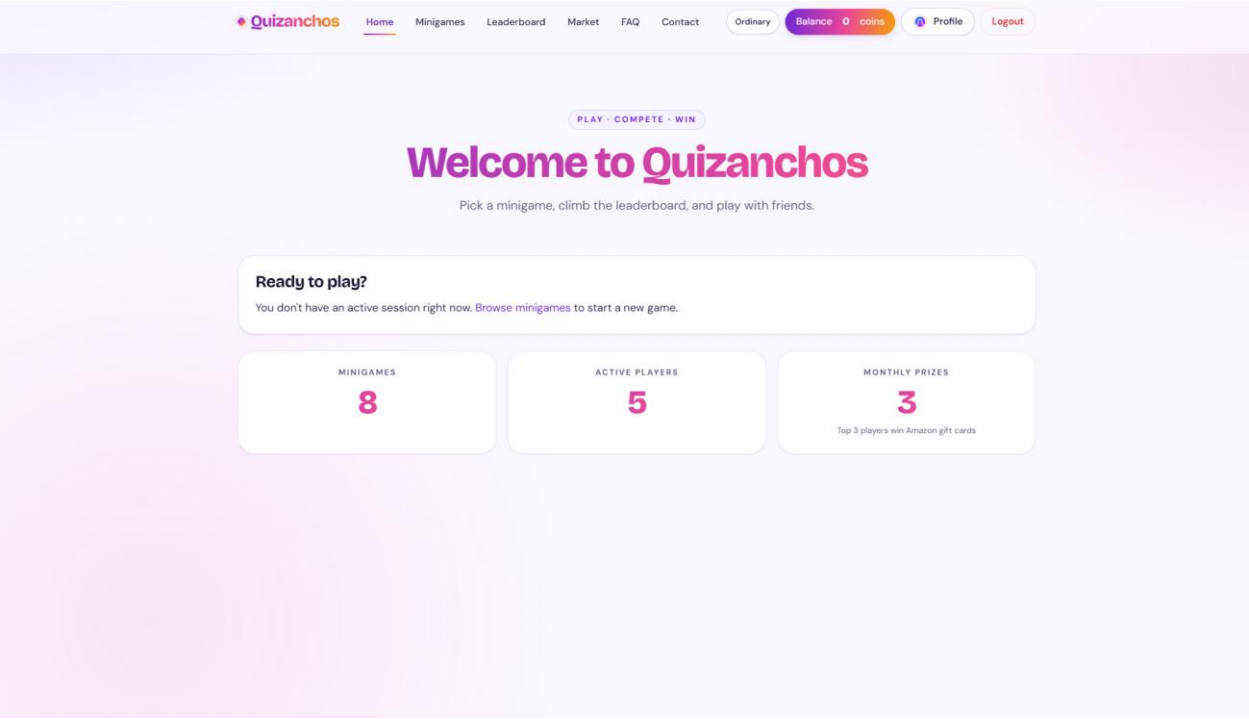


Рисунок 4.3 – Головна сторінка платформи

Новий користувач натискає кнопку «Зареєструватися», після чого відкривається форма створення облікового запису (рисунок 4.4). На вказану електронну адресу надсилається лист підтвердження.

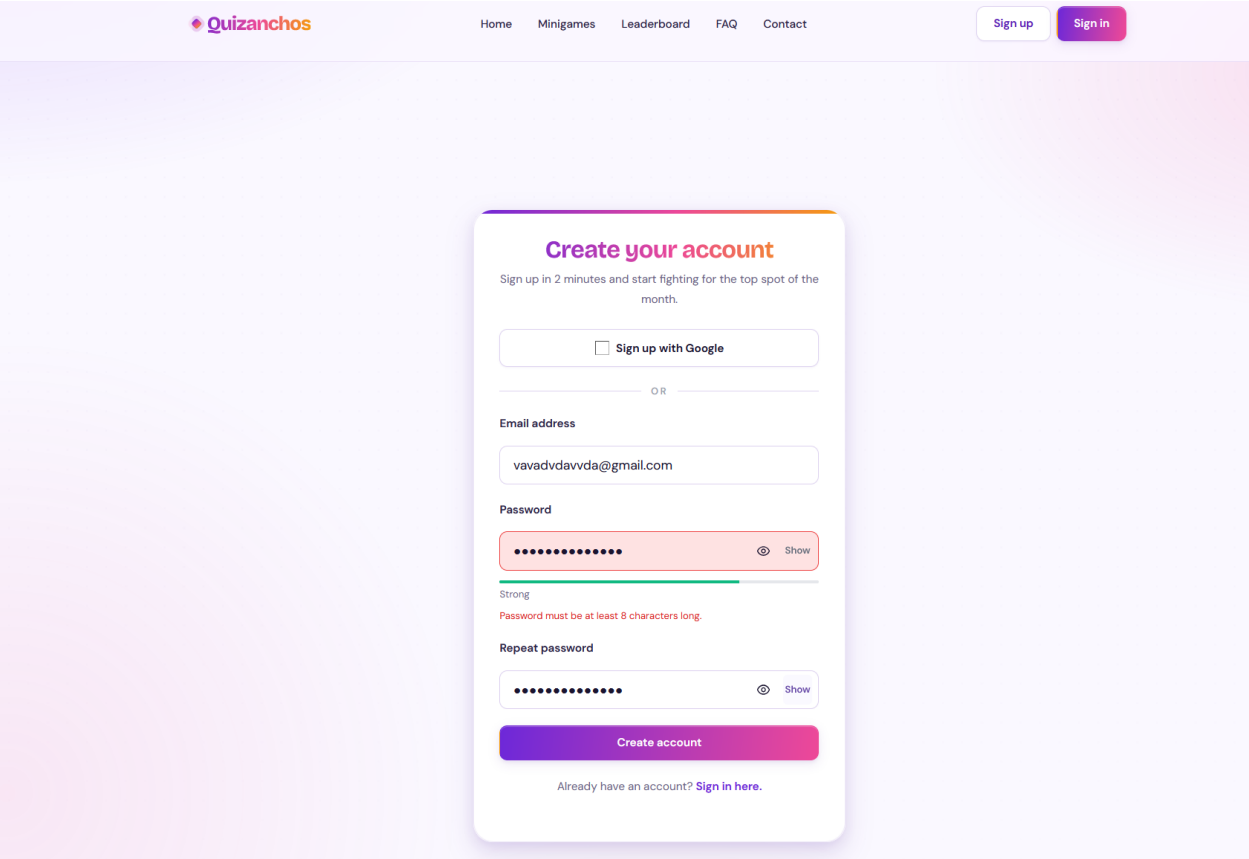


Рисунок 4.4 – Форма реєстрації нового користувача

Після реєстрації користувач опиняється у каталозі мініігор. Каталог містить як вбудовані мініігри (Quiz, Quiz Multiplayer), так і ті, що завантажуються динамічно у вигляді плагінів (рисунок 4.5).

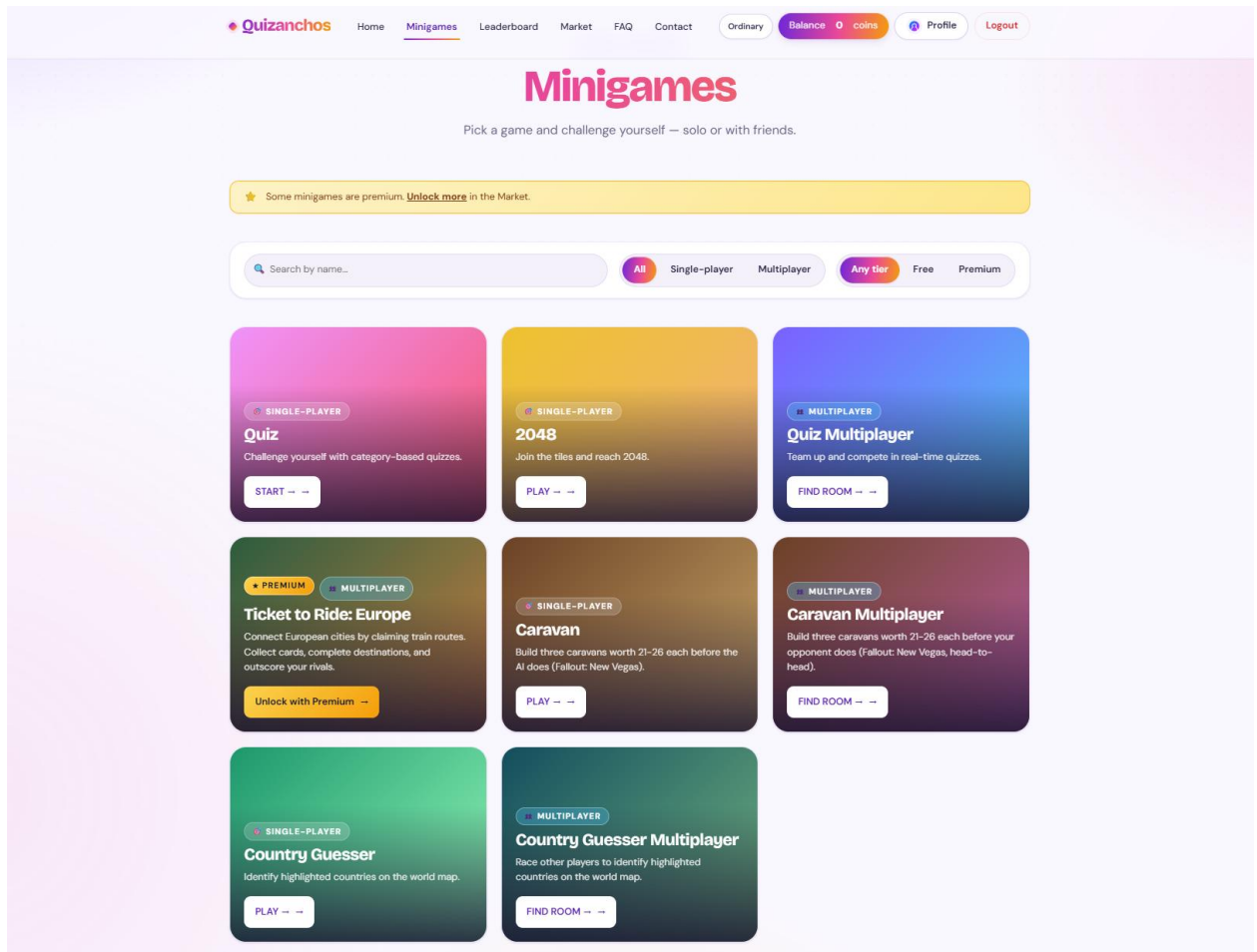


Рисунок 4.5 – Каталог доступних мініігор

Користувач обирає мінігру Quiz та переходить до екрана налаштувань сесії, де задає категорію питань, рівень складності та кількість раундів (рисунок 4.6).

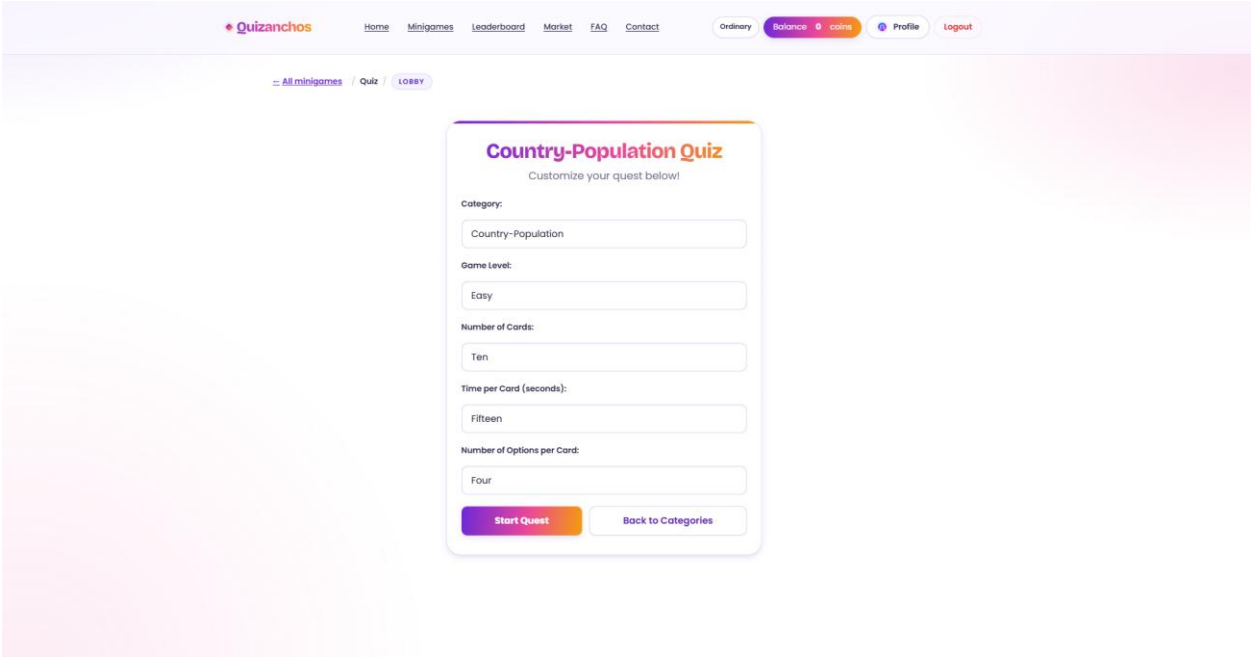


Рисунок 4.6 – Налаштування ігрової сесії

Після підтвердження параметрів запускається безпосередньо ігровий процес. Для кожної картки виводяться сутність, варіанти відповіді та таймер раунду (рисунок 4.7).

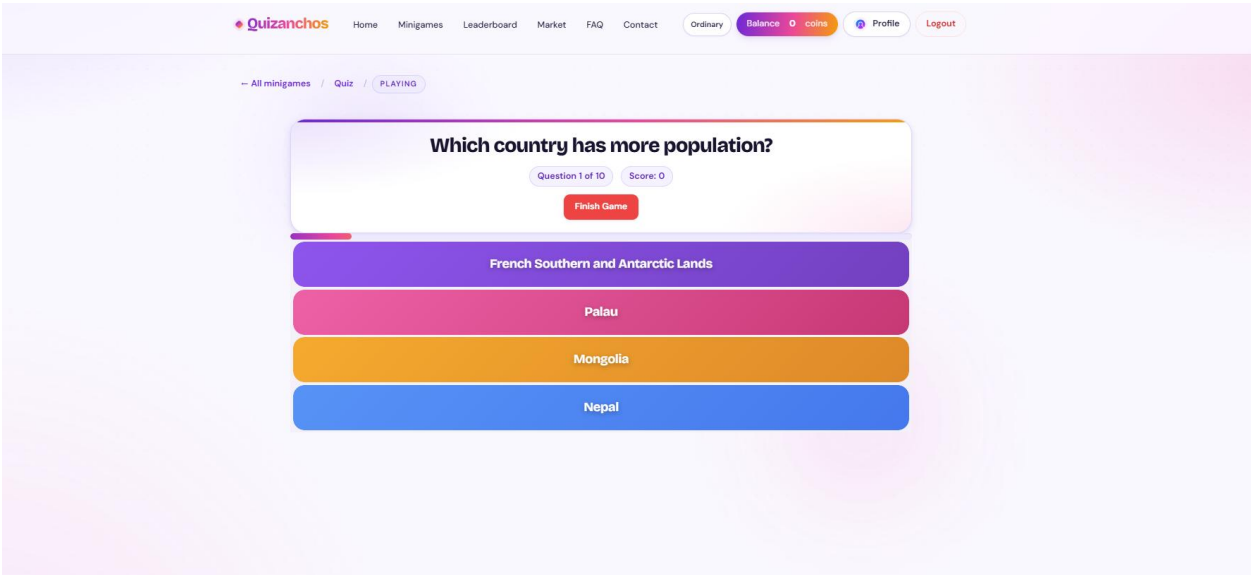


Рисунок 4.7 – Проходження ігрового раунду

Після відповіді на останню картку платформа підраховує підсумковий результат, нараховує очки рейтингу та виводить підсумковий екран із кнопками повернення до каталогу або повторного запуску сесії (рисунок 4.8).

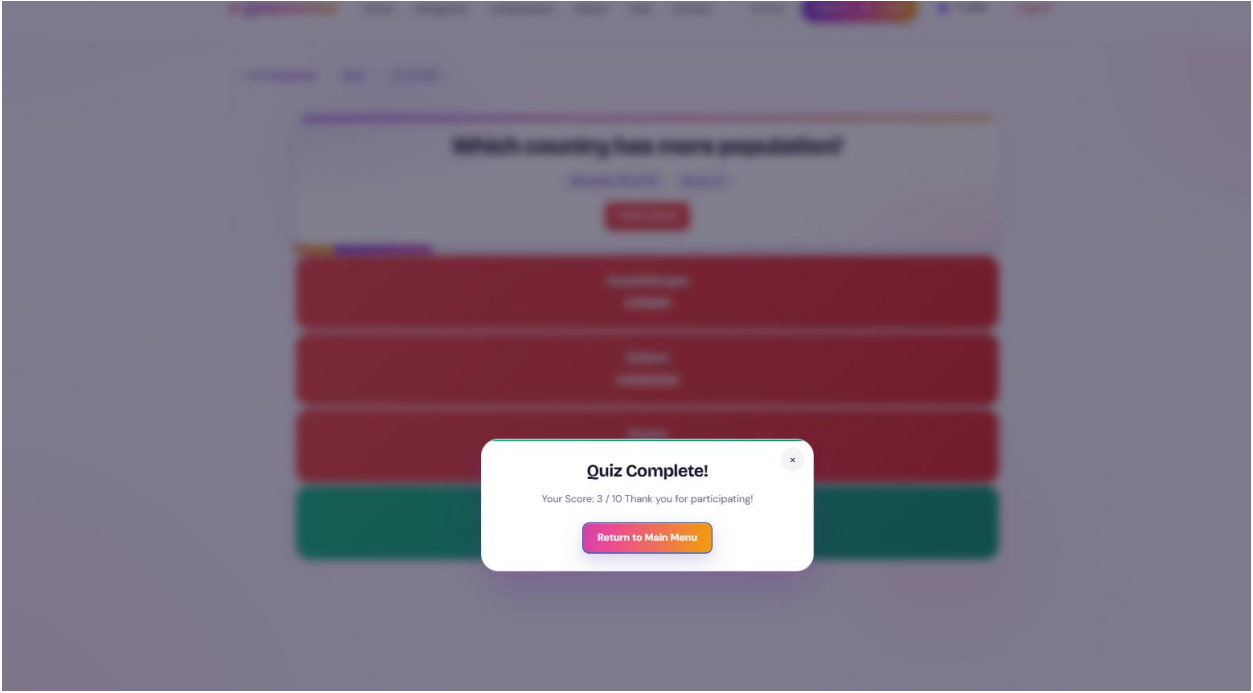


Рисунок 4.8 – Підсумковий екран ігрової сесії

Завершивши партію, користувач переходить до сторінки рейтингу. Сторінка містить таблицю лідерів із можливістю фільтрації за мінігрою та часовим інтервалом (рисунок 4.9).

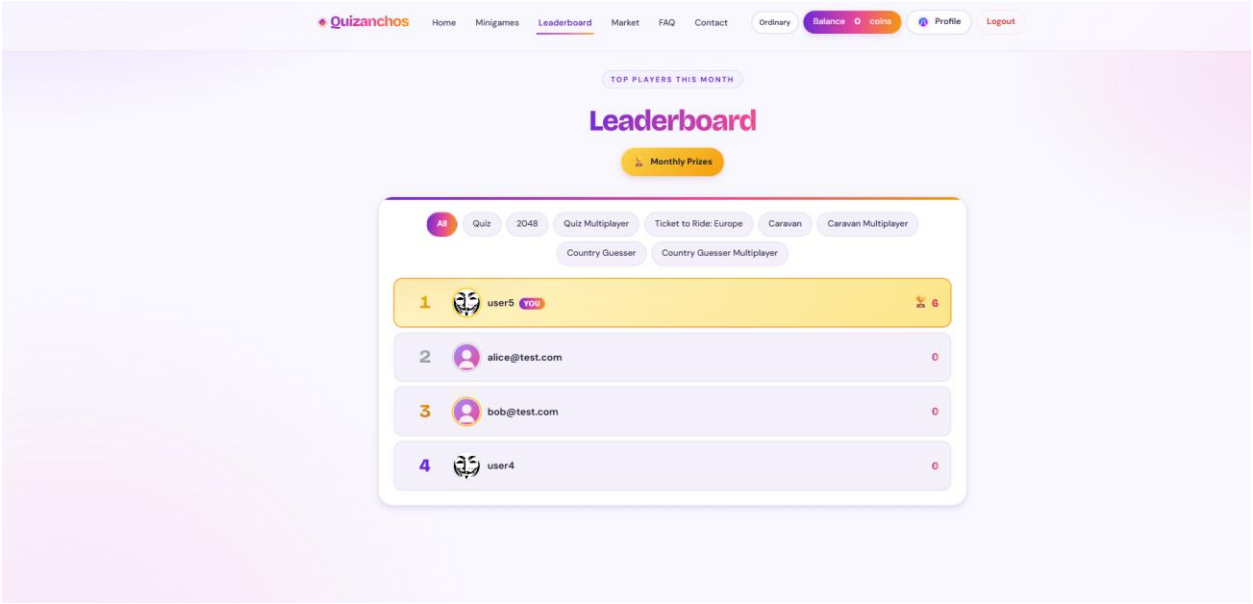


Рисунок 4.9 – Таблиця лідерів платформи

У розділі профілю користувач переглядає власні дані, баланс внутрішньоігрової валюти, історію транзакцій та інвентар придбаних активів (рисунок 4.10).

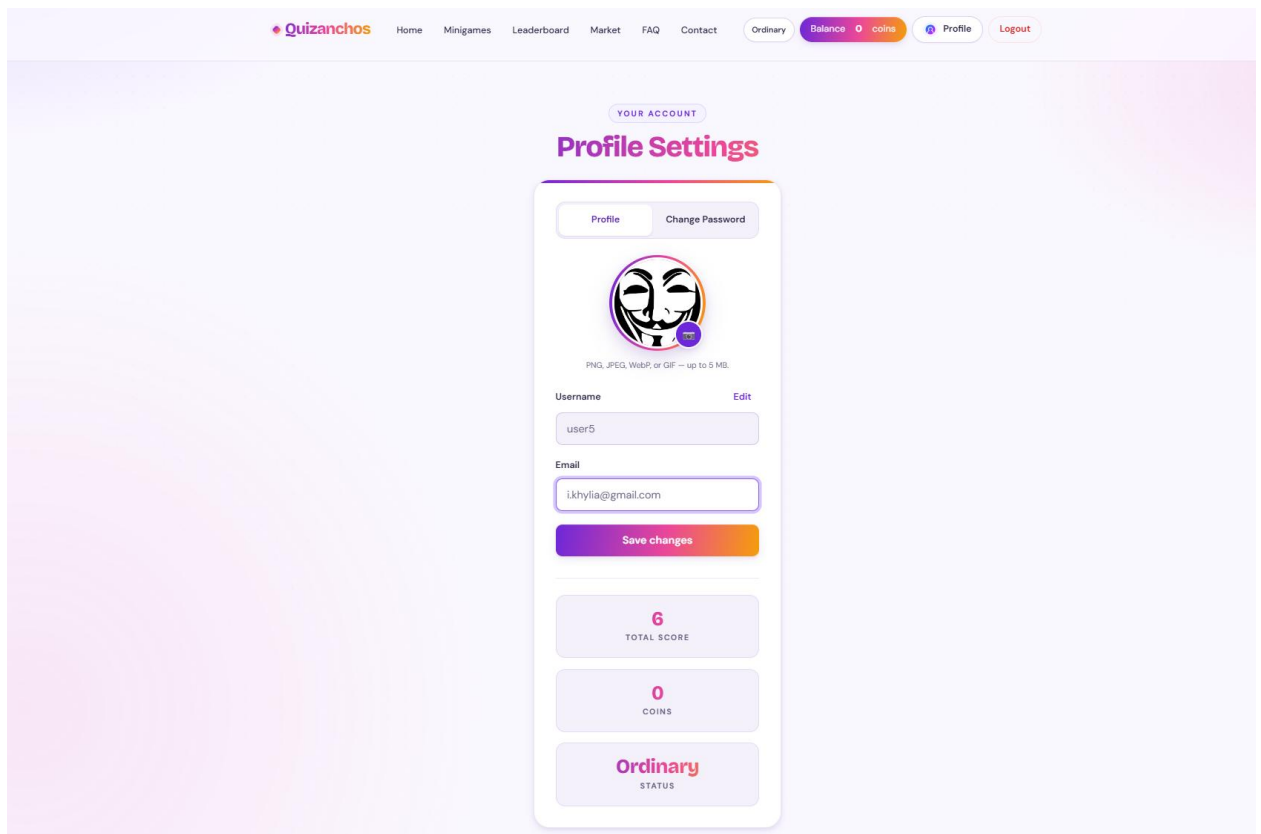


Рисунок 4.10 – Сторінка профілю користувача

На сторінці маркету відображається каталог внутрішньоігрових предметів з можливістю придбання за внутрішньоігрову валюту. Після успішної покупки баланс автоматично зменшується, а предмет потрапляє до інвентарю користувача (рисунок 4.11).

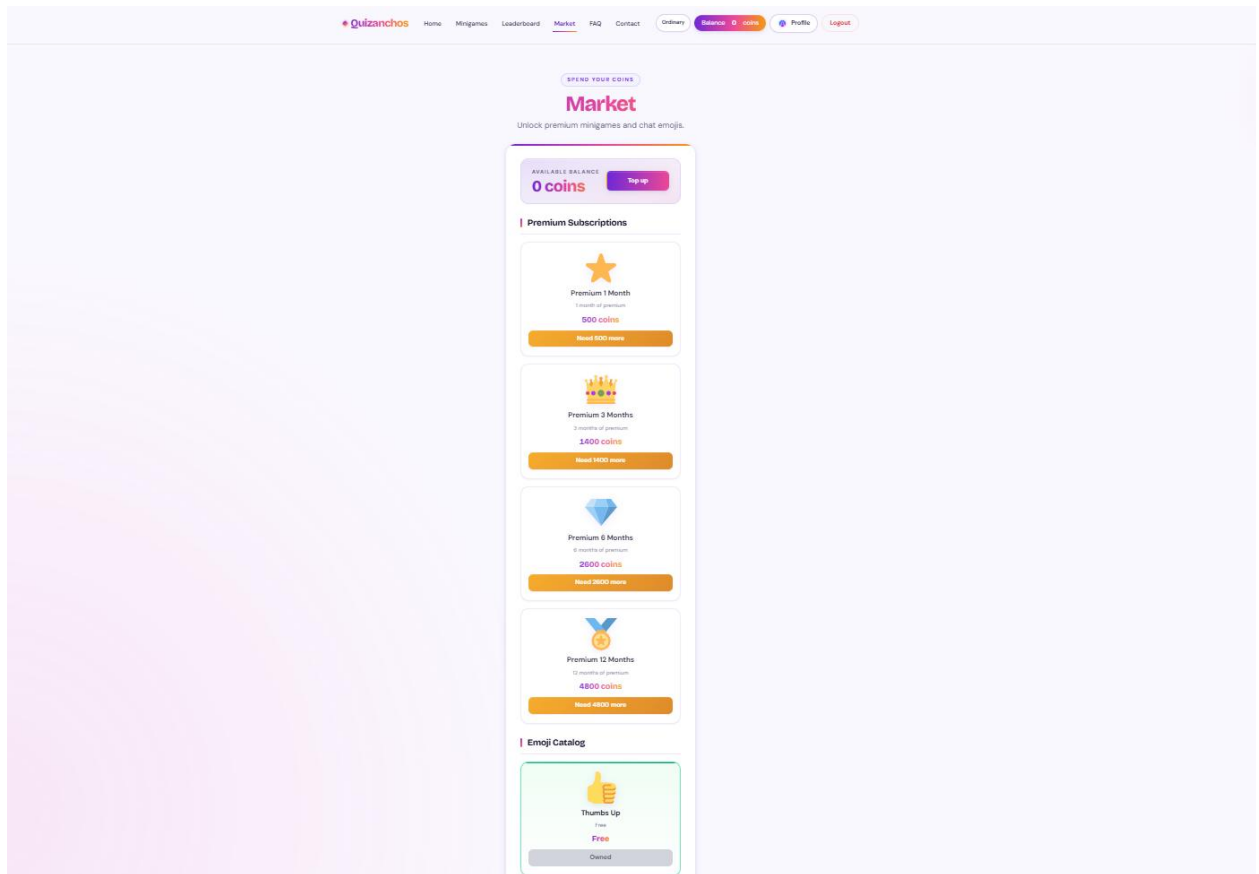


Рисунок 4.11 – Сторінка внутрішньоігрового маркету

Розглянутий контрольний приклад послідовно демонструє роботу всіх основних модулів платформи: обліку користувачів, каталогу мініігор, проходження ігрової сесії, рейтингу, віртуальних активів. Це підтверджує, що поведінка програмного забезпечення відповідає очікуваним сценаріям.

4.3 Висновки до розділу

У четвертому розділі виконано перевірку коректності реалізації сервісів індивідуальної частини програмного забезпечення веб-платформи Quizanchos, у результаті чого підтверджено відповідність розробленого функціоналу сформульованим у другому розділі вимогам та готовність до етапу розгортання.

У підрозділі 4.1 для перевірки правильності реалізації сервісів застосовано дворівневий підхід: автоматизоване модульне тестування на основі фреймворка xUnit.net та мануальне тестування ключових користувацьких сценаріїв у браузері. Розроблено 36 модульних тестів,

згрупованих у вісім тестових класів, що охоплюють життєвий цикл мультиплеєрної кімнати з автоматичним запуском гри, операції внутрішньоігрового маркету з транзакційною купівлею звичайних предметів та преміум підписок, формування рейтингу, контроль преміум доступу, нарахування очок, керування профілем користувача, агреговану статистику адмін панелі та відновлення пароля за одноразовим кодом. Залежності тестованих сервісів підмінюються заглушками Moq, а доступ до бази даних емулюється провайдером `Microsoft.EntityFrameworkCore.InMemory`, що забезпечує детермінованість прогону без піднімання Postgres контейнера. Додатково проведено серію з 20 мануальних тестів, що охоплюють користувацькі сценарії реєстрації та авторизації, відновлення пароля, керування аватаром та профілем, створення мультиплеєрних кімнат, купівлі активів і преміум підписки, а також перегляду таблиці лідерів; усі тестові сценарії виконано успішно, фактичний результат збігається з очікуванням.

У підрозділі 4.2 на основі реалізованих сервісів продемонстровано контрольний приклад, що послідовно розкриває наскрізний сценарій взаємодії користувача з платформою: від першого відкриття головної сторінки та реєстрації облікового запису з підтвердженням електронної пошти, авторизації з видачею cookie сесії, вибору мініігри з каталогу та налаштування параметрів сесії, до проходження ігрового раунду, перегляду підсумкового екрана з нарахуванням очок, оновлення таблиці лідерів з фільтрацією за мінігрою, керування персональним профілем та інвентарем, а також придбання внутрішньоігрових активів у маркеті. Розглянутий приклад охопив усі ключові потоки даних реалізованої частини, а саме автентифікацію, керування профілем, мультиплеєрні кімнати, рейтинг та маркет, і підтвердив, що поведінка розроблених сервісів узгоджена між собою та відповідає очікуванім сценаріям використання.

ВИСНОВКИ

У ході виконання індивідуальної частини дипломного проєкту було реалізовано сервіси веб-платформи Quizanchos, що відповідають за облік користувачів, мультиплеєрну взаємодію в реальному часі, систему рейтингу та обігу віртуальних активів. Реалізована частина побудована на стеку ASP.NET Core 10, Entity Framework Core та PostgreSQL з використанням SignalR для двосторонньої комунікації клієнтів у режимі реального часу.

Поставлену мету щодо забезпечення стабільної та ефективної роботи серверних сервісів платформи шляхом організації узгодженої взаємодії між модулями автентифікації, мультиплеєрних кімнат, рейтингу та внутрішньоігрового маркету досягнуто у повному обсязі. Результати метричного аналізу коду, успішне проходження 36 автоматизованих модульних тестів, 20 мануальних тестів та послідовне виконання сценаріїв контрольного прикладу, наведені у четвертому розділі, підтверджують відповідність розробленої частини усім сформульованим функціональним та нефункціональним вимогам.

Задачу організації мультиплеєрної взаємодії у реальному часі та виконання ігрової логіки мініігор розв'язано шляхом реалізації узагальненого ігрового рушія GameEngine з типобезпечною обгорткою, менеджера кімнат на основі ConcurrentDictionary та двох хабів реального часу. Така архітектура забезпечує однорідне виконання ходів для довільної мініігри, повний життєвий цикл кімнати від створення до автоматичного запуску гри при заповненні останнього місця, а також синхронізацію стану між усіма підключеними клієнтами.

Задачу побудови прозорого механізму обігу віртуальних активів вирішено шляхом реалізації внутрішньоігрового маркету у поєднанні з рейтинговою системою та системою нарахування очок. Цілісність фінансових операцій купівлі звичайних предметів і преміум підписок гарантується транзакційним виконанням засобами Entity Framework Core з відкратом при

помилках, а формування таблиці лідерів та щомісячне нагородження переможців забезпечується фоновим завданням Quartz.

Задачу централізованого обліку користувачів та контролю доступу розв'язано за допомогою ASP.NET Identity з cookie автентифікацією QAuth. Реалізована модель охоплює реєстрацію та авторизацію з підтвердженням електронної пошти, відновлення пароля за одноразовим кодом, керування персональним профілем із валідованим завантаженням аватара у Cloudinary, а також перевірку активної преміум підписки при доступі до преміум мініігор.

Розроблена частина платформи є придатною для подальшої експлуатації, супроводу та розширення новими функціональними можливостями.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Clements P., Bass L., Kazman R. Software Architecture in Practice. Pearson Education, Limited, 2021.
- 2) Kalbach J. Designing Web Navigation: Optimizing the User Experience. O'Reilly Media, Inc., 2007. 456 p.
- 3) GDPR працює: 7 принципів захисту даних інтернет-користувачів. *Mind.ua*. URL: <https://mind.ua/openmind/20256738-gdpr-pracyue-7-principiv-zahistu-danih-internet-koristuvachiv> (дата звернення: 23.04.2026).
- 4) Brute force attacks explained: types, risks, and how to stay safe. *Malwarebytes*. URL: <https://www.malwarebytes.com/cybersecurity/basics/brute-force-attack> (дата звернення: 24.04.2026).
- 5) Galluccio E., Caselli E., Lombardi G. SQL Injection Strategies: Practical Techniques to Secure Old Vulnerabilities Against Modern Attacks. Packt Publishing, Limited, 2020. 210 p.
- 6) What's new in ASP.NET Core in .NET 10. Microsoft Learn: Build with answers in reach. URL: <https://learn.microsoft.com/en-us/aspnet/core/release-notes/aspnetcore-10.0?view=aspnetcore-10.0> (дата звернення: 30.04.2026).
- 7) Image and Video Upload, Storage, Optimization and CDN. *Cloudinary*. URL: <https://cloudinary.com/> (дата звернення: 01.05.2026).
- 8) Home | Quartz.NET. Home | Quartz.NET. URL: <https://www.quartz-scheduler.net/> (дата звернення: 04.05.2026).
- 9) JSON Types. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/current/datatype-json.html> (дата звернення: 10.05.2026).
- 10) Aguilar J. M. SignalR Programming in Microsoft ASP.NET (Developer Reference). Microsoft Press, 2014. 272 c.
- 11) Overview of Entity Framework Core - EF Core. Microsoft Learn: Build with answers in reach. URL: <https://learn.microsoft.com/en-us/ef/core/> (дата звернення: 13.05.2026).

- 12) Unit testing C# code in .NET using dotnet test and xUnit - .NET. Microsoft Learn: Build with answers in reach. URL: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-csharp-with-xunit> (дата звернення: 15.05.2026).
- 13) Moq 4.20.72. NuGet Gallery | Home. URL: <https://www.nuget.org/packages/moq/> (дата звернення: 17.05.2026).
- 14) Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, Incorporated, 2017. 616 p.
- 15) Black R. Pragmatic Software Testing. Indianapolis, Indiana : John Wiley & Sons, Inc., 2016. URL: <https://doi.org/10.1002/9781119419297> (дата звернення: 20.05.2026).
- 16) PostgreSQL: Documentation. PostgreSQL: The world's most advanced open source database. URL: <https://www.postgresql.org/docs/> (дата звернення: 22.05.2026).

ДОДАТОК А ЗВІТ ПОДІБНОСТІ

Метадані

ДОКУМЕНТ

Заголовок

ІП-22_Хиля_ПЗ

Автор

ІП-22_Хиля

Науковий керівник / Експерт

Павлов О.А.

ІД документа

334108828

ОРГАНІЗАЦІЯ

Назва організації

National Technical University of Ukraine Igor Sikorskyi Kyiv
Politech Institute

підрозділ

ФІОТ, К-а інформатики та програмної інженерії

ЗВІТ

Дата звіту

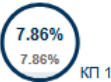
5/31/2026

Дата редагування

6/1/2026

Обсяг знайдених подібностей

Коефіцієнт подібності визначає, який відсоток тексту по відношенню до загального обсягу тексту було знайдено в різних джерелах. Зверніть увагу, що високі значення коефіцієнта не автоматично означають плагіат. Звіт має аналізувати компетентна / уповноважена особа.



11244
Кількість слів

90021
Кількість символів

Тривога

У цьому розділі ви знайдете інформацію щодо текстових спотворень. Ці спотворення в тексті можуть говорити про МОЖЛИВІ маніпуляції в тексті. Спотворення в тексті можуть мати навмисний характер, але частіше характер технічних помилок при конвертації документа та його збереженні, тому ми рекомендуємо вам підходити до аналізу цього модуля відповідально. У разі виникнення запитань, просимо звертатися до нашої служби підтримки.

Заміна букв		2
Інтервали		0
Мікропробіли		10
Білі знаки		0
Парафрази (SmartMarks)	a	52

Джерела

Нижче наведений список джерел. В цьому списку є джерела із різних баз даних. Колір тексту означає в якому джерелі він був знайдений. Ці джерела і значення Коефіцієнту Подібності не відображають прямого плагіату. Необхідно відкрити кожне джерело і проаналізувати зміст і правильність оформлення джерела.

10 найдовших фраз

Колір тексту

#	НАЗВА ТА АДРЕСА ДЖЕРЕЛА URL (НАЗВА БАЗИ)	КОЛЬКОСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
---	--	--

1	Мобільний застосунок для пошуку музичного гурту 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	21 (0.19 %)
2	https://ela.kpi.ua/bitstreams/e9cda8e5-c3a6-498a-993d-4ceff6f6c78a/download	19 (0.17 %)
3	ІП-24_Долобан_ПЗ 5/30/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	17 (0.15 %)
4	ІП-12_Кушнір_ПЗ 6/7/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	15 (0.13 %)
5	https://ela.kpi.ua/bitstreams/e9cda8e5-c3a6-498a-993d-4ceff6f6c78a/download	15 (0.13 %)
6	ІП-15_Чорний_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	13 (0.12 %)
7	ПЗДП_ІСТ_КПІ_2025_Росновська Ольга Олегівна 6/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-па інформаційних систем та технологій)	13 (0.12 %)
8	Високонавантажена система словічень на основі мікросервісів та асинхронного зв'язку 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	13 (0.12 %)
9	ІП-21_Сергієнко_ПЗ 5/30/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	12 (0.11 %)
10	ІП-11_Дякунчак_ПЗ 6/14/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	12 (0.11 %)

Домашня база даних (6.32 %)



#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
1	ІП-12_Кушнір_ПЗ 6/7/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	172 (24) (1.53 %)
2	ПЗДП_ІСТ_КПІ_2025_Росновська Ольга Олегівна 6/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-па інформаційних систем та технологій)	91 (12) (0.81 %)
3	ІП-24_Долобан_ПЗ 5/30/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	71 (8) (0.63 %)

національного технічного університету України Ігор Сікорський Київ Політехнічний інститут (ФІОТ, К-а інформатики та програмної інженерії)

4	ІП-15_Чорний_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	40 (4) (0.36 %)
5	Мобільний застосунок для пошуку музичного гурту 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	38 (3) (0.34 %)
6	ІП-11_Друзенко_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	35 (5) (0.31 %)
7	Високонавантажена система сповіщень на основі мікросервісів та асинхронного зв'язку 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	32 (4) (0.28 %)
8	ІП-21_Поплавський_ПЗ 5/25/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	30 (5) (0.27 %)
9	ІП-21_Серпінко_ПЗ 5/30/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	29 (3) (0.26 %)
10	Вебзастосунок для пошуку репетиторів 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	26 (4) (0.23 %)
11	ІП-15_Дацько_ПЗ 6/2/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	25 (3) (0.22 %)
12	Вебсервіс для перегляду відеоконтенту в жанрі аніме 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	23 (3) (0.2 %)
13	ІП-13_Нещерет_ПЗ 6/3/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	19 (3) (0.17 %)
14	ІП-11_Дякунчак_ПЗ 6/14/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	18 (2) (0.16 %)
15	Вебзастосунок для підтримки роботи бібліотеки 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	17 (3) (0.15 %)
16	Веб-застосунок для відстеження цін на криптоактиви з оповіщенням про зміну ціни 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	14 (2) (0.12 %)

Назва: Інститут енергетики та енергії Ігор Сікорського Київського національного університету України Ігор Сікорський Київський політехнічний інститут (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)		
17	Мобільний застосунок для допомоги художникам 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	12 (2) (0.11 %)
18	ІП-13_Макарчук_ПЗ 5/27/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	8 (1) (0.07 %)
19	ІП-22_Монтицький_ПЗ 5/31/2026 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (ФІОТ, К-а інформатики та програмної інженерії)	6 (1) (0.05 %)
20	Веб-сервіс для генерації та розгортання лендінгів 3/16/2025 National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute (National Technical University of Ukraine Igor Sikorskyi Kyiv Politech Institute)	5 (1) (0.04 %)
Програма обміну базами даних (0.09 %)		
#	ЗАГОЛОВОК	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
21	ФКПІ_2024_121_Кириленко_А.О._ 7/11/2024 Ukrainian national aviation university (Ukrainian national aviation university)	10 (1) (0.09 %)
Інтернет (1.45 %)		
#	ДЖЕРЕЛО URL	КІЛЬКІСТЬ ІДЕНТИЧНИХ СЛІВ (ФРАГМЕНТІВ)
22	https://ela.kpi.ua/bitstreams/e9cda8e5-c3a6-498a-993d-4ceff6f6c78a/download	88 (9) (0.78 %)
23	https://ela.kpi.ua/bitstreams/90b23404-4886-462c-bce3-e2e11e6ab86f/download	35 (5) (0.31 %)
24	https://ela.kpi.ua/bitstreams/3d887913-a7b0-496f-aa44-0e66830f1fdc/download	22 (3) (0.2 %)
25	https://ela.kpi.ua/bitstreams/3c6a001c-9d77-4cb6-b587-e9b2c95ae301/download	18 (3) (0.16 %)
Список прийнятих фрагментів		
#	ЗМІСТ	КІЛЬКІСТЬ ОДНАКОВИХ СЛІВ (ФРАГМЕНТІВ)

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ОНЛАЙН ПЛАТФОРМИ МІНІГОР З
ПІДТРИМКОЮ МУЛЬТИПЛЕЄРНОЇ ВЗАЄМОДІЇ ТА СИСТЕМИ
ВІРТУАЛЬНИХ АКТИВІВ**

Текст програми

КПІ.ПІ-2227.045440.03.12

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

_____ Максим

ГОЛОВЧЕНКО

Виконавець:

_____ Іван ХИЛЯ

Київ – 2026

Посилання на репозиторій з повним текстом програмного коду:
<https://github.com/navarti/Quizanchos>

Лістинг – Quizanchos.Core/GameEngine.cs:

```
using System.Collections.Immutable;

namespace Quizanchos.Core;

public class GameEngine<TState, TMove>
    where TState : IGameState
    {
        private readonly IGameLogic<TState, TMove> _logic;
        public TState State { get; }

        private readonly HashSet<string> _movesReceived = new();

        public GameEngine(
            IGameLogic<TState, TMove> logic,
            Guid gameId,
            ImmutableArray<string> players)
        {
            _logic = logic;
            State = _logic.CreateInitialState(gameId, players);
        }

        public GameEngine(
            IGameLogic<TState, TMove> logic,
            TState existingState)
        {
            _logic = logic;
            State = existingState;
        }

        public MoveResult MakeMove(string playerId, TMove move)
        {
            if (State.IsFinished)
                return MoveResult.Failure("Game is finished");

            var expected = _logic.GetExpectedPlayers(State);
            if (!expected.Contains(playerId))
                return MoveResult.Failure("Player can not commit now");

            MoveResult moveValidationResult = _logic.ValidateMove(State, move, playerId);
            if (!moveValidationResult.IsSuccess)
                return moveValidationResult;

            _logic.ApplyMove(State, move, playerId);
            _movesReceived.Add(playerId);

            // Якщо гра очікувала всіх – чекати
            if (_movesReceived.Count < expected.Count())
                return MoveResult.Success;

            // Усі гравці цього раунду зробили свій хід
            _movesReceived.Clear();

            if (_logic.CheckFinished(State))
            {
                State.IsFinished = true;
            }
        }
    }
}
```

```

        State.Winner = _logic.DetermineWinner(State);
    }

    return MoveResult.Success;
}

public bool NeedToFinish()
{
    if (State.IsFinished)
        return false;

    bool needToFinish = _logic.NeedToFinish(State);
    if (needToFinish)
    {
        State.IsFinished = true;
        State.Winner = _logic.DetermineWinner(State);
    }
    return needToFinish;
}

public IReadOnlyDictionary<string, int> GetPlayerScores()
{
    return _logic.GetPlayerScores(State);
}
}

```

Лістинг – Services/GameLogic/GameEngineWrapper.cs:

```

using Quizanchos.Core;

namespace Quizanchos.WebApi.Services.GameLogic;

public class GameEngineWrapper<TState, TMove> : IGameEngine
    where TState : IGameState
{
    private readonly GameEngine<TState, TMove> _engine;

    public GameEngineWrapper(GameEngine<TState, TMove> engine)
    {
        _engine = engine;
    }

    public Guid GameId => _engine.State.GameId;
    public IReadOnlyList<string> Players => _engine.State.Players;
    public bool IsFinished => _engine.State.IsFinished;
    public string? Winner => _engine.State.Winner;

    public MoveResult MakeMove(string playerId, GameMove move)
    {
        if (move is not TMove typedMove)
        {
            return MoveResult.Failure($"Invalid move type. Expected {typeof(TMove).Name}");
        }

        return _engine.MakeMove(playerId, typedMove);
    }

    public IGameState GetState()
    {
        return _engine.State;
    }

    public bool NeedToFinish()
    {
        return _engine.NeedToFinish();
    }
}

```

```

    }

    public IReadOnlyDictionary<string, int> GetPlayerScores()
    {
        return _engine.GetPlayerScores();
    }
}

```

Лістинг – Services/Rooms/GameRoomService.cs:

```

using Microsoft.AspNetCore.Identity;
using Quizanchos.Core;
using Quizanchos.Domain.Entities;
using Quizanchos.WebApi.Models.Rooms;
using Quizanchos.WebApi.Services.Users;

namespace Quizanchos.WebApi.Services.Rooms;

/// <summary>
/// Orchestrates the game-room lifecycle: create ? join / leave / switch team ? auto-launch.
/// Delegates persistence to <see cref="IGameRoomManager"/> and notifications to <see cref="IRoomNotifier"/>.
/// </summary>
public class GameRoomService
{
    private readonly IGameRoomManager _roomManager;
    private readonly IRoomNotifier _roomNotifier;
    private readonly GameService _gameService;
    private readonly ILogger<GameRoomService> _logger;
    private readonly PremiumAccessService _premiumAccessService;
    private readonly UserManager<ApplicationUser> _userManager;

    public GameRoomService(
        IGameRoomManager roomManager,
        IRoomNotifier roomNotifier,
        GameService gameService,
        ILogger<GameRoomService> logger,
        PremiumAccessService premiumAccessService,
        UserManager<ApplicationUser> userManager)
    {
        _roomManager = roomManager;
        _roomNotifier = roomNotifier;
        _gameService = gameService;
        _logger = logger;
        _premiumAccessService = premiumAccessService;
        _userManager = userManager;
    }

    public async Task<RoomActionResult> CreateRoomAsync(CreateRoomRequest request, string hostPlayerId)
    {
        await _premiumAccessService
            .EnsureUsersCanAccessMinigameAsync([hostPlayerId], request.MinigameType)
            .ConfigureAwait(false);

        var existingRoom = _roomManager.GetRoomByPlayerId(hostPlayerId);
        if (existingRoom != null)
            return RoomActionResult.Error($"Player is already in room {existingRoom.RoomId}");

        if (request.MaxPlayers < 2)
            return RoomActionResult.Error("Multiplayer rooms require at least 2 players");

        if (request.TeamCount < 1)
            return RoomActionResult.Error("At least 1 team is required");

        if (request.MaxPlayers < request.TeamCount)
            return RoomActionResult.Error("Max players cannot be less than team count");
    }
}

```

```

Guid roomId = Guid.NewGuid();
int playersPerTeam = (int)Math.Ceiling((double)request.MaxPlayers / request.TeamCount);

var teams = new List<GameRoomTeam>();
for (int i = 0; i < request.TeamCount; i++)
{
    teams.Add(new GameRoomTeam(i, $"Team {i + 1}", playersPerTeam));
}

var room = _roomManager.CreateRoom(
    roomId, request.MinigameType, hostPlayerId,
    request.MaxPlayers, teams, request.GameParameters);

// Host automatically joins the first team
lock (room.SyncRoot)
{
    room.Teams[0].AddPlayer(hostPlayerId);
}

_logger.LogInformation(
    "Room created: RoomId={RoomId}, Host={HostId}, Type={Type}, MaxPlayers={Max}, Teams={Teams}",
    roomId, hostPlayerId, request.MinigameType, request.MaxPlayers, request.TeamCount);

return RoomActionResult.Success(await MapToDtoAsync(room));
}

public async Task<RoomActionResult> JoinRoomAsync(Guid roomId, int teamIndex, string playerId)
{
    var existingRoom = _roomManager.GetRoomByPlayerId(playerId);
    if (existingRoom != null && existingRoom.RoomId != roomId)
        return RoomActionResult.Error($"Player is already in room {existingRoom.RoomId}");

    var room = _roomManager.GetRoom(roomId);
    if (room == null)
        return RoomActionResult.Error("Room not found");

    await _premiumAccessService
        .EnsureUsersCanAccessMinigameAsync([playerId], room.MinigameType)
        .ConfigureAwait(false);

    bool shouldLaunch;

    lock (room.SyncRoot)
    {
        if (room.IsJoinExpired)
            return RoomActionResult.Error("Room has expired and is no longer accepting players");

        if (room.Status != GameRoomStatus.WaitingForPlayers)
            return RoomActionResult.Error("Room is no longer accepting players");

        if (room.ContainsPlayer(playerId))
            return RoomActionResult.Error("Player is already in this room");

        if (teamIndex < 0 || teamIndex >= room.Teams.Count)
            return RoomActionResult.Error($"Invalid team index: {teamIndex}");

        var team = room.Teams[teamIndex];
        if (!team.AddPlayer(playerId))
            return RoomActionResult.Error("Team is full");

        shouldLaunch = room.IsFull;
        if (shouldLaunch)
    }

```

```

        room.Status = GameRoomStatus.Launching;
    }

    _logger.LogInformation("Player {PlayerId} joined room {RoomId} team {TeamIndex}",
        playerId, roomId, teamIndex);

    await _roomNotifier.NotifyPlayerJoinedRoom(roomId, playerId, teamIndex);
    await _roomNotifier.NotifyRoomUpdated(roomId, await MapToDtoAsync(room));

    if (shouldLaunch)
    {
        await LaunchGameFromRoomAsync(room);
    }

    return RoomActionResult.Success(await MapToDtoAsync(room));
}

public async Task<RoomActionResult> LeaveRoomAsync(Guid roomId, string playerId)
{
    var room = _roomManager.GetRoom(roomId);
    if (room == null)
        return RoomActionResult.Error("Room not found");

    bool hostLeft;

    lock (room.SyncRoot)
    {
        if (room.Status != GameRoomStatus.WaitingForPlayers)
            return RoomActionResult.Error("Cannot leave room in current state");

        if (!room.ContainsPlayer(playerId))
            return RoomActionResult.Error("Player is not in this room");

        foreach (var team in room.Teams)
            team.RemovePlayer(playerId);

        hostLeft = playerId == room.HostPlayerId;
    }

    _logger.LogInformation("Player {PlayerId} left room {RoomId}", playerId, roomId);
    await _roomNotifier.NotifyPlayerLeftRoom(roomId, playerId);

    if (hostLeft)
    {
        await CloseRoomAsync(roomId, "Host left the room");
        return RoomActionResult.Success(await MapToDtoAsync(room));
    }

    await _roomNotifier.NotifyRoomUpdated(roomId, await MapToDtoAsync(room));
    return RoomActionResult.Success(await MapToDtoAsync(room));
}

public async Task<RoomActionResult> SwitchTeamAsync(Guid roomId, int newTeamIndex, string playerId)
{
    var room = _roomManager.GetRoom(roomId);
    if (room == null)
        return RoomActionResult.Error("Room not found");

    lock (room.SyncRoot)
    {
        if (room.Status != GameRoomStatus.WaitingForPlayers)
            return RoomActionResult.Error("Cannot switch teams in current state");
    }
}

```



```

    if (!room.ContainsPlayer(playerId))
        return RoomActionResult.Error("Player is not in this room");

    if (newTeamIndex < 0 || newTeamIndex >= room.Teams.Count)
        return RoomActionResult.Error($"Invalid team index: {newTeamIndex}");

    var newTeam = room.Teams[newTeamIndex];
    if (newTeam.IsFull)
        return RoomActionResult.Error("Target team is full");

    foreach (var team in room.Teams)
        team.RemovePlayer(playerId);

    newTeam.AddPlayer(playerId);
}

_logger.LogInformation("Player {PlayerId} switched to team {TeamIndex} in room {RoomId}",
    playerId, newTeamIndex, roomId);

await _roomNotifier.NotifyRoomUpdated(roomId, await MapToDtoAsync(room));
return RoomActionResult.Success(await MapToDtoAsync(room));
}

public async Task<RoomActionResult> GetRoomAsync(Guid roomId)
{
    var room = _roomManager.GetRoom(roomId);
    if (room == null)
        return RoomActionResult.Error("Room not found");

    return RoomActionResult.Success(await MapToDtoAsync(room));
}

public async Task<IReadOnlyList<GameRoomDto>> GetAvailableRoomsAsync(int? minigameType = null)
{
    var rooms = _roomManager.GetAvailableRooms(minigameType);
    var dtos = new List<GameRoomDto>(rooms.Count);
    foreach (var room in rooms)
    {
        dtos.Add(await MapToDtoAsync(room));
    }
    return dtos;
}

private async Task LaunchGameFromRoomAsync(GameRoom room)
{
    _logger.LogInformation("Launching game from room {RoomId}", room.RoomId);

    try
    {
        var playerIds = room.AllPlayerIds.ToList();

        await _premiumAccessService
            .EnsureUsersCanAccessMinigameAsync(playerIds, room.MinigameType)
            .ConfigureAwait(false);

        var teams = room.Teams
            .Select(t => new TeamInfo(t.TeamIndex, t.Name, t.Players.ToList()))
            .ToList();

        IReadOnlyDictionary<string, string> nicknames =
            await ResolveNicknamesAsync(playerIds);

        var response = await _gameService.CreateMultiPlayerGameAsync(

```

```

        room.MinigameType,
        playerIds,
        room.GameParameters,
        teams,
        nicknames);

    lock (room.SyncRoot)
    {
        room.LaunchedGameId = response.GameId;
        room.Status = GameRoomStatus.GameStarted;
    }

    await _roomNotifier.NotifyGameStarting(room.RoomId, response.GameId);

    _logger.LogInformation("Game {GameId} launched from room {RoomId}",
        response.GameId, room.RoomId);
}
catch (Exception ex)
{
    _logger.LogError(ex, "Failed to launch game from room {RoomId}", room.RoomId);

    lock (room.SyncRoot)
    {
        room.Status = GameRoomStatus.WaitingForPlayers;
    }

    await _roomNotifier.NotifyRoomUpdated(room.RoomId, await MapToDtoAsync(room));
}
}

private async Task CloseRoomAsync(Guid roomId, string reason)
{
    var room = _roomManager.GetRoom(roomId);
    if (room != null)
    {
        lock (room.SyncRoot)
        {
            room.Status = GameRoomStatus.Closed;
        }
    }

    await _roomNotifier.NotifyRoomClosed(roomId, reason);
    _roomManager.RemoveRoom(roomId);

    _logger.LogInformation("Room {RoomId} closed: {Reason}", roomId, reason);
}

internal async Task<GameRoomDto> MapToDtoAsync(GameRoom room)
{
    IReadOnlyDictionary<string, string> nicknames =
        await ResolveNicknamesAsync(room.AllPlayerIds);

    return new GameRoomDto
    {
        RoomId = room.RoomId,
        MinigameType = room.MinigameType,
        HostPlayerId = room.HostPlayerId,
        MaxPlayers = room.MaxPlayers,
        CurrentPlayerCount = room.AllPlayerIds.Count,
        Status = room.Status,
        Teams = room.Teams.Select(t => new GameRoomTeamDto
        {
            TeamIndex = t.TeamIndex,

```

```

        Name = t.Name,
        MaxSize = t.MaxSize,
        Players = t.Players,
        IsFull = t.IsFull
    }).ToList(),
    PlayerNicknames = nicknames,
    CreatedAt = room.CreatedAt,
    ExpiresAtUtc = room.ExpiresAtUtc,
    LaunchedGameId = room.LaunchedGameId
    };
}

private async Task<IReadOnlyDictionary<string, string>> ResolveNicknamesAsync(
    IReadOnlyCollection<string> playerIds)
{
    Dictionary<string, string> result = new(playerIds.Count);
    foreach (string playerId in playerIds.Distinct())
    {
        ApplicationUser? user = await _userManager.FindByIdAsync(playerId);
        result[playerId] = user?.UserName ?? playerId;
    }
    return result;
}
}

```

Лістинг – Services/Market/MarketService.cs:

```

using Microsoft.EntityFrameworkCore;
using Quizanchos.Common.Enums;
using Quizanchos.Common.Util;
using Quizanchos.Domain;
using Quizanchos.Domain.Entities;
using Quizanchos.Domain.Repositories.Interfaces;
using Quizanchos.WebApi.Dto.Market;
using System.Linq.Expressions;

namespace Quizanchos.WebApi.Services.Market;

public class MarketService
{
    private readonly QuizanchosDbContext _dbContext;
    private readonly IMarketItemRepository _marketItemRepository;
    private readonly IUserOwnedItemRepository _userOwnedItemRepository;

    public MarketService(
        QuizanchosDbContext dbContext,
        IMarketItemRepository marketItemRepository,
        IUserOwnedItemRepository userOwnedItemRepository)
    {
        _dbContext = dbContext;
        _marketItemRepository = marketItemRepository;
        _userOwnedItemRepository = userOwnedItemRepository;
    }

    public async Task<IReadOnlyList<MarketCatalogItemDto>> GetCatalog(MarketItemType itemType)
    {
        var orderBy = new Dictionary<Expression<Func<MarketItem, object>>, SortDirection>
        {
            { x => x.IsFree, SortDirection.Descending },
            { x => x.PriceCoins, SortDirection.Ascending },
            { x => x.Name, SortDirection.Ascending }
        };

        var items = await _marketItemRepository
            .Get(whereExpression: x => x.Type == itemType && x.IsActive, orderBy: orderBy, asNoTracking: true)

```

```

        .ToListAsync()
        .ConfigureAwait(false);

return items
    .Select(x => new MarketCatalogItemDto(
        x.Id,
        (int)x.Type,
        x.Name,
        x.ImageUrl,
        x.PriceCoins,
        x.IsFree,
        x.IsActive,
        x.DurationMonths))
    .ToListAsync();
}

public async Task<IReadOnlyList<UserOwnedItemDto>> GetUserOwnership(string userId, MarketItemType
itemType)
{
    if (string.IsNullOrEmpty(userId))
    {
        throw HandledExceptionFactory.CreateNullException(nameof(userId));
    }

    var ownedItems = await _userOwnedItemRepository
        .GetByUserAndTypeAsync(userId, itemType)
        .ConfigureAwait(false);

    return ownedItems
        .Select(x => new UserOwnedItemDto(x.MarketItemId, x.PurchasedAtUtc))
        .ToListAsync();
}

public async Task<MarketPurchaseResultDto> Purchase(string userId, Guid marketItemId)
{
    if (string.IsNullOrEmpty(userId))
    {
        throw HandledExceptionFactory.CreateNullException(nameof(userId));
    }

    if (marketItemId == Guid.Empty)
    {
        throw HandledExceptionFactory.Create("Invalid market item id.");
    }

    await using var transaction = await _dbContext.Database.BeginTransactionAsync().ConfigureAwait(false);

    try
    {
        var marketItem = await _marketItemRepository.FindById(marketItemId).ConfigureAwait(false)
            ?? throw HandledExceptionFactory.CreateIdNotFoundException(marketItemId);

        if (!marketItem.IsActive)
        {
            throw HandledExceptionFactory.Create("This item is not available for purchase.");
        }

        var user = await _dbContext.ApplicationUsers
            .FirstOrDefaultAsync(x => x.Id == userId)
            .ConfigureAwait(false)
            ?? throw HandledExceptionFactory.Create("User was not found.");

        if (marketItem.Type != MarketItemType.PremiumSubscription)

```

```

{
    var existingOwnership = await _userOwnedItemRepository
        .FindByUserAndItemAsync(userId, marketItemId)
        .ConfigureAwait(false);

    if (existingOwnership is not null)
    {
        throw HandledExceptionFactory.Create("Item is already owned.");
    }
}

if (!marketItem.IsFree)
{
    if (user.Coins < marketItem.PriceCoins)
    {
        throw HandledExceptionFactory.Create("Not enough coins.");
    }

    user.Coins -= marketItem.PriceCoins;
}

var purchasedAtUtc = DateTime.UtcNow;

if (marketItem.Type == MarketItemType.PremiumSubscription)
{
    if (marketItem.DurationMonths is null or <= 0)
    {
        throw HandledExceptionFactory.Create("Subscription item is configured incorrectly.");
    }

    DateTime premiumBaseUtc = user.PremiumUntilUtc.HasValue && user.PremiumUntilUtc.Value >
        purchasedAtUtc
        ? user.PremiumUntilUtc.Value
        : purchasedAtUtc;

    user.PremiumUntilUtc = premiumBaseUtc.AddMonths(marketItem.DurationMonths.Value);

    await _dbContext.SaveChangesAsync().ConfigureAwait(false);
    await transaction.CommitAsync().ConfigureAwait(false);

    return new MarketPurchaseResultDto(
        marketItem.Id,
        purchasedAtUtc,
        user.Coins,
        user.PremiumUntilUtc);
}

var ownership = new UserOwnedItem
{
    Id = Guid.NewGuid(),
    ApplicationUserId = userId,
    MarketItemId = marketItem.Id,
    PurchasedAtUtc = purchasedAtUtc
};

await _dbContext.UserOwnedItems.AddAsync(ownership).ConfigureAwait(false);
await _dbContext.SaveChangesAsync().ConfigureAwait(false);

await transaction.CommitAsync().ConfigureAwait(false);

return new MarketPurchaseResultDto(
    ownership.MarketItemId,
    ownership.PurchasedAtUtc,

```

```

        user.Coins,
        user.PremiumUntilUtc);
    }
    catch (DbUpdateException)
    {
        await transaction.RollbackAsync().ConfigureAwait(false);
        throw HandledExceptionFactory.Create("Could not complete purchase.");
    }
    catch
    {
        await transaction.RollbackAsync().ConfigureAwait(false);
        throw;
    }
}

public async Task<MarketCatalogItemDto> GetUsableItemForUser(string userId, Guid marketItemId,
MarketItemType expectedType)
{
    if (string.IsNullOrEmpty(userId))
    {
        throw HandledExceptionFactory.CreateNullException(nameof(userId));
    }

    if (marketItemId == Guid.Empty)
    {
        throw HandledExceptionFactory.Create("Invalid market item id.");
    }

    var marketItem = await _marketItemRepository.FindById(marketItemId).ConfigureAwait(false)
        ?? throw HandledExceptionFactory.CreateIdNotFoundException(marketItemId);

    if (!marketItem.IsActive)
    {
        throw HandledExceptionFactory.Create("This item is not available.");
    }

    if (marketItem.Type != expectedType)
    {
        throw HandledExceptionFactory.Create("Item type is invalid for this action.");
    }

    if (!marketItem.IsFree && marketItem.Type != MarketItemType.PremiumSubscription)
    {
        var existingOwnership = await _userOwnedItemRepository
            .FindByUserAndItemAsync(userId, marketItemId)
            .ConfigureAwait(false);

        if (existingOwnership is null)
        {
            throw HandledExceptionFactory.Create("Item is not owned.");
        }
    }

    return new MarketCatalogItemDto(
        marketItem.Id,
        (int)marketItem.Type,
        marketItem.Name,
        marketItem.ImageUrl,
        marketItem.PriceCoins,
        marketItem.IsFree,
        marketItem.IsActive,
        marketItem.DurationMonths);
}

```

```
}
```

Лістинг – Services/Users/LeaderBoardService.cs:

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Quizanchos.Domain.Entities;
using Quizanchos.WebApi.Constants;
using Quizanchos.WebApi.Dto;
using Quizanchos.Quiz.Util;
using System.Security.Claims;
using System.Linq;
using Quizanchos.Domain;

namespace Quizanchos.WebApi.Services.Users;

public class LeaderBoardService
{
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly UserRetrieverService _userRetrieverService;
    private readonly QuizanchosDbContext _dbContext;
    private readonly Quizanchos.Domain.Repositories.Interfaces.IUserMinigameScoreRepository _scoreRepository;

    public LeaderBoardService(UserManager<ApplicationUser> userManager, UserRetrieverService
userRetrieverService, QuizanchosDbContext dbContext,
Quizanchos.Domain.Repositories.Interfaces.IUserMinigameScoreRepository scoreRepository)
    {
        _userManager = userManager;
        _userRetrieverService = userRetrieverService;
        _dbContext = dbContext;
        _scoreRepository = scoreRepository;
    }

    public async Task<List<ApplicationUserInLeaderBoardDto>> GetLeaderBoardAsync(int take, int skip)
    {
        return await GetLeaderBoardAsync(take, skip, null).ConfigureAwait(false);
    }

    public async Task<List<ApplicationUserInLeaderBoardDto>> GetLeaderBoardAsync(int take, int skip, int?
minigameType)
    {
        var users = await GetAppUsesLeaderBoardAsync(take, skip, minigameType).ConfigureAwait(false);

        var userIds = users.Select(u => u.Id).ToList();
        var scores = (await _scoreRepository.Get(
            whereExpression: s => userIds.Contains(s.ApplicationUserId) && (!minigameType.HasValue || s.MinigameType
== minigameType.Value),
            asNoTracking: true).ToListAsync().ConfigureAwait(false));

        return users.Select((user, position) =>
        {
            int score;
            if (minigameType.HasValue)
            {
                score = scores.FirstOrDefault(s => s.ApplicationUserId == user.Id)?.Score ?? 0;
            }
            else
            {
                score = scores.Where(s => s.ApplicationUserId == user.Id).Sum(s => s.Score);
            }

            return new ApplicationUserInLeaderBoardDto(user.UserName!, user.AvatarUrl, score, user.Coins, position + 1,
user.Status);
        }).ToList();
    }
}
```

```

public async Task<List<ApplicationUser>> GetAppUsesLeaderBoardAsync(int take, int skip)
{
    return await GetAppUsesLeaderBoardAsync(take, skip, null).ConfigureAwait(false);
}

public async Task<List<ApplicationUser>> GetAppUsesLeaderBoardAsync(int take, int skip, int? minigameType)
{
    SkipTakeValidator.Validate(skip, take);

    HashSet<string> excludedUserIds = await GetExcludedUserIdsAsync().ConfigureAwait(false);

    // Load users and scores into memory, then order in-memory to support per-minigame ranking
    var users = (await _dbContext.Users.ToListAsync()).ConfigureAwait(false)
        .Where(u => !excludedUserIds.Contains(u.Id))
        .ToList();
    var scores = await _dbContext.UserMinigameScores
        .Where(s => !minigameType.HasValue || s.MinigameType == minigameType.Value)
        .ToListAsync()
        .ConfigureAwait(false);

    IEnumerable<ApplicationUser> ordered = minigameType.HasValue
        ? users.OrderByDescending(u => scores.FirstOrDefault(s => s.ApplicationUserId == u.Id)?.Score ?? 0)
        : users.OrderByDescending(u => scores.Where(s => s.ApplicationUserId == u.Id).Sum(s => s.Score));

    return ordered.Skip(skip).Take(take).ToList();
}

public async Task<HashSet<string>> GetExcludedUserIdsAsync()
{
    IList<ApplicationUser> owners = await
    _userManager.GetUsersInRoleAsync(AppRole.Owner).ConfigureAwait(false);
    return owners.Select(o => o.Id).ToHashSet(StringComparer.Ordinal);
}

public async Task<ApplicationUserInLeaderBoardDto> GetUserPositionAsync(ClaimsPrincipal claimsPrincipal)
{
    return await GetUserPositionAsync(claimsPrincipal, null).ConfigureAwait(false);
}

public async Task<ApplicationUserInLeaderBoardDto> GetUserPositionAsync(ClaimsPrincipal claimsPrincipal,
int? minigameType)
{
    ApplicationUser user = await _userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);
    HashSet<string> excludedUserIds = await GetExcludedUserIdsAsync().ConfigureAwait(false);

    // Use DB scores to order
    var usersWithScores = (await _dbContext.Users
        .Select(u => new
        {
            User = u,
            Scores = _dbContext.UserMinigameScores.Where(s => s.ApplicationUserId == u.Id)
        })
        .ToListAsync())
        .Where(x => !excludedUserIds.Contains(x.User.Id))
        .ToList();

    var ordered = minigameType.HasValue
        ? usersWithScores.OrderByDescending(x => x.Scores.FirstOrDefault(s => s.MinigameType ==
minigameType.Value)?.Score ?? 0).Select(x => x.User).ToList()
        : usersWithScores.OrderByDescending(x => x.Scores.Sum(s => s.Score)).Select(x => x.User).ToList();

    int position = ordered.FindIndex(u => u.Id == user.Id);

```



```

// compute user's score from DB
var userScores = await _dbContext.UserMinigameScores
    .Where(s => s.ApplicationUserId == user.Id && (!minigameType.HasValue || s.MinigameType ==
minigameType.Value))
    .ToListAsync()
    .ConfigureAwait(false);

var score = minigameType.HasValue ? userScores.FirstOrDefault()?.Score ?? 0 : userScores.Sum(s => s.Score);

return new ApplicationUserInLeaderBoardDto(user.UserName!, user.AvatarUrl, score, user.Coins, position + 1,
user.Status);
}

}

```

Лістинг – Services/Users/PremiumAccessService.cs:

```

using Microsoft.EntityFrameworkCore;
using Quizanchos.Core;
using Quizanchos.Domain;

namespace Quizanchos.WebApi.Services.Users;

public class PremiumAccessService
{
    private const string PremiumRequiredMessage = "You need an active premium subscription to play this minigame.";

    private readonly QuizanchosDbContext _dbContext;
    private readonly IMinigameRegistry _minigameRegistry;

    public PremiumAccessService(QuizanchosDbContext dbContext, IMinigameRegistry minigameRegistry)
    {
        _dbContext = dbContext;
        _minigameRegistry = minigameRegistry;
    }

    public async Task EnsureUsersCanAccessMinigameAsync(IEnumerable<string> userIds, int minigameType)
    {
        var descriptor = _minigameRegistry.GetDescriptor(minigameType);
        if (descriptor?.IsPremium != true)
        {
            return;
        }

        var uniqueUserIds = userIds
            .Where(x => !string.IsNullOrEmpty(x))
            .Distinct(StringComparer.Ordinal)
            .ToList();

        if (uniqueUserIds.Count == 0)
        {
            throw new UnauthorizedAccessException(PremiumRequiredMessage);
        }

        DateTime nowUtc = DateTime.UtcNow;

        bool hasNonPremiumUser = await _dbContext.ApplicationUsers
            .AsNoTracking()
            .Where(x => uniqueUserIds.Contains(x.Id))
            .AnyAsync(x => !x.PremiumUntilUtc.HasValue || x.PremiumUntilUtc <= nowUtc)
            .ConfigureAwait(false);

        if (hasNonPremiumUser)
        {

```

```

        throw new UnauthorizedAccessException(PremiumRequiredMessage);
    }

    int matchedUsersCount = await _dbContext.ApplicationUsers
        .AsNoTracking()
        .CountAsync(x => uniqueUserIds.Contains(x.Id))
        .ConfigureAwait(false);

    if (matchedUsersCount != uniqueUserIds.Count)
    {
        throw new UnauthorizedAccessException(PremiumRequiredMessage);
    }
}

```

Лістинг – Services/Users/UserScoreService.cs:

```

using Quizanchos.Domain.Entities;
using Quizanchos.Domain.Repositories.Interfaces;

namespace Quizanchos.WebApi.Services.Users;

/// <summary>
/// Service for managing user scores per minigame type.
/// Follows repository pattern for CRUD operations with UserMinigameScore entity.
/// </summary>
public class UserScoreService
{
    private readonly IUserMinigameScoreRepository _scoreRepository;

    public UserScoreService(IUserMinigameScoreRepository scoreRepository)
    {
        _scoreRepository = scoreRepository;
    }

    /// <summary>
    /// Increment or set user's score for a specific minigame type.
    /// </summary>
    public async Task IncrementScoreAsync(string userId, int minigameType, int points)
    {
        if (points <= 0)
            return;

        var existingScore = await _scoreRepository.FindByUserAndTypeAsync(userId,
            minigameType).ConfigureAwait(false);

        if (existingScore != null)
        {
            // Update existing score
            existingScore.Score += points;
            await _scoreRepository.Update(existingScore).ConfigureAwait(false);
        }
        else
        {
            // Create new score record
            var newScore = new UserMinigameScore
            {
                Id = Guid.NewGuid(),
                ApplicationUserId = userId,
                MinigameType = minigameType,
                Score = points
            };
            await _scoreRepository.Create(newScore).ConfigureAwait(false);
        }
    }
}

```

```

/// <summary>
/// Get total score for a user across all minigames.
/// </summary>
public async Task<int> GetTotalScoreAsync(string userId)
{
    var scores = await _scoreRepository.GetByFilter(s => s.ApplicationUserId == userId).ConfigureAwait(false);
    return scores.Sum(s => s.Score);
}

/// <summary>
/// Get score for a specific minigame.
/// </summary>
public async Task<int> GetScoreAsync(string userId, int minigameType)
{
    var score = await _scoreRepository.FindByUserAndTypeAsync(userId, minigameType).ConfigureAwait(false);
    return score?.Score ?? 0;
}
}

```

Лістинг – Services/Users/UserProfileService.cs:

```

using CloudinaryDotNet.Actions;
using CloudinaryDotNet;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Quizanchos.Common.Util;
using Quizanchos.Domain;
using Quizanchos.Domain.Entities;
using Quizanchos.WebApi.Dto;
using System.Security.Claims;

namespace Quizanchos.WebApi.Services.Users;

public class UserProfileService
{
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly SignInManager<ApplicationUser> _signInManager;
    private readonly UserRetrieverService _userRetrieverService;
    private readonly ICloudinary _cloudinary;
    private readonly UserScoreService _userScoreService;
    private readonly QuizanchosDbContext _dbContext;

    public UserProfileService(
        UserManager<ApplicationUser> userManager,
        SignInManager<ApplicationUser> signInManager,
        UserRetrieverService userRetrieverService,
        ICloudinary cloudinary,
        UserScoreService userScoreService,
        QuizanchosDbContext dbContext)
    {
        _userManager = userManager;
        _signInManager = signInManager;
        _userRetrieverService = userRetrieverService;
        _cloudinary = cloudinary;
        _userScoreService = userScoreService;
        _dbContext = dbContext;
    }

    public async Task<FullApplicationUserDto> GetUserInfo(ClaimsPrincipal claimsPrincipal)
    {
        ApplicationUser user = await _userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);
        int totalScore = await _userScoreService.GetTotalScoreAsync(user.Id).ConfigureAwait(false);
        return new FullApplicationUserDto(user.Email!, user.UserName!, user.AvatarUrl, totalScore, user.Coins,
            user.Status, user.PremiumUntilUtc);
    }
}

```

```

}

public async Task UpdateNickname(ClaimsPrincipal claimsPrincipal, string nickNameToUpdate)
{
    ApplicationUser user = await _userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);
    if(user.UserName == nickNameToUpdate)
    {
        return;
    }

    ApplicationUser? userWithNickname = await _userManager.FindByNameAsync(nickNameToUpdate);
    if(userWithNickname is not null)
    {
        throw HandledExceptionFactory.Create("The user with this nickname exists");
    }

    user.UserName = nickNameToUpdate;
    await _userManager.UpdateAsync(user).ConfigureAwait(false);
}

public async Task UpdateAvatarUrl(ClaimsPrincipal claimsPrincipal, string avatarUrl)
{
    ApplicationUser user = await _userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);
    user.AvatarUrl = avatarUrl;
    await _userManager.UpdateAsync(user).ConfigureAwait(false);
}

private const long MaxAvatarSizeBytes = 5L * 1024 * 1024;

private static readonly HashSet<string> AllowedAvatarContentTypes = new(StringComparer.OrdinalIgnoreCase)
{
    "image/png", "image/jpeg", "image/webp", "image/gif"
};

private static readonly HashSet<string> AllowedAvatarExtensions = new(StringComparer.OrdinalIgnoreCase)
{
    ".png", ".jpg", ".jpeg", ".webp", ".gif"
};

public async Task<string> UpdateAvatar(ClaimsPrincipal claimsPrincipal, IFormFile formFile)
{
    ApplicationUser user = await _userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);

    if (formFile is null || formFile.Length == 0)
        throw HandledExceptionFactory.Create("No file uploaded.");

    if (formFile.Length > MaxAvatarSizeBytes)
        throw HandledExceptionFactory.Create("Avatar must be 5 MB or smaller.");

    if (!AllowedAvatarContentTypes.Contains(formFile.ContentType ?? string.Empty))
        throw HandledExceptionFactory.Create("Unsupported image format. Use PNG, JPEG, WebP, or GIF.");

    string extension = Path.GetExtension(formFile.FileName ?? string.Empty);
    if (!AllowedAvatarExtensions.Contains(extension))
        throw HandledExceptionFactory.Create("Unsupported image format. Use PNG, JPEG, WebP, or GIF.");

    using Stream stream = formFile.OpenReadStream();
    ImageUploadParams uploadParams = new ImageUploadParams
    {
        File = new FileDescription(formFile.FileName, stream),
        Transformation = new Transformation().Width(512).Height(512).Crop("limit")
    };
};

```

```

ImageUploadResult uploadResult = await _cloudinary.UploadAsync(uploadParams);

if (uploadResult?.StatusCode != System.Net.HttpStatusCode.OK)
    throw HandledExceptionFactory.Create("Error uploading image.");

user.AvatarUrl = uploadResult.SecureUrl.AbsoluteUri;
await _userManager.UpdateAsync(user).ConfigureAwait(false);
return user.AvatarUrl;
}

public async Task AddCoins(ClaimsPrincipal claimsPrincipal, int coinsToAdd)
{
    if (coinsToAdd <= 0)
    {
        throw HandledExceptionFactory.Create("Coins to add must be greater than zero");
    }

    ApplicationUser user = await _userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);
    user.Coins += coinsToAdd;
    await _userManager.UpdateAsync(user).ConfigureAwait(false);
}

public async Task<UserDataExportDto> ExportDataAsync(ClaimsPrincipal claimsPrincipal)
{
    ApplicationUser user = await _userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);

    var scores = await _dbContext.UserMinigameScores
        .Where(s => s.ApplicationUserId == user.Id)
        .Select(s => new MinigameScoreExportDto(s.MinigameType, s.Score))
        .ToListAsync()
        .ConfigureAwait(false);

    var ownedItems = await _dbContext.UserOwnedItems
        .Where(o => o.ApplicationUserId == user.Id)
        .Select(o => new OwnedItemExportDto(
            o.Id,
            o.MarketItemId,
            o.MarketItem.Name,
            o.MarketItem.Type,
            o.PurchasedAtUtc))
        .ToListAsync()
        .ConfigureAwait(false);

    var gameSessions = await _dbContext.GameSessionPlayers
        .Where(p => p.ApplicationUserId == user.Id)
        .Select(p => new GameSessionExportDto(
            p.GameSession.Id,
            p.GameSession.MinigameType,
            p.GameSession.CreatedAt,
            p.GameSession.FinishedAt,
            p.GameSession.IsFinished,
            p.GameSession.WinnerId == user.Id,
            p.JoinedAt))
        .ToListAsync()
        .ConfigureAwait(false);

    var topUpOrders = await _dbContext.TopUpOrders
        .Where(o => o.ApplicationUserId == user.Id)
        .Select(o => new TopUpOrderExportDto(
            o.Id,
            o.Status,
            o.CoinsToCredit,
            o.AmountUSDT,

```

```

        o.Network,
        o.CreatedAtUtc,
        o.CompletedAtUtc,
        o.BinanceTxId))
        .ToListAsync()
        .ConfigureAwait(false);

var profile = new UserProfileExportDto(
    user.Id,
    user.Email ?? string.Empty,
    user.UserName ?? string.Empty,
    user.AvatarUrl,
    user.Coins,
    user.Status,
    user.PremiumUntilUtc,
    user.EmailConfirmed);

return new UserDataExportDto(profile, scores, ownedItems, gameSessions, topUpOrders);
}

public async Task DeleteAccountAsync(ClaimsPrincipal claimsPrincipal, string currentPassword)
{
    ApplicationUser user = await _userRetrieverService.GetUserByClaims(claimsPrincipal).ConfigureAwait(false);

    if (!await _userManager.CheckPasswordAsync(user, currentPassword).ConfigureAwait(false))
    {
        throw HandledExceptionFactory.Create("Current password is incorrect.");
    }

    // TopUpOrder rows are configured with onDelete(Restrict) to preserve financial
    // history. Surface a clear message rather than letting EF throw a foreign-key
    // exception. Users with order history must contact support for deletion.
    bool hasOrders = await _dbContext.TopUpOrders
        .AnyAsync(o => o.ApplicationUserId == user.Id)
        .ConfigureAwait(false);
    if (hasOrders)
    {
        throw HandledExceptionFactory.Create(
            "Cannot delete account with order history. Please contact support.");
    }

    IdentityResult result = await _userManager.DeleteAsync(user).ConfigureAwait(false);
    if (!result.Succeeded)
    {
        throw HandledExceptionFactory.Create(string.Concat(result.Errors.Select(e => e.Description)));
    }

    await _signInManager.SignOutAsync().ConfigureAwait(false);
}
}

```

Лістинг – Services/Users/StatisticsService.cs:

```

using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using Quizanchos.Common.Util;
using Quizanchos.Core;
using Quizanchos.Domain;
using Quizanchos.Domain.Entities;
using Quizanchos.WebApi.Dto.Statistics;
using Quizanchos.WebApi.Services.Rooms;

namespace Quizanchos.WebApi.Services.Users;

/// <summary>

```

```

/// Aggregates platform usage data for the admin statistics dashboard.
/// All time arguments are interpreted as UTC.
/// </summary>
public class StatisticsService
{
    public enum BucketKind { Hour, Day, Week, Month }

    private const int MaxBuckets = 366;

    private readonly QuizanchosDbContext _db;
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly IMinigameRegistry _registry;
    private readonly IGameRoomManager _roomManager;

    public StatisticsService(
        QuizanchosDbContext db,
        UserManager<ApplicationUser> userManager,
        IMinigameRegistry registry,
        IGameRoomManager roomManager)
    {
        _db = db;
        _userManager = userManager;
        _registry = registry;
        _roomManager = roomManager;
    }

    public IReadOnlyList<StatisticsGameInfoDto> GetRegisteredGames()
    {
        return _registry.GetAllDescriptors().Values
            .OrderBy(d => d.MinigameTypeId)
            .Select(d => new StatisticsGameInfoDto(d.MinigameTypeId, d.GameKey, d.DisplayName))
            .ToList();
    }

    public async Task<StatisticsOverviewDto> GetOverviewAsync(DateTime fromUtc, DateTime toUtc)
    {
        ValidateRange(fromUtc, toUtc);

        var inRange = _db.GameSessions.Where(s => s.CreatedAt >= fromUtc && s.CreatedAt < toUtc);

        int totalSessions = await inRange.CountAsync().ConfigureAwait(false);
        int finishedSessions = await inRange.CountAsync(s => s.IsFinished).ConfigureAwait(false);

        int distinctPlayers = await _db.GameSessionPlayers
            .Where(p => p.GameSession.CreatedAt >= fromUtc && p.GameSession.CreatedAt < toUtc)
            .Select(p => p.ApplicationUserId)
            .Distinct()
            .CountAsync()
            .ConfigureAwait(false);

        // Identity users don't carry a created-at column, so approximate "new users in range"
        // as users whose first-ever session join falls inside the range. Off by people who
        // registered but never played, but a reasonable proxy for engagement.
        int newUsers = await _db.GameSessionPlayers
            .GroupBy(p => p.ApplicationUserId)
            .Select(g => g.Min(p => p.JoinedAt))
            .CountAsync(firstJoin => firstJoin >= fromUtc && firstJoin < toUtc)
            .ConfigureAwait(false);

        // Compute average session duration client-side because Npgsql doesn't expose
        // EF.Functions.DateDiffSecond. The set of finished sessions in any reasonable
        // admin range is small enough to materialize.
        var finishedTimes = await inRange

```

```

        .Where(s => s.IsFinished && s.FinishedAt != null)
        .Select(s => new { s.CreatedAt, s.FinishedAt })
        .ToListAsync()
        .ConfigureAwait(false);

double avgDuration = 0;
if (finishedTimes.Count > 0)
{
    var positive = finishedTimes
        .Select(x => (x.FinishedAt!.Value - x.CreatedAt).TotalSeconds)
        .Where(s => s > 0)
        .ToList();
    avgDuration = positive.Count > 0 ? positive.Average() : 0;
}

double avgConcurrent = await ComputeAverageConcurrentSessionsAsync(fromUtc,
toUtc).ConfigureAwait(false);

double completionRate = totalSessions > 0
    ? Math.Round(100.0 * finishedSessions / totalSessions, 1)
    : 0;

int liveActiveSessions = await _db.GameSessions
    .CountAsync(s => s.IsActive && !s.IsFinished)
    .ConfigureAwait(false);

var lobbyRooms = _roomManager.GetAvailableRooms();
int liveLobbyRooms = lobbyRooms.Count;
int liveLobbyPlayers = lobbyRooms.Sum(r => r.AllPlayerIds.Count);

int totalUsers = await _userManager.Users.CountAsync().ConfigureAwait(false);
int totalSessionsAllTime = await _db.GameSessions.CountAsync().ConfigureAwait(false);

return new StatisticsOverviewDto(
    fromUtc,
    toUtc,
    totalSessions,
    finishedSessions,
    distinctPlayers,
    newUsers,
    Math.Round(avgConcurrent, 2),
    Math.Round(avgDuration, 1),
    completionRate,
    liveActiveSessions,
    liveLobbyRooms,
    liveLobbyPlayers,
    totalUsers,
    totalSessionsAllTime);
}

public async Task<List<SessionsByGameDto>> GetSessionsByGameAsync(DateTime fromUtc, DateTime toUtc)
{
    ValidateRange(fromUtc, toUtc);

    var inRange = _db.GameSessions.Where(s => s.CreatedAt >= fromUtc && s.CreatedAt < toUtc);

    var perGameRaw = await inRange
        .GroupBy(s => s.MinigameType)
        .Select(g => new
        {
            MinigameType = g.Key,
            Total = g.Count(),
            Finished = g.Count(s => s.IsFinished),

```



```

    })
    .ToListAsync()
    .ConfigureAwait(false);

var finishedTimesByGame = await inRange
    .Where(s => s.IsFinished && s.FinishedAt != null)
    .Select(s => new { s.MinigameType, s.CreatedAt, s.FinishedAt })
    .ToListAsync()
    .ConfigureAwait(false);

var avgDurationByGame = finishedTimesByGame
    .GroupBy(x => x.MinigameType)
    .ToDictionary(
        g => g.Key,
        g =>
        {
            var positive = g.Select(x => (x.FinishedAt!.Value - x.CreatedAt).TotalSeconds)
                .Where(s => s > 0)
                .ToList();
            return positive.Count > 0 ? positive.Average() : 0;
        });

var distinctPlayersByGame = await _db.GameSessionPlayers
    .Where(p => p.GameSession.CreatedAt >= fromUtc && p.GameSession.CreatedAt < toUtc)
    .GroupBy(p => p.GameSession.MinigameType)
    .Select(g => new { MinigameType = g.Key, Players = g.Select(x => x.ApplicationUserId).Distinct().Count() })
    .ToListAsync()
    .ConfigureAwait(false);

var liveByGame = await _db.GameSessions
    .Where(s => s.IsActive && !s.IsFinished)
    .GroupBy(s => s.MinigameType)
    .Select(g => new { MinigameType = g.Key, Live = g.Count() })
    .ToListAsync()
    .ConfigureAwait(false);

int total = perGameRaw.Sum(x => x.Total);
var descriptors = _registry.GetAllDescriptors().Values.ToDictionary(d => d.MinigameTypeId);

return perGameRaw
    .Select(row =>
    {
        descriptors.TryGetValue(row.MinigameType, out var descriptor);
        int distinct = distinctPlayersByGame.FirstOrDefault(x => x.MinigameType == row.MinigameType)?.Players ??
0;
        int live = liveByGame.FirstOrDefault(x => x.MinigameType == row.MinigameType)?.Live ?? 0;
        double share = total > 0 ? Math.Round(100.0 * row.Total / total, 1) : 0;
        avgDurationByGame.TryGetValue(row.MinigameType, out double avg);
        return new SessionsByGameDto(
            row.MinigameType,
            descriptor?.GameKey ?? $"unknown-{row.MinigameType}",
            descriptor?.DisplayName ?? $"Unknown ({row.MinigameType})",
            row.Total,
            row.Finished,
            distinct,
            live,
            Math.Round(avg, 1),
            share);
    })
    .OrderByDescending(x => x.TotalSessions)
    .ToList();
}

```

```

public async Task<SessionsTimeSeriesDto> GetSessionsTimeSeriesAsync(
    DateTime fromUtc,
    DateTime toUtc,
    BucketKind bucket,
    int? minigameType)
{
    ValidateRange(fromUtc, toUtc);

    var buckets = BuildBuckets(fromUtc, toUtc, bucket);
    if (buckets.Count == 0)
    {
        return new SessionsTimeSeriesDto(fromUtc, toUtc, bucket.ToString(), buckets, new());
    }

    var query = _db.GameSessions.Where(s => s.CreatedAt >= fromUtc && s.CreatedAt < toUtc);
    if (minigameType.HasValue)
    {
        query = query.Where(s => s.MinigameType == minigameType.Value);
    }

    var rows = await query
        .Select(s => new { s.MinigameType, s.CreatedAt })
        .ToListAsync()
        .ConfigureAwait(false);

    var descriptors = _registry.GetAllDescriptors().Values.ToDictionary(d => d.MinigameTypeId);

    var grouped = rows
        .GroupBy(r => r.MinigameType)
        .Select(group =>
        {
            descriptors.TryGetValue(group.Key, out var descriptor);
            var counts = buckets.ToDictionary(b => b, _ => 0);
            foreach (var row in group)
            {
                var key = AlignToBucket(row.CreatedAt, bucket);
                if (counts.ContainsKey(key))
                {
                    counts[key]++;
                }
            }
        })
        .return new GameTimeSeriesDto(
            group.Key,
            descriptor?.GameKey ?? $"unknown-{group.Key}",
            descriptor?.DisplayName ?? $"Unknown ({group.Key})",
            buckets.Select(b => new TimeSeriesPointDto(b, counts[b])).ToList());
        })
        .OrderByDescending(s => s.Points.Sum(p => p.Count))
        .ToList();

    return new SessionsTimeSeriesDto(fromUtc, toUtc, bucket.ToString(), buckets, grouped);
}

public async Task<List<TopPlayerDto>> GetTopPlayersAsync(DateTime fromUtc, DateTime toUtc, int limit)
{
    ValidateRange(fromUtc, toUtc);
    if (limit <= 0 || limit > 100) limit = 10;

    var topUserIds = await _db.GameSessionPlayers
        .Where(p => p.GameSession.CreatedAt >= fromUtc && p.GameSession.CreatedAt < toUtc)
        .GroupBy(p => p.ApplicationUserId)
        .Select(g => new
        {

```

```

        UserId = g.Key,
        Sessions = g.Count(),
        Wins = g.Count(p => p.GameSession.WinnerId == p.ApplicationUserId),
    })
    .OrderByDescending(x => x.Sessions)
    .Take(limit)
    .ToListAsync()
    .ConfigureAwait(false);

if (topUserIds.Count == 0) return new();

var userIds = topUserIds.Select(x => x.UserId).ToList();

var users = await _userManager.Users
    .Where(u => userIds.Contains(u.Id))
    .Select(u => new { u.Id, u.UserName, u.AvatarUrl })
    .ToListAsync()
    .ConfigureAwait(false);

var totalScores = await _db.UserMinigameScores
    .Where(s => userIds.Contains(s.ApplicationUserId))
    .GroupBy(s => s.ApplicationUserId)
    .Select(g => new { UserId = g.Key, Total = g.Sum(s => s.Score) })
    .ToListAsync()
    .ConfigureAwait(false);

return topUserIds
    .Select(row =>
    {
        var user = users.FirstOrDefault(u => u.Id == row.UserId);
        int score = totalScores.FirstOrDefault(s => s.UserId == row.UserId)?.Total ?? 0;
        return new TopPlayerDto(
            user?.UserName ?? "(deleted)",
            user?.AvatarUrl ?? "",
            row.Sessions,
            row.Wins,
            score);
    })
    .ToList();
}

/// <summary>
/// Average concurrent active sessions over the range = sum of seconds each session
/// was alive within [fromUtc, toUtc] divided by the range duration.
/// </summary>
private async Task<double> ComputeAverageConcurrentSessionsAsync(DateTime fromUtc, DateTime toUtc)
{
    var rangeSeconds = (toUtc - fromUtc).TotalSeconds;
    if (rangeSeconds <= 0) return 0;

    DateTime nowUtc = DateTime.UtcNow;
    DateTime cap = toUtc < nowUtc ? toUtc : nowUtc;

    var overlaps = await _db.GameSessions
        .Where(s => s.CreatedAt < toUtc
            && (s.FinishedAt == null || s.FinishedAt > fromUtc))
        .Select(s => new
        {
            s.CreatedAt,
            s.FinishedAt,
        })
        .ToListAsync()
        .ConfigureAwait(false);

```

```

double totalCoverageSeconds = 0;
foreach (var s in overlaps)
{
    DateTime start = s.CreatedAt < fromUtc ? fromUtc : s.CreatedAt;
    DateTime end = s.FinishedAt ?? cap;
    if (end > toUtc) end = toUtc;
    if (end > start)
    {
        totalCoverageSeconds += (end - start).TotalSeconds;
    }
}

return totalCoverageSeconds / rangeSeconds;
}

private static void ValidateRange(DateTime fromUtc, DateTime toUtc)
{
    if (toUtc <= fromUtc)
    {
        throw HandledExceptionFactory.Create("End date must be after start date.");
    }
    if ((toUtc - fromUtc).TotalDays > 400)
    {
        throw HandledExceptionFactory.Create("Range must be at most 400 days.");
    }
}

private static List<DateTime> BuildBuckets(DateTime fromUtc, DateTime toUtc, BucketKind bucket)
{
    var start = AlignToBucket(fromUtc, bucket);
    var result = new List<DateTime>();
    var cursor = start;
    while (cursor < toUtc && result.Count < MaxBuckets)
    {
        result.Add(cursor);
        cursor = bucket switch
        {
            BucketKind.Hour => cursor.AddHours(1),
            BucketKind.Day => cursor.AddDays(1),
            BucketKind.Week => cursor.AddDays(7),
            BucketKind.Month => cursor.AddMonths(1),
            _ => cursor.AddDays(1),
        };
    }
    return result;
}

private static DateTime AlignToBucket(DateTime value, BucketKind bucket) => bucket switch
{
    BucketKind.Hour => new DateTime(value.Year, value.Month, value.Day, value.Hour, 0, 0, value.Kind),
    BucketKind.Day => new DateTime(value.Year, value.Month, value.Day, 0, 0, 0, value.Kind),
    BucketKind.Week => StartOfWeek(value),
    BucketKind.Month => new DateTime(value.Year, value.Month, 1, 0, 0, 0, value.Kind),
    _ => value.Date,
};

private static DateTime StartOfWeek(DateTime value)
{
    int diff = (7 + (value.DayOfWeek - DayOfWeek.Monday)) % 7;
    return new DateTime(value.Year, value.Month, value.Day, 0, 0, 0, value.Kind).AddDays(-diff);
}

```

Лістинг – Services/Auth/EmailConfirmationPasswordUpdaterService.cs:

```

using Microsoft.AspNetCore.Identity;
using Quizanchos.Common.Util;
using Quizanchos.Domain.Entities;
using Quizanchos.WebApi.Services;
using Quizanchos.WebApi.Util;
using Quizanchos.WebApi.Validation;

namespace Quizanchos.WebApi.Services.Auth;

public class EmailConfirmationPasswordUpdaterService : IUserPasswordUpdaterService
{
    public class UserData
    {
        public required string UserId { get; set; }
        public required string Email { get; set; }
        public required string Token { get; set; }
        public DateTime RequestedTime { get; set; }
    }

    private const int CodeLength = 6;
    private static readonly TimeSpan CodeLifetime = TimeSpan.FromHours(1);

    private readonly EmailSenderService _emailSenderService;
    private readonly UserManager<ApplicationUser> _userManager;
    private readonly ContainerService _containerService;

    public EmailConfirmationPasswordUpdaterService(EmailSenderService emailSenderService,
        UserManager<ApplicationUser> userManager, ContainerService containerService)
    {
        _emailSenderService = emailSenderService;
        _containerService = containerService;
        _userManager = userManager;
    }

    public async Task RequestPasswordResetAsync(string email)
    {
        // Always succeed silently: do not leak whether an account exists for this email.
        ApplicationUser? user = await _userManager.FindByEmailAsync(email);
        if (user is null)
        {
            return;
        }

        string code = StringGenerator.GenerateRandomString(_containerService.Random, CodeLength);
        string token = await _userManager.GeneratePasswordResetTokenAsync(user);

        UserData userData = new()
        {
            UserId = user.Id,
            Email = user.Email!,
            Token = token,
            RequestedTime = DateTime.UtcNow
        };

        lock (_containerService.PendingPasswordDictionary)
        {
            _containerService.PendingPasswordDictionary[code] = userData;
        }

        string title = "Reset your password";
        string content = $"Your password reset code: {code} (valid for 1 hour). If you did not request this, ignore this email.";
    }

```

```

        await _emailSenderService.SendEmailAsync(user.Email!, title, content);
    }

    public async Task ConfirmPasswordResetAsync(string email, string code, string newPassword)
    {
        PasswordPolicy.Validate(newPassword, "New password");

        UserData? userData;
        lock (_containerService.PendingPasswordDictionary)
        {
            if (!_containerService.PendingPasswordDictionary.TryGetValue(code, out userData))
            {
                throw HandledExceptionFactory.Create("Invalid or expired reset code");
            }
            _containerService.PendingPasswordDictionary.Remove(code);
        }

        if (DateTime.UtcNow - userData.RequestedTime > CodeLifetime)
        {
            throw HandledExceptionFactory.Create("Invalid or expired reset code");
        }

        if (!string.Equals(userData.Email, email, StringComparison.OrdinalIgnoreCase))
        {
            throw HandledExceptionFactory.Create("Invalid or expired reset code");
        }

        ApplicationUser? user = await _userManager.FindByIdAsync(userData.UserId);
        _ = user ?? throw HandledExceptionFactory.Create("Invalid or expired reset code");

        IdentityResult result = await _userManager.ResetPasswordAsync(user, userData.Token, newPassword);
        if (!result.Succeeded)
        {
            throw HandledExceptionFactory.Create(string.Concat(result.Errors.Select(e => e.Description)));
        }
    }
}

```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2026 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ОНЛАЙН ПЛАТФОРМИ МІНІГОР З
ПІДТРИМКОЮ МУЛЬТИПЛЕЄРНОЇ ВЗАЄМОДІЇ ТА СИСТЕМИ
ВІРТУАЛЬНИХ АКТИВІВ**

Програма та методика тестування

КПІ.ПІ-2227.045440.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

_____ Максим

ГОЛОВЧЕНКО

Виконавець:

_____ Іван ХИЛЯ

Київ – 2026

ЗМІСТ

1	ОБ'ЄКТ ВИПРОБУВАНЬ.....	3
2	МЕТА ТЕСТУВАННЯ	4
3	МЕТОДИ ТЕСТУВАННЯ.....	6
4	ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ.....	7

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробувань є сервіси індивідуальної частини веб-платформи мініігор Quizanchos – GameLogic, Rooms, Market та Users, реалізовані у складі застосунку Quizanchos.WebApi. Сервіс GameLogic відповідає за виконання ігрової логіки сесії та маршрутизацію викликів до реалізацій мініігор, Rooms – за життєвий цикл мультиплеєрних кімнат, Market – за каталог віртуальних активів і транзакційну купівлю, а Users – за облік користувачів, ролі, рейтинг, перевірку преміум-доступу та агреговану статистику адміністративної панелі.

Програмний код сервісів написано мовою C# 14 та виконується середовищем .NET у складі вебзастосунку, побудованого з використанням фреймворку ASP.NET Core 10. Роль сховища відіграє PostgreSQL, звернення до якої відбуваються через ORM Entity Framework Core 10. Розгортання серверної частини підтримується як в ОС Windows, так і в Linux, а інтерфейс користувача функціонує у вебпереглядачі на рушії Chromium. Автоматизовані перевірки зосереджено в окремому проєкті Quizanchos.WebApi.Tests, тоді як поведінку з боку користувача контролюють ручним проходженням сценаріїв у браузері.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є підтвердження правильності реалізації сервісів індивідуальної частини та їх відповідності функціональним вимогам FR-1 – FR-30, сформованим у другому розділі пояснювальної записки, а також виявлення невідповідностей фактичної поведінки очікуваній. Тестування має досягти таких цілей:

- перевірка повного життєвого циклу мультиплеєрної кімнати: створення кімнати з додаванням хоста до команди, приєднання учасників, контролю переповнення команд та строку очікування, автоматичного запуску гри при заповненні останнього місця й закриття кімнати при виході хоста;
- перевірка цілісності операцій внутрішньоігрового маркету: правил сортування каталогу, списання внутрішньоігрової валюти, створення запису про володіння активом, запобігання повторній купівлі звичайного предмета та коректного подовження преміум-підписки, у тому числі додавання строку до вже активної підписки;
- перевірка формування таблиці лідерів: сортування за сумою очок з усіх мініігор та за обраною мінігрою, виключення користувачів з роллю Owner і валідації параметрів пагінації;
- перевірка нарахування рейтингових очок: створення та накопичення записів у таблиці результатів, ігнорування нульових і від’ємних значень та підрахунку сумарного результату користувача за всіма мінігровими типами;
- перевірка контролю преміум-доступу до відповідних мініігор для одного та для групи користувачів, у тому числі коректної реакції на прострочену підписку;
- перевірка правил роботи з обліковим записом: обмеження розміру та MIME-типу файлу аватара, унікальності нікнейму, заборони від’ємних поповнень балансу та неможливості видалення акаунта за наявності історії поповнень;

- перевірка коректності агрегованих метрик адміністративної панелі та механізму одноразового коду відновлення пароля;
- виявлення дефектів і помилок реалізації з метою їх усунення до моменту впровадження.

3 МЕТОДИ ТЕСТУВАННЯ

Перевірку сервісів індивідуальної частини побудовано на двох рівнях: автоматизованому модульному тестуванні серверної бізнес-логіки на базі xUnit.net та мануальній перевірці відповідних користувацьких сценаріїв у браузері. Для тестування програмного забезпечення використовуються такі методи:

- Динамічне тестування передбачає перевірку фактичної поведінки методів сервісів у процесі виконання шляхом їх безпосереднього виклику з контрольованими вхідними даними та подальшим порівнянням отриманих результатів з очікуваними.
- Функціональне тестування використовується для перевірки відповідності поведінки сервісів функціональним вимогам FR-1–FR-30 шляхом проходження сценаріїв реєстрації, авторизації та відновлення пароля, керування профілем користувача, створення та наповнення мультиплеєрних кімнат, виконання операцій з віртуальними активами та перегляду рейтингової інформації.
- Мануальне тестування полягає у виконанні тест-кейсів вручну через браузер без використання засобів автоматизації та застосовується для перевірки наскрізних користувацьких сценаріїв клієнтської частини, що взаємодіють із сервісами Users, Rooms, Market та модулем рейтингу.
- Тестування методом «білої скриньки» використовується для модульної перевірки внутрішньої поведінки методів класів GameRoomService, MarketService, LeaderBoardService, PremiumAccessService, UserScoreService, UserProfileService, StatisticsService та EmailConfirmationPasswordUpdateService із прямим доступом до їхнього стану та залежностей шляхом інстанціювання об'єктів у тестах xUnit.net.
- Тестування методом «чорної скриньки» передбачає мануальну перевірку користувацьких сценаріїв виключно на основі аналізу вхідних і вихідних даних без урахування внутрішньої реалізації відповідних компонентів.

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Під час проведення тестування будуть використовуватись наступні допоміжні засоби:

Таблиця 4.1 – Перелік допоміжних засобів тестування

Засіб	Призначення
.NET 10 SDK (dotnet test)	Збирання тестового проєкту та запуск автоматизованих модульних тестів з командного рядка
xUnit.net 2.9	Фреймворк модульного тестування; організація тест-класів і перевірок за шаблоном Arrange–Act–Assert
Moq 4.20	Створення заглушок (mock-об'єктів) залежностей сервісів та перевірка викликів через Verify
Microsoft.EntityFrameworkCore.InMemory	Емуляція контексту QuizanchosDbContext для детермінованої перевірки EF-запитів без розгортання PostgreSQL
Visual Studio 2022	Середовище розробки, редагування та покрокового налагодження тестів

Порядок проведення тестування є наступним:

- автоматизоване модульне тестування сервісів індивідуальної частини у проєкті Quizanchos.WebApi.Tests фреймворком xUnit.net 2.9 за шаблоном Arrange–Act–Assert; залежності сервісів (IGameRoomManager, IRoomNotifier, IMarketItemRepository, IUserOwnedItemRepository, IUserMinigameScoreRepository, IMinigameRegistry та інші) підмінюються заглушками Moq 4.20, а доступ до бази даних емулюється провайдером Microsoft.EntityFrameworkCore.InMemory без піднімання Postgres-контейнера;
- перевірка помилкових сценаріїв через очікування типізованого виключення Assert.ThrowsAsync<HandledException> (неіснуючий предмет

маркету, недостатній баланс, перевищення розміру аватара, інвертований діапазон статистики, прострочений код відновлення пароля тощо);

- запуск усього набору з 36 модульних тестів командою «dotnet test Quizanchos.WebApi.Tests» з кореневого каталогу рішення та контроль того, що всі тести проходять успішно, без помилок і пропущених тестів;

- мануальне функціональне тестування наскрізних користувацьких сценаріїв у браузері Chromium 132: реєстрація з підтвердженням пошти, авторизація з видачею cookie QAuth і відновлення пароля, керування профілем (аватар, нікнейм, баланс, експорт та видалення персональних даних), створення мультиплеєрної кімнати, приєднання до команди й автоматичний запуск гри, придбання активів і преміум-підписки на маркеті, перегляд таблиці лідерів з фільтром за мінігрою;

- звіряння фактичного результату кожного мануального тесту з очікуваним та фіксація результатів у відповідних тест-таблицях пояснювальної записки.

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ” _____ 2026 р.

**ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ОНЛАЙН ПЛАТФОРМИ МІНІГОР З
ПІДТРИМКОЮ МУЛЬТИПЛЕЄРНОЇ ВЗАЄМОДІЇ ТА СИСТЕМИ
ВІРТУАЛЬНИХ АКТИВІВ**

Графічний матеріал

КПІ.ПІ-2227.045440.06.99

“ПОГОДЖЕНО”

Керівник проекту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

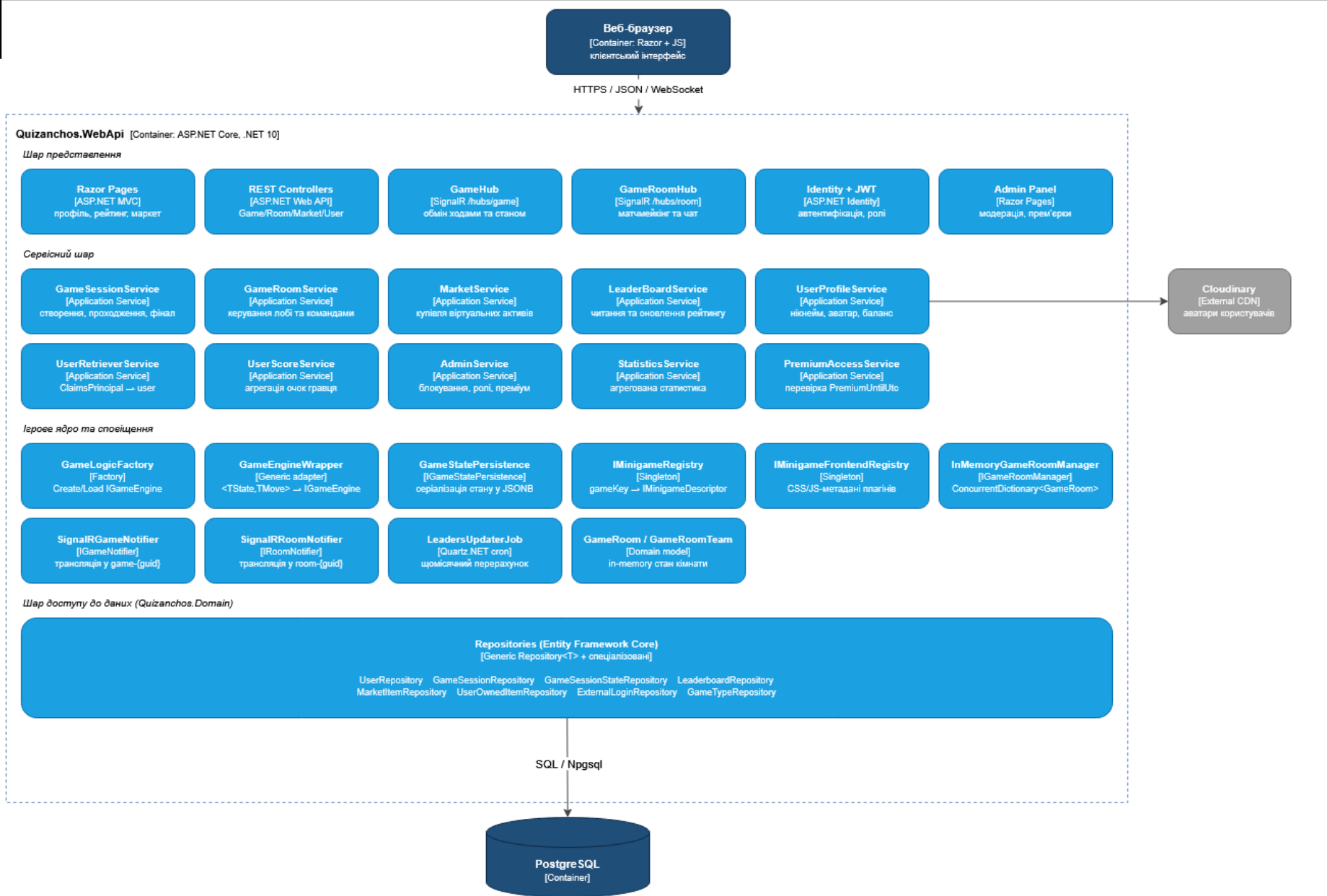
_____ Максим

ГОЛОВЧЕНКО

Виконавець:

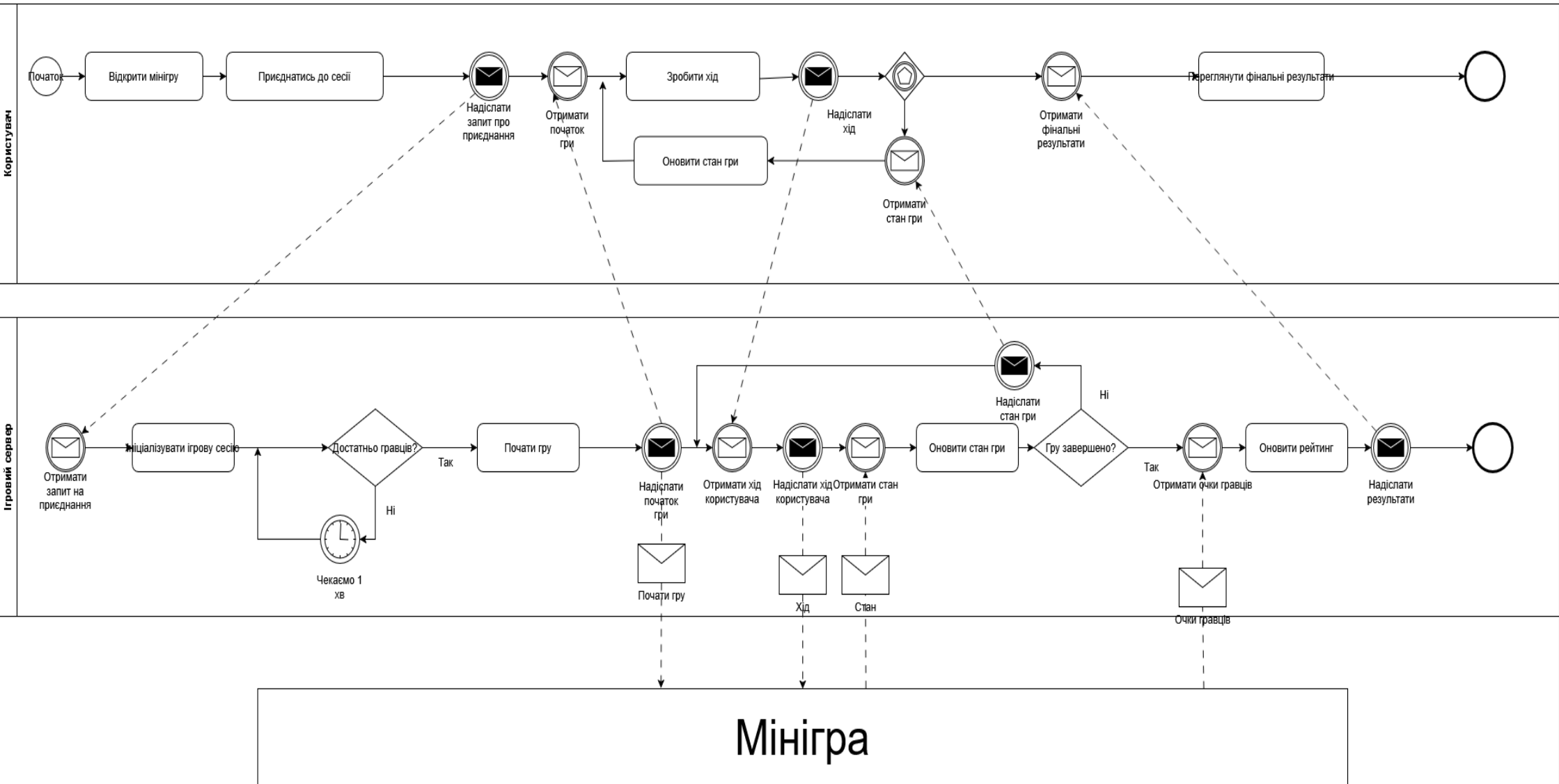
_____ Іван ХИЛЯ

Київ – 2026



					КПІ.ІП-2227.045440.06.99.CCM							
					Схема структурна компонентів програмного забезпечення	Літера			Маса		Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата								
Розробив	Хиля І.В.											
Перевірив	Павлов О.А.											
Т. контр.						Аркуш			Аркушів			
					Програмне забезпечення онлайн платформи мінігор з підтримкою мультиплеерної взаємодії та системи віртуальних активів(комплексна тема)	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-22						
Н. контр.	Головченко М.М.											
Затвердив	Жаріков Е.В.											

Схема бізнес-процесу проходження мінігри



Демонстраційний плакат до дипломного проекту
 Програмне забезпечення онлайн платформи мінігор з підтримкою мультиплеерної взаємодії та системи віртуальних активів(комплексна тема)
 Виконав студент гр. ІП-22 Хиля І.В.
 Керівник Павлов О.А.